



Series 3000



Application Programmer's Guide



Series 3000 Application Programmer's Guide



70-16308-03
Revision A — April 2000

Series 3000
Application Programmer's Guide

70-16308-03

Revision A

April, 2000



© 1996-2000 by Symbol Technologies, Inc. All rights reserved.

No part of this publication may be reproduced or used in any form, or by any electrical or mechanical means, without permission in writing from Symbol. This includes electronic or mechanical means, such as photocopying, recording, or information storage and retrieval systems. The material in this manual is subject to change without notice.

The software is provided strictly on an “as is” basis. All software, including firmware, furnished to the user is on a licensed basis. Symbol grants to the user a non-transferable and non-exclusive license to use each software or firmware program delivered hereunder (licensed program). Except as noted below, such license may not be assigned, sublicensed, or otherwise transferred by the user without prior written consent of Symbol. No right to copy a licensed program in whole or in part is granted, except as permitted under copyright law. The user shall not modify, merge, or incorporate any form or portion of a licensed program with other program material, create a derivative work from a licensed program, or use a licensed program in a network without written permission from Symbol. The user agrees to maintain Symbol’s copyright notice on the licensed programs delivered hereunder, and to include the same on any authorized copies it makes, in whole or in part. The user agrees not to decompile, disassemble, decode, or reverse engineer any licensed program delivered to the user or any portion thereof.

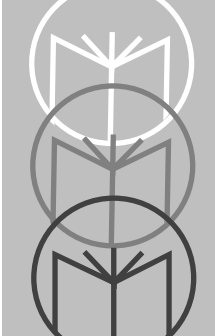
Symbol reserves the right to make changes to any software or product to improve reliability, function, or design.

Symbol does not assume any product liability arising out of, or in connection with, the application or use of any product, circuit, or application described herein.

No license is granted, either expressly or by implication, estoppel, or otherwise under any Symbol Technologies, Inc., intellectual property rights. An implied license only exists for equipment, circuits, and subsystems contained in Symbol products.

Symbol and Spectrum One are registered trademarks of Symbol Technologies, Inc. Other product names mentioned in this manual may be trademarks or registered trademarks of their respective companies and are hereby acknowledged.

Symbol Technologies, Inc.
One Symbol Plaza
Holtsville, NY 11742-1300
<http://www.symbol.com>



Contents

Chapter. About This Manual

Introduction and Overview	vii
Chapter Descriptions	ix
Notational Conventions	xi
Sample Programs	xii
Related Documentation	xii

Chapter 1. Installing the ADK

Introduction	1-3
System Requirements	1-4
Installing the ADK Software	1-5
Modifying C Libraries	1-8
Configuring the PC Display	1-9
Connecting a Terminal to Your Development System	1-10

Chapter 2. Programming a Series 3000 Terminal

Introduction	2-3
Creating and Downloading a Program	2-4
The Terminal Operating Environment	2-8
Moving On	2-14

Chapter 3. Terminal Environment

Introduction	3-3
Hardware Environment	3-3
Power Management	3-10
PC BIOS Compatibility	3-11

Chapter 4. NVM Configuration

Introduction	4-3
System Memory Structure	4-5
Basic NVM Configurations	4-7
Storing Programs	4-9
Building an NVM Image	4-10
Sample USRCFG Response Files	4-20
NVM Image Size	4-23

Chapter 5. Terminal Initialization

Introduction	5-3
Running BLDINIT	5-4
Font Table Selection	5-8
Files Linked with BLDINIT	5-8

Chapter 6. Managing Files

Introduction	6-3
UBASIC File Manager Features	6-4
Using the File Manager	6-8
Loading the File Manager	6-10
DOS File Management APIs	6-13
Microsoft Visual C Runtime Functions	6-14
Sample Programs	6-16

Chapter 7. Managing Memory

Introduction	7-3
Program Memory Requirements	7-4
Maximizing Memory Use	7-6
Handling Low-Memory Situations	7-8
Managing Available Memory	7-9
EMS	7-13
Sample Programs	7-16

Chapter 8. Display Handling

Introduction	8-3
Improving Display Performance	8-12
Cursor Control	8-16
Miscellaneous Screen Operations	8-18
ANSI Support	8-19
Display Fonts	8-21
Font Builder	8-22
Making a Font TSR	8-31

Chapter 9. Keyboard Processing

Introduction	9-3
Keyboard Operation	9-4
Spanish Language Support on Series 3000	9-12
Processing Key Input	9-14
Using DOS I/O Control Functions	9-21
Conserving Power	9-23

Using the Keyboard Mapping Utility	9-28
Using a Translation Table	9-36
Sample Programs	9-37

Chapter 10. Scanning

Introduction	10-3
Changes in This Release	10-3
The Scanning Interface	10-6
Loading SCAN3000	10-9
Loading SCAN3500	10-11
Adding Scanning to an Application	10-12
BLDSCAN.EXE	10-14
Parameter Descriptions	10-32
Parameter Menu Scanning	10-39
Modifying the Scanner, Reader, and Decoders	10-77
Creating the Initialization File	10-88
Updating the Scanner, Readers, and Decoders	10-89

Chapter 11. Data Communications

Introduction	11-3
Using the URM API	11-12
Using the DR DOS API	11-30
Using the BIOS API	11-34
Sample Communication Programs	11-36
Managing Terminal Resources	11-41
Unattended Communications	11-43
Communicating Through a Cradle	11-44
Communication Problems	11-50
Remote PC Communication Software	11-54

Chapter 12. Writing Resident Programs

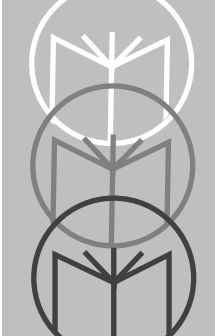
Introduction	12-3
Types of Resident Programs	12-5
Separating the Data from the Code	12-8
Calling a Resident Program	12-9
Memory Models for Resident Programs	12-10
Writing Resident Programs	12-11
The Resident Library Builder	12-14

Appendix A. Monarch Printer Drivers

Appendix Contents	A-1
Introduction	A-3

Installing the Monarch 9490 Printer Driver	A-3
Installing the Monarch 9450 Printer Driver	A-4

Index



About This Manual

Introduction and Overview

The *Series 3000 Application Programmer's Guide* is written for programmers who are using the Application Development Kit (ADK) to create applications for Symbol Technologies Series' Series 3000 terminals.

The ADK constitutes the primary application development environment provided by Symbol Technologies and consists of a large number and wide variety of programming tools and resources. The most important tool is a series of C libraries that provide a variety of application programming interfaces (APIs). The ADK supports Microsoft Visual C++ 1.5 and 1.52 as the development language. This manual assumes that you are using Visual C++ as your primary development vehicle and are already thoroughly familiar with it. It also assumes that you are familiar with programming in a PC environment and have a good knowledge of DOS operations.

This application programmer's guide describes the use of ADK tools to write applications for Series 3000 terminals. Its focus is on those aspects of programming Series 3000 terminals that are different from programming a typical PC. It is *not* a standalone document, however; it must be used in conjunction with three other documents supplied with the ADK. These other documents provide detailed descriptions of libraries, utilities, APIs, etc. referred to in this guide. They should be kept readily available for easy reference when you are using this manual to write applications.

The three reference manuals provided with the ADK for use with this programmer's guide are:

- *Series 3000 Application Programmer's Reference Manual*
This manual describes C language interface libraries included in the ADK. Of these the largest libraries are the BIOS and DOS function libraries. By



using the functions in these libraries, the programmer has C language access to the features of the Series 3000 terminals necessary to integrate the terminal into the target application. These central features include bar code scanning and communications, including Spectrum One[®] radio communications. Additional libraries allow for further enhancements, such as graphics display capabilities and custom keyboard definitions.

In addition to the libraries, this reference includes descriptions of a number of utilities also included in the ADK. Some utilities are special purpose programs that can be included in the terminal environment, usually running as TSRs (Terminate and Stay Resident programs). Others are programming tools necessary to build files or programs to run on the Series 3000 terminal.

It also describes a modem command set that includes commands supported by the IM3, IM4, IM5, IM6 and IM7 internal modems.

- *Series 3000 Application Developer's Library*
This library routines described in this reference facilitate the development of 'C' programs to execute on the Symbol Technologies Series 3000 terminals. These routines are supplied as a supplement to ADK libraries and to Microsoft 'C' libraries and make calls to those libraries.

The functions supplied cover many aspects of typical hand-held computer applications, including file data management, keyboard entry, scanning, screen output, sound, data manipulation, communications, program update and memory status.

- *Series 3000 System Software Manual*
This reference manual provides detailed information on low level utilities that may be required for programming Series 3000 terminals at system level. It will be useful to the application programmer or software engineer who, while writing applications for Series 3000 terminals, must use low-level function calls to:
 - develop or modify higher level structures
 - access special features of the Series 3000 terminal that are available only through these low level functions

Chapter Descriptions

The following brief chapter descriptions will help you focus on your area of interest:

Chapter 1, Installing the ADK, describes the installation procedure for the ADK, system requirements, and set-up options.

Chapter 2, Programming a Series 3000 Terminal, introduces the processes involved in writing a C program for a Series 3000 terminal. It deals primarily with the following three processes:

1. Writing and compiling an application program
2. Building and downloading an operating environment into the terminal NVM (Non-Volatile Memory)
3. Downloading the program to the terminal

Chapter 3, Terminal Environment, lists all of the models of Series 3000 hand-held computers and describes the features associated with and the options available on each model.

Chapter 4, NVM Configuration, describes different configuration options available on Series 3000 terminals.

Chapter 5, Terminal Initialization, describes the Terminal Initialization Tool (BLDINIT.EXE), which builds the INIT.EXE initialization program for Series 3000 terminals.

Chapter 6, Managing Files, focuses primarily on Symbol's UBASIC File Management Resource for Series 3000 terminals. This file manager provides high level file storage and access methods for fast and efficient file management. (See also the chapter on the *UBASIC Record Manager* in the *Series 3000 Application Programmer's Reference Manual*, which is supplied with the ADK.)

Chapter 7, Managing Memory, describes a number of methods for managing how your applications use memory. It also some ways of solving memory limitations.



Chapter 8, Display Handling, describes a number of BIOS tools that have been provided for managing displays on Series 3000 terminals, especially the use of physical, virtual, and logical display screens.

Chapter 9, Keyboard Processing, discusses the processing of both keyboard and scanned input by the console driver. The new Keyboard Mapper utility—which provides a graphical interface to allow for quick and intuitive application keyboard design—is just one of the topics discussed in this chapter.

Chapter 10, Scanning, provides information on the wide range of bar code scanning input devices and symbologies which are supported by the Series 3000 terminals. (See also the *Console/Scanner Device Driver* chapter in the Series 3000 System Software Manual, which is supplied with the ADK.)

Chapter 11, Data Communications, addresses the issues associated with direct connect and modem data communications via Series 3000 terminals.

Chapter 12, Writing Resident Programs, explains how to build both NVM (non-volatile memory) and split resident programs, which can be used in developing library files. This chapter also includes instructions on building libraries.

Appendix A, Monarch Printer Drivers, explains how to install the Monarch 9450 and 9490 Printer Drivers.

Notational Conventions

The following conventions are used in this manual:

- "Operator" and "User" refer to anyone using an application on a Series 3000 hand-held computer.
- "PC" refers to the IBM personal computer or compatible system that you are using to develop applications.
- "Terminal" refers to a Series 3000 hand-held computer.
- "You" refers to the programmer who is using this manual as an aid in creating applications for Series 3000 terminals.
- Keystrokes in **bold** type within angle brackets indicate keystrokes on the development PC or on a Series 3000 terminal. For example:

Select the <F1> key on the terminal to access on-line help.

- **Bold** type is used:
 - for the names of services, functions, and commands
 - to identify menu items and input or text fields on a terminal screen
 - to display command lines

Note: Program command names and required parameters appear **without** brackets; optional parameters appear in square [] brackets.
- *Italics* are used:
 - for the names of parameters in function prototypes and variable names in usage and syntax descriptions
 - to highlight specific items in the general text
 - to identify chapters and sections in this and related documents
- Square brackets [] in a command line enclose optional command-line parameters.
- The piping symbol | has the effect of "or" when it is used to separate command-line parameters on a command line; i.e., it separates alternative values for parameters.



- Bullets (•) indicate:
 - action items
 - lists of alternatives
 - lists of required steps that are not necessarily sequential
- Sequential lists (e.g., those that describe step-by-step procedures) appear as numbered lists.

Sample Programs

Sample programs for this manual are provided as a hypertext document in the diskette supplied with it. This hypertext document provides linked access to the sample programs, which are standard ASCII DOS files. You can access these files directly for use in your own application programs.

To view the hypertext samples document enter:

```
cd\3000\sample
samples (samples.bat)
```

Related Documentation

The following is a list of documents and publications that you will find useful if you want to know more about Series 3000 terminals or about the tools and utilities that are available for writing applications for them.

Documents Available from Symbol Technologies:

- *Series 3000 Application Programmer's Reference Manual (70-16309-xx)*

This manual provides complete reference information about the libraries and utilities included in the ADK.

- *Series 3000 System Software Manual (70-16310-xx)*

This manual provides low-level reference information for the Series 3000 terminals. The information is generally intended for the system programmer who needs to know what lies behind the functions provided in the C interface libraries.

- *Series 3000 Application Developer's Library (70-16311-xx)*

This manual contains routines designed to enable application programmers to develop 'C' programs for Series 3000 terminals quickly and efficiently.

- *Spectrum One Development System Application Programmer's Guide (59044-00-92)*

This manual describes how to include Spectrum One radio communications in Series 3000 applications.

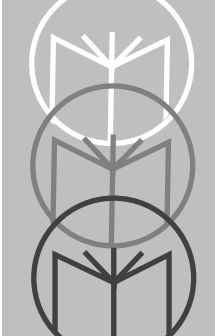
- *Series 3100/3500 Product Reference Guide (70-16645-xx)*
- *Series 3300 System Administration Manual (59040-00-90)*
- *Series 3800 Product Reference Guide (70-32230-xx)*
- *Series 3900 System Administration Manual (61068-00-90)*
- *PDT 6100 Product Reference Guide (70-33222-xx)*
- *PDT 6800 Product Reference Guide (70-32645-xx)*

These manuals describe a number of procedures you need to be familiar with for programming Series 3000 terminals.

External Documents and Publications:

- *The Peter Norton PC Programmer's Bible, Third Edition*, Peter Norton, Peter Aiken, Richard Wilton, 1993, Microsoft Press
- *PC Programmer's Guide to Low-Level Functions and Interrupts*, Marcus Johnson, 1994, Sams Publishing
- *MS-DOS Programmer's Reference*, 1993, Microsoft Press
- *The Bar Code Book, Second Edition*, 1991, by Roger C. Palmer, Helmers Publishing, Inc., 174 Concord Street, Peterborough, NH.





Chapter 1

Installing the ADK

Chapter Contents

Introduction	1-3
System Requirements	1-4
Installing the ADK Software	1-5
Modifying C Libraries	1-8
Configuring the PC Display	1-9
Connecting a Terminal to Your Development System	1-10



Introduction

The Application Development Kit (ADK) software comes on a set of diskettes. This software must be installed on the hard disk of your development PC before you can use it.

This chapter describes the procedure for installing procedure the ADK software and a few additional setup options. Installation is performed by the **INSTALL** program supplied on ADK "Disk 1" and is for the most part automatic.

In addition to the ADK software, you must also have installed, on your development PC, Microsoft Visual C++, Professional edition, version 1.5.

Note: As of January, 1995, Microsoft Visual C++ version 1.5 is available as an installable component of the version 2.0 package. Version 1.5 of this software is no longer available as a stand-alone package.

Future ADK Updates

Look for future software releases on Symbol Technologies' worldwide web site, <http://www.symbol.com>.



System Requirements

This section lists the minimum and recommended hardware setup for developing applications that run on a Series 3000 terminal.

Hardware

Your development system must meet the following minimum requirements:

- IBM Personal Computer or 100% compatible
- Hard disk with at least 20 MB available
- 1.44MB diskette drive
- VGA Display monitor
- 640K main memory (RAM)
- DOS Version 3.1 or later
- Serial port

For testing your program, you also need the following:

- One or more Symbol Technologies Series 3000 terminals
- Optical cradle for Series 3000 terminals with optical comm ports (optional)
- Null modem cable
- Auxiliary power supply

Software

For writing applications for Series 3000 terminals:

- The Microsoft Visual C++ 1.5 development system should be installed on your development PC.

Note: It is important that you use the Microsoft Visual C++ **Professional** edition, and not just Microsoft Visual C++. The latter does *not* support the development of DOS applications.

- It is recommended that you install the Borland's Turbo Debugger.

Make sure that the directories containing these software packages are included in the PATH statement in your AUTOEXEC.BAT.

Installing the ADK Software

Symbol Technologies supplies the ADK software on a set of disks, which contain the following:

- an Application Development Kit, labelled “APDVn”
- a Scanning Kit, labelled “SCANKIT”
- Hypertext Samples
- an Application Developer’s Library
- an Acoustic Coupler Developer’s Kit
- a Microsoft Visual C++ Kit

Back Up Your Disks

Before installing the ADK, use the DOS **DISKCOPY** utility to make working copies of the ADK disks as follows:

```
DISKCOPY A: A:
```

assuming that A: is the disk drive you are using.

Store the originals in a safe place and use the copies to install the software. This will protect your original diskettes from accidental damage during installation.

Install the ADK

To install the ADK files on your development PC's hard disk, insert the disk labeled “Disk 1” into the appropriate drive, usually A or B (we’ll use A for illustration purposes). At the DOS prompt, enter:

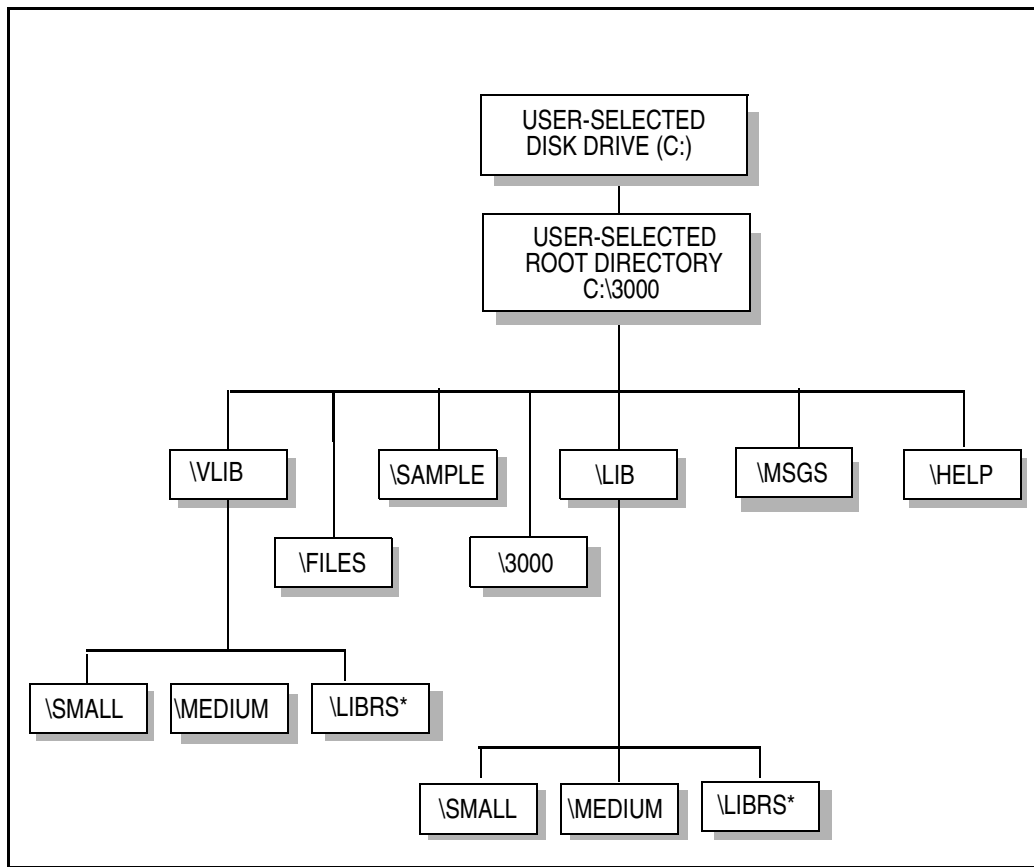
```
A: INSTALL
```

The **INSTALL** program leads you through the installation process and copies all necessary files from the ADK disks to your hard disk.

The installation program copies the ADK files to the directories shown in Figure 1-1.



The installation completes by displaying a final review screen to remind you to modify your files and to reboot.



* Created only after executing modallv.bat or modall.bat

Figure 1-1. How INSTALL Organizes ADK Directories on the Development PC

Update AUTOEXEC.BAT

The AUTOEXEC.BAT file on your development PC must be updated to define environment variables that tell the runtime routines where to find various files.

Your AUTOEXEC.BAT file should include the following:

```
PATH=C:\3000;C:\msvc\bin;C:\3000\msgs
SET LIB=C:\3000\VLIB;C:\3000\lib;C:\msvc\lib;C:\msvc\mfc\lib
SET INCLUDE=C:\3000;C:\msvc\include;C:\msvc\mfc\include
SET DPATH=C:\3000\msgs
SET EXE3000=C:\3000
SET FILES3000=C:\3000\files
SET CLIB=C:\msvc\lib
```

The Scan Kit also requires the following additions to AUTOEXEC.BAT:

```
SET 3000INC=C:\3000\3000
SET LIB3000=C:\3000\vlb
```

These settings assume that the Microsoft Visual C++ libraries are in the subdirectories under C:\msvc and that the ADK is installed in its default directory. If your configuration is different, then the settings above must be changed accordingly.

Update CONFIG.SYS

Your CONFIG.SYS file must set the sizes of **FILES** and **BUFFERS** to at least the following sizes:

```
FILES=20
BUFFERS=20
```

The installation program will verify these values for you.

Reboot the System

When the installation procedure is complete, you must reboot the development PC so that the changes you have made to AUTOEXEC.BAT and CONFIG.SYS take effect.



These changes are required before you run MODALLV.BAT (refer to the *Running MODALLV.BAT* section, below, in this chapter).

Modifying C Libraries

You may wish to create modified versions of the Microsoft Visual C++ runtime libraries that are compatible with the Series 3000 terminals. Only the modified versions of these libraries can be downloaded to and made resident in Series 3000 terminals.

The file MODALLV.BAT is provided to create modified Microsoft Visual C++ libraries and stores them by default in the c:\3000\VLIB\LIBRS and c:\3000\LIB\LIBRS directories (see Figure 1-1). Before executing MODALLV.BAT, your computer's environment variables must have been modified as described above in the *Update AUTOEXEC.BAT* section.

The following Microsoft libraries must also be present before running the MODALLV.BAT files

SLIBC_x.LIB
MLIBC_x.LIB

where x=E, A and/or 7

Running MODALLV.BAT

To execute the MODALLV.BAT file, assuming Microsoft Visual C++ and the ADK are installed in their default directories, issue this command:

```
MODALLV.BAT C:\msvc\lib C:\3000\plib
```

This command creates the modified libraries for small and medium memory models and places the standard library files in the directory \3000\plib and the split-resident library files in the directory \3000\plib\librs.

In order for the Microsoft Linker to find these modified libraries instead of the standard libraries, make sure that the directory C:\3000\plib precedes C:\msvc\lib in your development PC's AUTOEXEC.BAT file.

Configuring the PC Display

You can change the way the colors are displayed by some of the ADK programs by using the PC Configuration Utility, MSICFG.EXE.

To run MSICFG, enter:

```
msicfg
```

The configuration menu is then displayed.

Select Video Mode

Use the Select Video Mode option to select Color Mode (the default) or Black & White Mode.

Color mode provides 16 foreground and eight background colors to choose from.

Black & White Mode provides five display options: Black, White, LightGray, DarkGray, and Underline.

Change Color Scheme

Use Change Color Scheme to select color pairs.

First, select foreground and background text colors. Select from the upper rows for Black & White Mode and from the lower rows for Color Mode.

Next, you select window colors.

Finally, you can select a border color. Note that most EGA boards will not display a border, even if you select one here.

If no text appears in the results column, you have selected the same color for the foreground and background. Change your selections until a readable contrast is obtained.



Monochrome Monitor Options

Monochrome monitors typically display only three colors: Black, Gray (normal intensity), and White (double intensity or bold). Depending on the monitor, you may be able to select LightGray, DarkGray, and/or Underline.

We recommend that you use the default video attributes for monochrome monitors. This guarantees that normal, bold, and highlighted characters match the descriptions in this manual.

Connecting a Terminal to Your Development System

Having installed Symbol Technologies' ADK and Microsoft Visual C++ 1.5, you have all the software you need to develop applications to run on Series 3000 terminals. In addition to this development software, you need to connect one or more terminals to your development system. Initially, you will use the terminal for testing and debugging the application software you develop. Ultimately, you will download the application software to the terminal for the end user.

Your test system will consist of at least one Series 3000 terminal. The terminal requires both a communications connection to your PC and an auxiliary power source. Instructions for making communications and power connections are provided in the *Quick Reference Guide* and the *System Administration Manual/Product Reference Guide* for each terminal. Refer to these documents for details.

Depending on your terminal model and optional equipment, you may or may not have a cradle—which also may be called a Charging and Communications Module (CCM). If you do have a cradle, it provides the communications connection through a null modem cable between the cradle and the PC. A power adapter in the cradle provides auxiliary power when it is plugged into a standard power outlet. To connect the terminal, place the terminal in the cradle. This setup is illustrated in Figure 1-2.

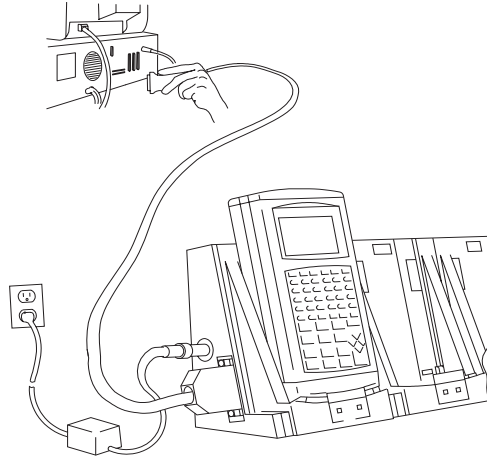


Figure 1-2. Terminal with Cradle Connected to PC

If you do not have a cradle, use a null modem cable to connect the terminal directly to the development PC. Auxiliary power is provided by connecting a power adapter module appropriate to the terminal. This setup is illustrated in Figure 1-3.

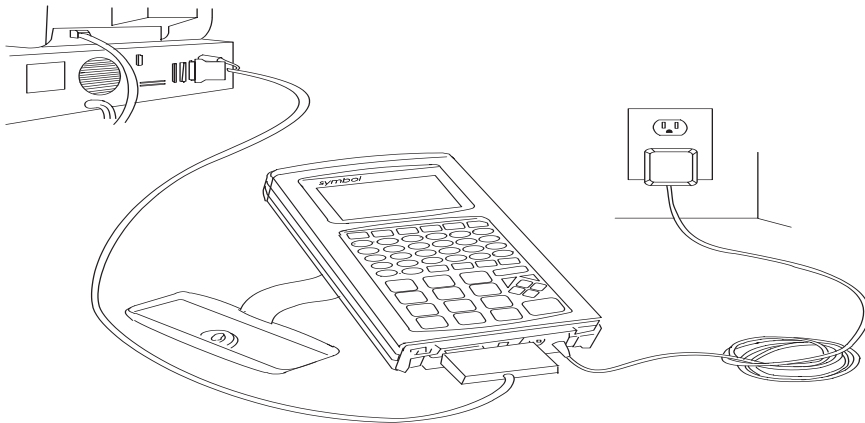
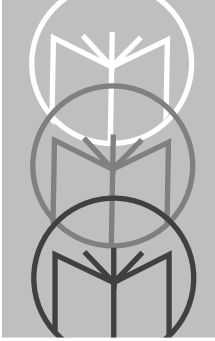


Figure 1-3. Terminal Connected Directly to PC



While Series 3000 terminals are battery powered, the batteries do not provide sufficient power for erasing and writing the NVM image (refer to *Chapter 2, Programming a Series 3000 Terminal*, in this guide for an explanation). The auxiliary power, provided by either the cradle or the power adapter module, is required for this operation.



Chapter 2

Programming a Series 3000 Terminal

Chapter Contents

Introduction	2-3
Creating and Downloading a Program	2-4
The Terminal Operating Environment	2-8
Moving On	2-14



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>BIOS Library Functions</i> <i>DR DOS</i> <i>UBASIC Record Manager</i>
<i>Series 3000 System Software Manual</i>	<i>ROM BIOS</i> <i>Expanded memory Manager</i>
<i>Series 3000 Application Developer's Library</i>	<i>Indexed File Manager</i> <i>Miscellaneous Functions</i> <i>Additional File Managers</i>

Writing a Visual C++ program for a Series 3000 terminal is similar to writing a Visual C++ program for any PC. The major differences are in the preparatory operations that must be performed on the program and in loading the program on the terminal. This chapter describes these processes.

The main tasks involved in programming a Series 3000 terminal are:

1. writing and compiling an application program
2. building and downloading an operating environment into the terminal NVM (non-volatile memory)
3. downloading the program to the terminal

If the application program is included in the NVM image, then Step 3 may be included in Step 2. If not, then the application is downloaded to the terminal RAM disk instead of to NVM. Both of these methods are described in the sections that follow in this chapter.



Creating and Downloading a Program

In this section, you will:

- compile the program QUICK.C, which prints “Hello, world”
- download it to the RAM Disk on the terminal
- execute it.

Downloading to the RAM disk is the quickest and simplest way of loading an application on a Series 3000 terminal and is the method commonly used for testing a program on the terminal.

For the following example, assume that the TDREM.EXE program is resident in the terminal, either in NVM or on the RAM disk. The default NVM image includes this program. If this is not the case, refer to NVM Configuration in this guide to build and download the default image, or use the NVM loading procedure described later in this chapter.

The following description goes through the complete process of writing, compiling, downloading, and running a sample program called QUICK.C.

Note: If you prefer, you can substitute your own program for QUICK.C.

For more detailed information on the remote debugging and downloading program (TDREM.EXE), the terminal file transfer program (TFT3000.EXE), and the symbol table converter tool (PDCONVRT.EXE) referred to in the following, see the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

1. *On the development PC*, use a text editor to create a simple Microsoft C source file. You can use your own program or the following program, which we'll call QUICK.C:

```
#include <stdio.h>
void main (void)
{
    printf("Hello, world\n");
}
```


2. Compile and link the program:

```
c:> cl quick.c
```

3. On the terminal, run TDREM.EXE. version 3.XX.

```
d:> TDREM -rs3
```

Note: TDREM version 4.0 *cannot* be used with TFT3000.EXE.

The command parameter “-rs3” sets up the terminal for a data transmission at 38K baud. The terminal then displays the following message and waits for data from the PC:

```
Waiting to connect
<Ctrl-Bksp to quit>
```

Note: To get a listing of available baud rate options, execute:

```
d:> TDREM -?
```

on the terminal provides a listing of all available baud rate options. The options displayed are:

Baud Rate Options:

```
-rs1=9600
-rs2=19200
-rs3=38400
-rs4=115000
```

4. On the development PC, run TFT3000.EXE to transmit QUICK.EXE to the terminal:

```
c:> TFT3000 -s38 -p1 p quick.exe
```

On this command line:

“-p1” indicates COM1

“-s38” indicates 38.4K baud (9600, 19200, 38400, and 115000 supported)

“p” indicates “Put (copy) to terminal”



Refer to the description of TFT3000.EXE in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual* for a complete list of command line options and arguments for this program.

Note: The value used with the “-s” option to specify 38400 bps varies for different versions of TFT3000. Refer to the documentation or on-line help for the TFT3000 command to get the correct parameter value.

The terminal displays the following series of status messages:

```
Link established
Copying "quick.exe"
5869 bytes sent
Download complete
Link Broken
```

QUICK.EXE is now on the terminal's RAM Disk (Drive D). Press **Ctrl-Bksp** on the terminal to exit TDREM.

5. Run QUICK.EXE by entering its name at the terminal command prompt:

```
D:\> quick
```

QUICK.EXE prints “**Hello, world**” and returns to the terminal command prompt.

Figure 2-1 depicts schematically the various interfaces between development compilers/debuggers and the Series 3000 terminal.

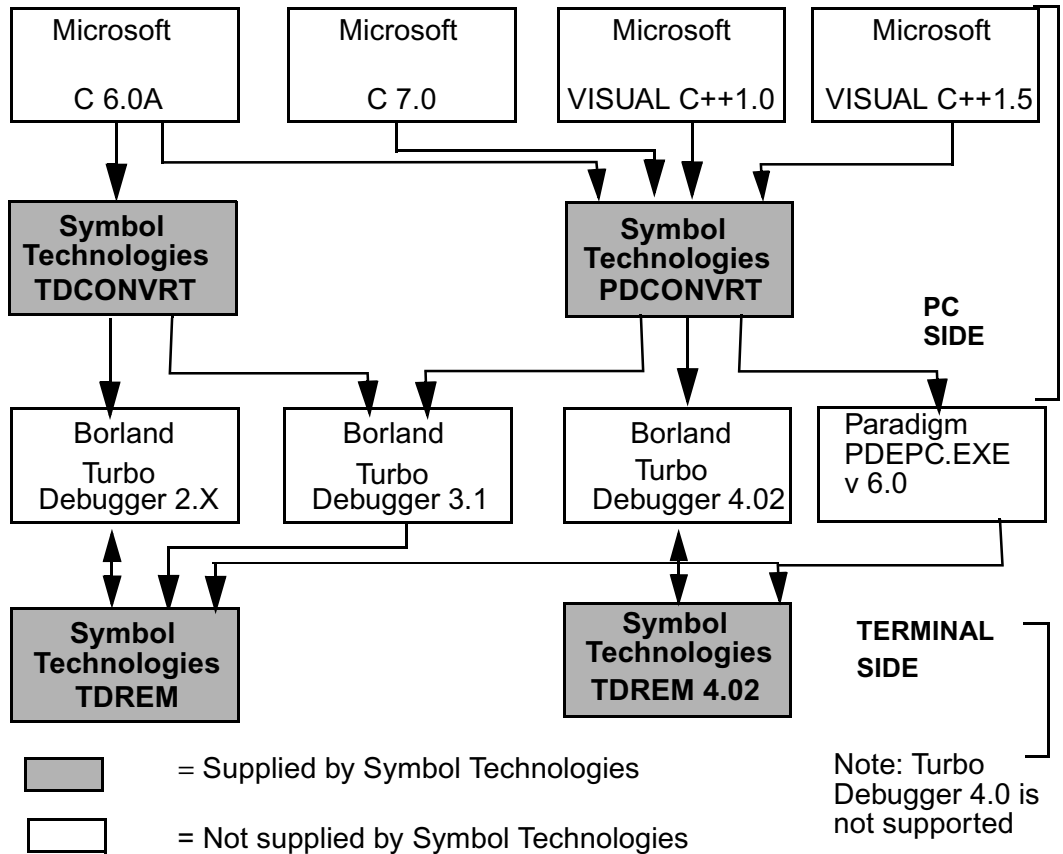


Figure 2-1. Series 3000 Terminals: C Compiler and Debuggers Map

For information on the remote debugging and downloading program (TDREM.EXE) and the symbol table converter tool (PDCONVRT.EXE) shown in Figure 2-1, refer to the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*. Note that PDCONVRT.EXE replaces TDCONVRT.EXE and is backward compatible to TDCONVRT.EXE.



The Terminal Operating Environment

The terminal operating software environment consists of two parts:

- *the operating system software contained in the system EPROM*

This consists primarily of operating system software that has been installed at the factory on an EPROM (Erasable Programmable Read Only Memory) in the terminal. It can be changed only by a hardware update.

- additional software loaded in the terminal's non-volatile memory (NVM).

Unlike the software contained in the system EPROM, this software is loadable and provides you with a lot of control over the terminal configuration.

A default NVM image is loaded into the terminal at the factory. It includes:

- DR DOS command processor (COMMAND.COM)
- AUTOEXEC.BAT
- CONFIG.SYS
- TDREM.EXE (A program for remote debugging and downloading)
Refer to the *TDREM.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual* for a detailed description of this program.

There are other files and programs included as well, but these are the only files we are concerned with in this chapter. These files are combined into a HEX image file by the User Configuration Tool (USRCFG.EXE) and then downloaded to NVM in the terminal. For a description of the User Configuration Tool, refer to the *USRCFG.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

You can also build the NVM image to include your application program. In this case, the application is loaded and run when the terminal is booted. This is advantageous especially in situations where only one application is required on a terminal. No special instructions are needed for the operator to run the application.

In the following paragraphs, we will build and download an NVM image that will run the QUICK.EXE program written in the previous section.

Create the HEX Image File

1. On the development PC, make a new directory called “\quick” and make it your current directory:

```
c:>md \quick
c:>cd \quick
```

2. Use a text editor to create the following batch file and call it “\quick\quick.bat.”

```
@echo off
path a:;b:;d:
rem The next line says there is no math coprocessor
set NO87=TRUE
rem The next line runs QUICK upon booting
b: quick.exe
```

3. Use a text editor to create the following file, and call it “\quick\quick.sys.”

```
break=off
files=20
shell=a:shell.com
```

4. Use a text editor to create the following response file and call it “\quick\quick.rsp.” You will use this file with the User Configuration Tool a little later. Each of these lines is described in *Chapter 4, NVM Configuration*, below, in this guide.

```
\3000\files\nullsys.hex
/s \3000\files\command.com
/a \quick\quick.bat
/c \quick\quick.sys
/nr
/u \quick\quick.exe
/u \3000\files\tdrem.exe
/l 00000-0000000
/h /256 /co quick.hex
```



5. Use the User Configuration Tool (USRCFG.EXE) to generate a downloadable hex file. At the command prompt enter:

```
c:>usrcfg @quick
```

For a detailed description of the User Configuration Tool, refer to the *USRCFG.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Download the NVM Image

1. Boot the terminal into Command Mode by executing the appropriate key combination from the following table:

Table 2-1. Series 3000 Terminal Boot Sequences

Terminal	Cold Boot	Warm Boot	Command Mode
3100 21-key / 6100 22-key	<UP ARROW> <4> <ENTER> <PWR>	<DOWN ARROW> <.> <PWR>	<SEND> <9> <PWR>
3100/3300/ 3800/6100/ 6800 35-key	<SPACE> <FUNC> <UP ARROW> <PWR>	</> <+> <PWR>	<BKSP> <SHIFT> <PWR>
3100/3800/ 6100/6800 46-key 3500 47-key	<A> <D> <PWR>	<4> <5> <PWR>	<F> <I> <PWR>
3300 56-key 3900/6900 54-key	<F1> <F4> <ENTER> <ON>	<SHIFT> <L> <ON>	<A> <D> <ON>
WSS 1000	<RIGHT ARROW> <ENTER> <PWR>	<F1> <ENTER> <ON>	<FUNC> <ENTER> <PWR>

The terminal displays:

COMMAND MODE

Select Function

Self Test

2. Use the Up or Down Arrow keys until Program Loader is displayed, and then press <ENTER> to select the Program Loader function.

Note: Erasing the terminal's NVM requires that the terminal be placed into the cradle and that a good power source is supplied.

When the terminal prompts you to verify erasing the NVM, press <ENTER> and wait until the NVM is erased.

You can exit the process without erasing NVM by pressing <CLEAR>.

3. Once the NVM is erased, you are prompted for communications parameters. Display the appropriate value for each parameter as you are prompted by pressing the Up or Down Arrow key. When the correct value is displayed, press <ENTER>.

The values you select must agree with the values set on the PC.
Recommended values are:

```
38400 baud
7 data bits
Odd parity
XON/XOFF
```

The terminal then displays a prompt to press <ENTER> to begin receiving data.

4. Run the SENDHEX utility on the PC to download the .HEX file into the terminal:

```
c:> sendhex quick 38400
```

This example communicates through the default COM1 PC port and sets the baud rate to 38400bps. For more detailed information on the SENDHEX



utility and its baud rate and communications port parameters, refer to the SENDHEX.EXE section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

5. Power the terminal back on if necessary and press <ENTER> to begin receiving QUICK.HEX.

Press <ENTER> when SENDHEX prompts you to begin sending data.

While the program is downloading the terminal displays:

```
Program Loader
Receiving: XXXX
```

When the program has been successfully downloaded, the terminal displays:

```
Program Loader
Status: 0000
```

The status code should show four zeros. Any other value indicates that an error occurred and that you must repeat the download. The table below lists the possible status codes.

Table 2-2. Communication Status Codes

Status Code	Meaning
0002	Receive overrun error
0004	Receive parity error
0008	Receive framing error
0010	Programming voltage not present
0020	Data Set Ready or Carrier Detect not detected on open
0040	Lost DSR while receiving
0080	ABORT key hit during comm
0100	CD lost during session
0200	Illegal Intel hexadecimal record
0600	NVM EEPROM failed to erase
0800	Receive time-out error

Table 2-2. Communication Status Codes (Continued)

Status Code	Meaning
1000	Control start character time-out
2000	Clear To Send inactive time-out error
4000	Receive buffer full

6. Power off the terminal and perform a cold boot. See Table 2-1 for boot sequences.

The terminal goes through its boot process, runs QUICK.EXE, and then displays the DRDOS prompt:

```
Hello, world
D:\>
```



Moving On

You have now seen an example of each of the two basic methods for programming Series 3000 terminals. The rest of the programming task builds on what you already know about writing application programs and on special features of the Series 3000 terminals. The remaining chapters of this manual describes these features and discusses how to include them in your applications. The following section summarizes these features and provides brief descriptions of the application programming interfaces (APIs) available for accessing them.

Application Programming Interfaces

The Series 3000 has a rich set of Application Programming Interfaces (APIs). These APIs can be grouped under the following headings:

File Function APIs

Three sets of file function APIs are provided to supplement the Microsoft C libraries:

- **Microsoft C Run-Time Library.** These are the standard Microsoft libraries, with some of the code modified for use in Series 3000 terminals.
- **DR DOS Library.** This library supplements the Microsoft library routines, providing functions necessary for accessing special functions of the Series 3000 terminals.
- **URM and File Manager Libraries.** The UBASIC Record Manager (URM) and File Manger provide UBASIC-compatible file organization. Some Series 3000 features are most easily accessed using these APIs.

For more detailed information on these file management APIs, refer to:

Chapter 6, Managing Files in this guide

The DR DOS and UBASIC Record Manager chapters in the Series 3000 Application Programmer's Reference Manual

The Indexed File Manager and Additional File Managers chapters of the Series 3000 Application Developer's Library

Memory Management APIs

In addition to the Microsoft and DR DOS libraries, the ADK provides the following:

- **EMS.** The EMM drivers provide management functions for expanded memory.
- **Memory Manager.** These functions are included in the File Manager, providing UBASIC compatible memory management.

For more detailed information on these memory management APIs, refer to:

Chapter 7, Managing Memory in this guide

The Expanded Memory Manager chapter in the *Series 3000 System Software Manual*

The Memory Management section of the *Miscellaneous Functions* chapter in the *Series 3000 Application Developer's Library*

BIOS Interface Libraries

The BIOS Interface Libraries provide 'C' interface routines for most BIOS functions in the Series 3000 terminals. This is probably the most powerful API available for creating applications. For more detailed information on these BIOS interface APIs, refer to:

The section on *Using the BIOS API* in *Chapter 11, Data Communications*, in this guide

The ROM BIOS chapter in the *Series 3000 System Software Manual*

The BIOS Library Functions chapter in the *Series 3000 Application Programmer's Reference Manual*



Soft Font Graphics Libraries

The Soft Font Graphics Libraries provide functions for graphics on Series 3000 terminals. For more detailed information on these libraries, refer to:

Chapter 8, Display Handling, in this guide

The *Font Builder* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*

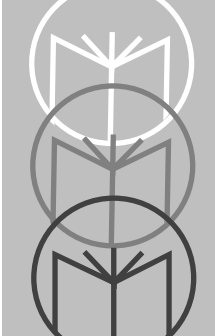
PS 10XX Interface Libraries

The PS 10xx Interface Libraries provide functions for printing labels to the symbol PS 10xx family of printers. For more detailed information on these functions, refer to:

The section on *Printer Services* of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*

PS 200 Interface Libraries

The PS 200 Interface Libraries make communication to a Symbol PS 200 printer fast and easy. The PS 200 printer is available for use as an integrated printer with the PDT 3300/3310 and as a tethered printer with PDT 3100/3110 and PDT 3300/3310.



Chapter 3

Terminal Environment

Chapter Contents

Introduction	3-3
Hardware Environment	3-3
Power Management	3-10
PC BIOS Compatibility	3-11



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>BIOS Library Functions</i> <i>Appendix A</i>
<i>Series 3000 System Software Manual</i>	<i>ROM BIOS</i> <i>Clock Device Driver</i>
<i>Series 3000 Application Developer's Library</i>	<i>NVM Programming</i>

Among Symbol Technologies' Series 3000 hand-held computers there are several models offering different features and options. When you are writing applications for Series 3000 terminals, you need to be aware of the variety of configurations available, and you should make sure your applications use only those features common to their target models.

The primary objective of this chapter is to help you understand which features and options are common and which are distinct in the different models.

Hardware Environment

Microprocessor

All Series 3000 terminals use the NEC V25 microprocessor. This microprocessor provides the following advantages:

- Low power consumption
- Machine language compatible with the Intel 80186
- Many integrated features, including:
 - Clock generator
 - Programmable timer



- Bus controller
- DMA (direct memory access) controller
- Programmable interrupt controller
- Programmable peripheral interface

Memory

All Series 3000 terminals have three general areas of memory:

- *EPROM (Erasable Programmable Read Only Memory)*

This memory area contains the default operating system and system configuration. It cannot be written to or erased in any manner; it can be modified only by a hardware change.

- *NVM (Non-Volatile Memory)*

This is an EEPROM (Electrically Erasable Programmable Read Only Memory) IC chip. It is a 128 KB or a 256KB area, depending on the model and amount installed. This memory area is used primarily to store application programs and data. From application programs, NVM is read-only memory. Writing to the NVM requires a special, off-line procedure, usually using the `SENDHEX.EXE` utility, described in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual*.

The EEPROM IC chip can be erased and reprogrammed, using the services described in the *Memory Services* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*. Refer also to the *NVM Programming* chapter of the *Series 3000 Application Developer's Library*.

- *RAM*

This memory area contains at least 640KB. The maximum amount of RAM that can be installed in a terminal varies with the model. The PDT 3300, for instance, supports up to 4.2MB. The first 640KB is conventional RAM; everything above that is EMS (Expanded Memory Specification). EMS memory is used primarily for RAM disk.



Disk Drives

Series 3000 terminals do not have physical disk drives. However, various areas of memory emulate disk drives, as follows:

- Disk drive A is emulated by the system EPROM which contains the default operating system. If a custom operating system is downloaded to the system area of NVM, this becomes drive A, and the EPROM becomes inaccessible.
- Disk drive B is emulated by the user area of NVM, if installed and configured.
- There is no drive C; it is mapped to a null device.
- Disk drive D is emulated by the RAM disk, if configured.

Application programs can write only to drive D, the RAM disk.

These drive designations are fixed by the system software and cannot be altered.

I/O Ports

PDT 3300/PRC 3310

Figure 3-1 depicts the I/O ports on the standard PDT 3300.

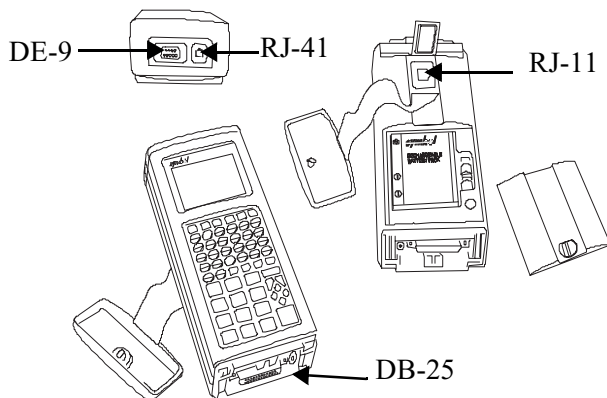


Figure 3-1. PDT 3300 I/O Ports

The standard PDT 3300/PRC 3310 has a DB-25 serial port, addressed as COM1, and a DE-9 scanner port.

An RJ-41 port providing limited RS-232 support and addressed as COM2 is provided on all current revisions of the CPU board. Older revisions of the CPU board require an optional communications board.

The optional internal modem includes a RJ-11 modem port, addressed as COM2.

An optional optical end-cap plugs into the DB-25 connector and provides an optical connection and battery charging contacts for interfacing with a cradle. Serial communications and battery charging are conducted through the cradle.

Also, an optional integrated scanner attaches to the top of the unit, eliminating access to the DE-9 and RJ-41 ports.

Refer to *Appendix A* of *The PDT 3300 System Administration Manual* for a description of the connector signals.

LRT/LDT 3800/3805/3840/PDT 6800

The LRT/LDT 3800/3805/3840/PDT 6800 has an optical interface for connecting to the cradle or the Printer Interface Module (PIM).

The PIM can interface with the PS 10xx Portable Bar Code Printers or the PC adapter.

When the terminal is installed in a cradle, all serial and modem communications and charging are conducted through the cradle.

The Symbol PC adapter contains a DE-9 connector for the PIM cable, a DB-25 connector for the PC, and a power supply port.

The PDT 3800 includes Spectrum One radio board, with the radio communications addressed as COM2. The PDT 3840 includes a Spectrum24 PCMCIA radio card.

Refer to the *Series 3800 Product Reference Guide* or *PDT 6800 Product Reference Guide* for further information.



VRC 3910/3940

The VRC 3910 includes a DB-25 serial port, addressed as COM1, and a DE-9 scanner port. It also includes the Spectrum One communications board, with the radio channel addressed as COM2.

PDT 3100/3110/3140/6100

PDT 3500/3510/3540/6800

The standard PDT 3100 has an RJ-41 serial port, addressed as COM1, and an optional DE-9 scanner port so that tethered wands and scanners can be attached. The PDT 3500 has an integrated 1-D and 2-D scanning element.

When the terminal is installed in a cradle, all serial and modem communications and charging are conducted through the cradle. The optional internal modem includes an RJ-11 port, addressed as COM2.

The PDT 3110/3510 includes a Spectrum One radio addressed as COM2 for radio communications. The PDT 3140/3540 includes a Spectrum24 PCMCIA radio card for radio communications in a Spectrum24 network.

A model which supports an integrated acoustic modem, addressed as COM2, is also available.

Refer to the *Series 3100/3500 Product Reference Guide* for further information.

Scanner

The PDT 3300 and VRC 3910 contain a DE-9 connector for a barcode scanning device, or an integrated scan module. The 3800 and 6800 series terminals contain an integrated scanner.

The PDT 3100 /3500 and PDT 6100 standard configurations contain an integrated scanner, but optionally support a DE-9 connector for tethered barcode scanning devices.

Modems

The PDT 3300 and the 3365 cradle support optional internal modems. There are four internal modem types: IM3, IM4, IM5, and IM6. The IM5 modem is replacing the IM3 and IM4, and contains all the features of both.

The PDT 3100 also supports an external acoustic modem using either the 2-way protocol or other send-only protocols. This modem is referred to as the IM7.

Spectrum One® Radio

The radio terminals (3310, 3110, 3510, 3800, 3910, 6110, and 6810) communicate over Symbol Technologies' Spectrum One™ network. For information on incorporating Spectrum One communications in your application, refer to the *Spectrum One™ Development System Application Programmer's Guide*, part number 59044-00-92.

Spectrum24® Radio

The radio terminals (3140, 3540, 3840, 3940, 6140, and 6840) communicate over Symbol Technologies' Spectrum24 network. For information on incorporating Spectrum24 communications in your application, refer to the Network Development Kit (NDK) for your product.

Real-Time Clock

A real-time clock keeps track of the date, time, and day of week. An alarm clock feature is standard.

Refer to the *Clock Device Driver* chapter in the *Series 3000 System Software Manual* for a description of the Clock Device Driver. Also consult the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* and to the *ROM BIOS* chapter in the *Series 3000 System Software Manual* for detailed descriptions of time-related functions and services.

Speaker

A wide range of sounds can be produced by the speaker. Both frequency and duration can be programmed. For detailed descriptions of sound generation services (Interrupt ADh), refer to the *Sound Generation* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*. Also refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for sound-related functions.



Display

The display screen is 8 by 20 characters on Series 3300 and Series 3800 terminals, 8 by 40 characters on the VRC 3910, either 8 by 20 or 4 by 20 characters on Series 3100 terminals, and 16 by 21 characters on the Series 3500 and Series 6800 terminals. The display font is configurable and limited graphics capability are provided as explained in *Chapter 8, Display Handling*, in this guide.

Keyboard

The Series 3000 keyboards support all key codes generated by the 101-key extended IBM PC-AT keyboard. However, the keyboard is much smaller, and the key layout is different for each Series 3000 model. You need to consider carefully how your application uses key sequences to provide ease of use to the end user. Refer to *Appendix A* of the *Series 3000 Application Programmer's Reference Manual* for a listing of the keyboard scan codes.

Power Management

Most Series 3000 terminals are battery powered during normal operation. Application programs should employ methods to increase battery life and maximize the time between charges. Methods for conserving power are mentioned throughout this manual in connection with various features. The terminal has a Power-Save operating mode that removes power from the processor during inactivity. The automatic power-down also saves power.

For additional information on Series 3000 power management options and services, refer to:

The *Conserving Power* section of the *Keyboard Processing* chapter in this guide.

The *Managing Terminal Resources* section of the *Data Communications* chapter in this guide.

The *Power Management* section of the ROM BIOS chapter in the *Series 3000 System Software Manual*.

PC BIOS Compatibility

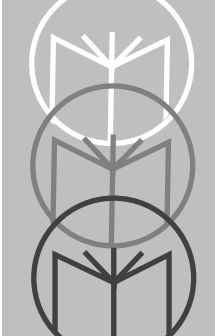
Beginning with System EPROM version 3.01, Series 3000 hand-held computers support PC compatible software interrupts. This EPROM allows software developed for the PC to run on Series 3000 terminals as long it does not make calls to PC hardware interrupts or devices not present on a Series 3000 terminal.

Terminals with pre-3.01 System EPROMs do not recognize the PC software vectors and so will not run a program using them.

As a general rule, an application program should not mix the two types of interrupts. Applications that will run on terminals with pre-3.01 EPROMs must use the Symbol proprietary interrupt numbers.

See the *PC BIOS Compatibility* section in the *ROM BIOS* chapter of the *Series 3000 System Software Manual* for detailed information on the System EPROM 3.01 interrupt services.





Chapter 4 *NVM Configuration*

Chapter Contents

Introduction	4-3
System Memory Structure	4-5
Basic NVM Configurations.....	4-7
Storing Programs	4-9
Building an NVM Image.....	4-10
Sample USRCFG Response Files.....	4-20
NVM Image Size	4-23



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i>

Series 3000 terminals are provided with DR DOS as the standard operating system and a number of default configuration files. Since DR DOS is a generic operating system, customizing the operating environment for a specific application may result in more effective and/or more efficient use of system resources.

A customized environment typically includes the application program it is designed to support. The terminal can be configured to boot directly to the program.

To modify the terminal environment, build a system image and download it to the terminal's NVM (Non-Volatile Memory). (See Figure 4-1). The NVM image provides you with a wealth of configuration options. The simplest use of the NVM image is to include an application program in non-volatile memory. Another common yet important task is to control the configuration of the terminal's memory. There are several ways of configuring memory and several ways to load an application into that memory. The NVM image also can contain an alternate operating system, overriding the default DR DOS.

Parameter switches provided with the User Configuration Tool (USRCFG.EXE) control the configuration and allocation of terminal NVM. The NVM image is generated by USRCFG and downloaded to the terminal NVM. For a detailed description of the User Configuration Tool, including parameter switches and supported interfaces, refer to the *USRCFG.EXE* section in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual*.

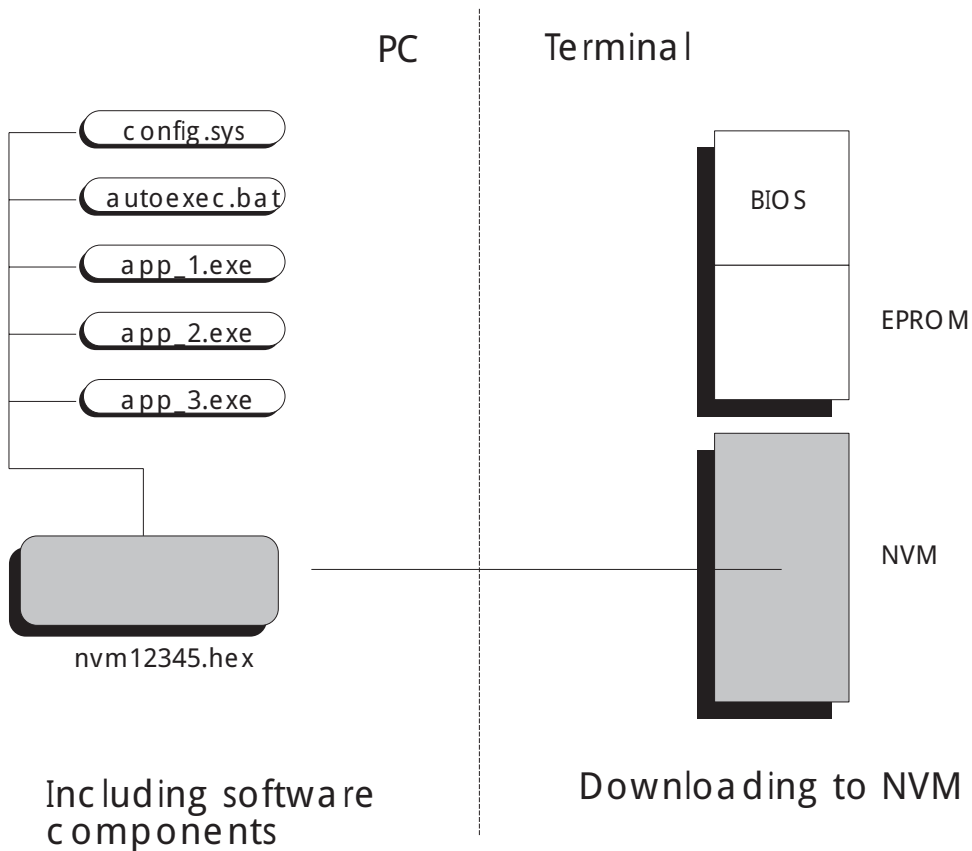


Figure 4-1. Configure and Download Process

System Memory Structure

Figure 4-2 depicts schematically the three areas that constitute Series 3000 memory:

- System EPROM (Erasable Programmable Memory)
- NVM (Non-Volatile Memory)
- RAM

See the *Memory* section, above, in the *Terminal Environment* chapter.

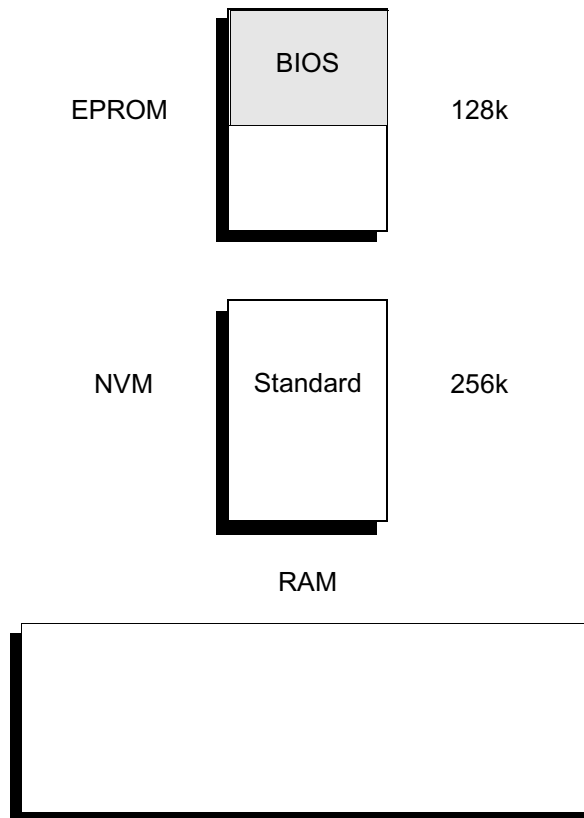


Figure 4-2. Series 3000 Memory Areas



Series 3000 terminal configuration, including the use of RAM, is determined by the contents of the system EPROM and the NVM memory areas.

System EPROM

The EPROM provides the default system configuration. It contains the system BIOS, the DR DOS operating system, and several standard system files. The operating system and other files are stored in an area configured as logical disk A, which can be accessed as on a standard PC. The EPROM is read only memory and therefore cannot be changed except by hardware modification.

In addition to the operating system itself, the following files are included in the EPROM and can be accessed on logical disk A:

- **CONFIG.SYS**, the default configuration file
- **AUTOEXEC.BAT**, the default startup file
- **SHELL.COM**, a stripped down command processor
- **2WAY3000.SYS**, the 2-WAY3000 protocol driver
- **S1.BIN**, the Spectrum One[®] protocol driver
- **LOADER.EXE**, the NVM loader
- **DSKBCHK.COM**, a program that checks for a drive B

NVM

The NVM is a configurable but **Non-Volatile Memory** region. In normal operation, NVM is read only memory, accessible as disk B. Writing to NVM requires a special download procedure, which first erases and then writes a new NVM image. Since this memory area is non-volatile, its contents are retained when power is removed from the terminal or when the terminal is rebooted.

NVM is used most commonly to store application programs and data when these are read-only and when the terminal is dedicated to a single application. If the program is changed frequently, it may be preferable to download programs and data to a RAM disk rather than to NVM.

The NVM also provides a method of overriding and modifying the default operating environment defined in EPROM. The NVM image commonly includes an AUTOEXEC.BAT and a CONFIG.SYS file, as well as special utilities required to support the application. Although uncommon, the NVM may contain an alternate operating system, replacing DR DOS.

Basic NVM Configurations

The NVM makes provides you with a great deal of control over the terminal configuration. There are three basic configurations. Which one you use is determined by what you choose to load into NVM. The three options are depicted schematically in Figure 4-3. Descriptions of these basic configurations follow the figure.

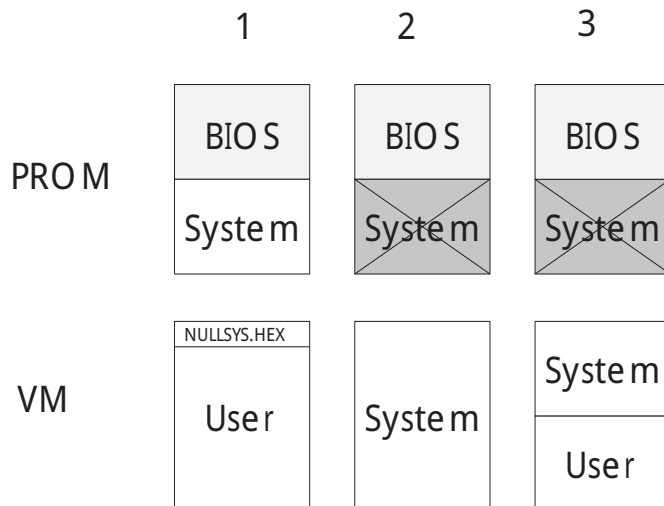


Figure 4-3. EPROM and NVM Configurations



Configuration 1

This option uses the default system configuration as defined in EPROM and downloads application software to NVM. Note that the system specified to the NVM must be NULLSYS.HEX. This configuration provides a large NVM area for applications and is used mostly for production systems.

The NVM also can contain a custom operating system, supplied by special request only from Symbol Technologies. This is rarely necessary, since DR DOS is a powerful and general operating system. However, if you choose to use a custom operating system with your applications, the following two configuration options become available:

Configuration 2

In this option you download a custom operating system — but *not* applications — to the NVM. The custom operating system overrides the default system in EPROM, leaving only the BIOS in effect in that memory area. Applications must be loaded to RAM disk.

Configuration 3

In this option you download both the custom operating system and application programs to NVM. The custom operating system overrides the default system in EPROM, leaving only the BIOS in effect in that memory area.

When a custom operating system is installed (i.e., Configuration 2 or 3, above), the default operating system in EPROM is no longer available. The logical drive A: is now the location of the custom operating system, while logical drive B: is still the location for user files .

File Precedence

Because system configuration files can be present on both drive A and drive B, an order of precedence has been established as follows:

- **CONFIG.SYS.**

When a CONFIG.SYS exists on drive B, it overrides the CONFIG.SYS file

on drive A. This is true whether the operating system (drive A) is in EPROM or NVM. If you attempt to view A:CONFIG.SYS, you actually view B:CONFIG.SYS.

- **AUTOEXEC.BAT.**

The last line of A:AUTOEXEC.BAT checks for the presence of B:AUTOEXEC.BAT on drive B. If it exists, it is executed following A:AUTOEXEC.BAT.

- **Command shell.**

A command shell on drive B takes precedence. The shell searcher looks for a command shell on drive B before looking on drive A.

Storing Programs

The User Configuration Tool (USRCFG.EXE) is the utility that generates an NVM image (.HEX file) for downloading to a Series 3000 terminal. For a detailed description of this utility, including interface options and the switches mentioned below, refer to the *USRCFG.EXE* section in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual*.

Programs downloaded to a Series 3000 terminal can be resident in the terminal in four ways:

- **RAM Disk.**

These programs are transferred to the system RAM disk. No special NVM configuration is required for this method. Since RAM is volatile, the program is erased from memory whenever the terminal is cold-booted. Programs too large to be loaded into NVM must be loaded onto RAM disk.

- **NVM Disk.** (USRCFG Switch=/u)

These programs are included in the NVM image as user files. The code and data are loaded into NVM on logical disk B. When executed, the program is loaded into RAM TPA (Transient Program Area), where it runs.



- **NVM.** (USRCFG Switch=/r)

NVM resident programs are downloaded to the NVM as code segments rather than as files. They run in place, requiring no TPA for their execution. A restriction on resident programs is that their data area is read only. This is unusual in that no memory is available for recording input or variable values. Accordingly, NVM resident programs are of limited usefulness.

- **Split-Resident.** (USRCFG Switch=/rs)

Split-resident programs are split into two parts:

- 1.The code part resides in the NVM code area and runs in place, like an NVM resident program.
- 2.The data part resides on the NVM disk as a part of the .EXE file, like an NVM disk resident program. It begins with a short code section linking the data and code segments; then it jumps to ROM and begins execution of the code in NVM.

Split-resident programs provide the advantages of NVM resident programs (code security and memory efficiency), while providing a writable data segment. Refer to *Chapter 12, Writing Resident Programs*, in this guide for instructions in writing NVM resident programs.

Note: /nr indicates that NO programs are resident. Any programs in the .RSP file with a /r designation will NOT be included in the .HEX image.

Building an NVM Image

The User Configuration Tool (USRCFG.EXE) is a PC software tool that configures the NVM area of the terminal. In this context, “configure” means indicating to the configuration tool which software components you want to reside in the NVM area of your terminal. When you have specified the software components, the User Configuration Tool outputs a hex image file. This file is then downloaded to the terminal. Figure 4-1 illustrates this process.

At minimum, the software image must include the System image (a custom system image created for you by Symbol Technologies, or a hex file that specifies a null system, NULLSYS.HEX) and software image tracking information. The image can also include a CONFIG.SYS file, a command shell (such as COMMAND.COM), an AUTOEXEC.BAT file, application programs, text/data files, drivers, other PC files, resident code files, and split resident files.

When you have specified all software components to reside in NVM, the User Configuration Tool combines them in a hex file format (.HEX file), ready to be downloaded to the terminal, where it is programmed or “burned” into the NVM.

USRCFG Output Files

The User Configuration Tool generates three output files:

- | | |
|-------------|---|
| .HEX | The NVM image file. The prefix is either the part number (default) or a name you specify to USRCFG. |
| .CFD | A machine-readable file that allows you to generate an ASCII report file detailing exactly what has been included in the image. The CFD can be read by USRCFG using the /p switch to recreate the .CFL. |
| .CFL | An ASCII file containing a report of each software component in the image. |

For a detailed description of the User Configuration Tool, including supported interface modes (interactive prompt, command line, and response file) and an annotated list of switch parameters, refer to the *USRCFG.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

NVM Configuration Tool Interface Modes

The User Configuration Tool supports three interface modes:

- **Prompt Mode**
USRCFG.EXE is invoked *without* arguments. This initiates an interactive interface in which you specify the desired NVM configuration by



responding to a series of prompts. See the *Prompt Mode* section below.

- **Command Line Mode**

USRCFG.EXE is invoked with command line arguments that specify the desired NVM configuration. See the *Command Line Mode* section below.

- **Response File Mode**

USRCFG.EXE is invoked with the name of a response file as the only argument. The response file specifies the desired options in the NVM configuration. This is the recommended mode. See the *Response File Mode* section below.

These interface modes are described separately in the following.

Prompt Mode

Prompt Mode provides an interactive method for entering NVM image file specifications. To run the User Configuration Tool in Prompt Mode, enter:

USRCFG

The program displays an initial message to your screen and begins with the first prompt.

Note: To terminate the program at any point, press
<ESCAPE>.

Enter System File Name:

To use the default DR DOS in the system EPROM, enter **NULLSYS.HEX** .

Or:

To use a custom (alternative) operating system enter the name of the custom operating system image.

Enter Output File Name [Use part #]:

Specify the output file name prefix (up to 8 characters) for the .HEX image file, or press <ENTER> to use the part number as the prefix.

Enter part number and version (99999-CCCCCCC):

Assign your own, unique part number and a unique version designator to the image. This helps you track changes to the configuration. This number is displayed during the boot sequence.

The first five digits are required and comprise the part number for the image.

The last seven characters are alphanumeric and allow you to distinguish versions of the image. The version designation need not use all 7 characters.

The hyphen is required.

Do you want to include a config file [Yes]?

Answer YES if you are including a CONFIG.SYS file in the image. This file supersedes the default CONFIG.SYS in EPROM. A prompt then asks for the name of the file to use:

Enter Config File Name:

USRCFG includes this file in the image, renaming it "CONFIG.SYS" in the image.

Do you want to include a shell file [Yes]?

Answer YES if you are including a command interpreter, such as COMMAND.COM, to replace the default SHELL interpreter in EPROM. A prompt then asks for the shell file name:

Enter shell file name:

Enter the file name. The file is not renamed in the image.

Do you want to include an autoexec file [Yes]?

Answer YES if you are including an AUTOEXEC.BAT file in the image. This batch file is executed during a system boot following the default AUTOEXEC.BAT in EPROM. You are then prompted for the file name:

Enter autoexec file name:

USRCFG includes this file in the image, renaming it "AUTOEXEC.BAT" in the image.

Do you want to include any user files [Yes]?

Answer YES if you are including any program or data files in the NVM image. These include application programs and data as well as additional utilities. A prompt then asks for the name of the first file:



Enter user file name to include [No more]:

Enter the first file name. The prompt is repeated for as many files as you want to include. When you have entered all your files, press <ENTER> alone at the prompt to proceed.

Note that if you do not include file extensions, .EXE is provided as the default. In general, It is recommended that you specify extensions.

Do you want to include any resident code files [No]?

Answer YES if you have resident code files to include. A prompt then asks for the name of the first file:

Enter resident code file name to include [No more]:

Enter the name of a resident file. You are then asked:

Do you want to split the code and data [No]?

Answer YES if the code and data sections are to be split, otherwise answer NO.

The file name prompt is then displayed again. Continue entering file names for your resident code files. When you have entered all the files, press <ENTER> alone at the prompt to proceed.

This ends the entry phase . USRCFG then generates the NVM image file according to your specification.

Command Line Mode

USRCFG accepts configuration options entered directly on the command line. Any response that you include on the command line suppresses the corresponding prompt. You are still prompted for responses to parameters that you do not specify on the command line.

If you enter parameters on the command line, the first parameter must be either "NULLSYS.HEX" or the name of a custom system image.

Switches fall into three categories: Input switches, Output switches and Print switches. *Switches must be entered on the command line in this order:*

1. Input switches

2. Output switches
3. Print switches

Each switch consists of either a slash (/) or dash followed by one or more characters, followed by a space and any parameter value. Switch letters may be in upper or lower case. Switches and parameters must be separated by a space. If a response requires a filename or other response, place it after the switch. If an output filename is not specified, use a default filename based on the part number.

The command line format is:

```
USRCFG system-image [input options] [output options] [print options]
```

For example:

```
USRCFG sys11111.hex /s c:\3000\files\command.com
      /c c:config.sys
      /na /rs hellom /r hellos /u c:\commands\beep.exe
      /l 12345-x1.01-1 /h
```

Table 4-1 lists valid User Configuration Tool command line switch parameters and their meanings.

Table 4-1. USRCFG Command Line Switches

Input Switches	
Switch	Description
/n arg	Specifies the start of data symbol used in the .MAP file containing pointers to locations in the .EXE file. USRCFG needs this information to separate code and data for split resident programs. The default symbol is BEGDATA, as used by Microsoft C. A /n switch applies to all following /rs arguments until another /n is specified.
/c file	Load file as CONFIG.SYS in the NVM image. The file is renamed "CONFIG.SYS" in the image.
/nc	No CONFIG.SYS file. The default CONFIG.SYS in EPROM will be used. Use this switch to suppress the prompt.



Table 4-1. USRCFG Command Line Switches (Continued)

/s file	Load file as the command shell.
/ns	No command shell file. The default file, SHELL, in EPROM will be used. Use this switch to suppress the prompt.
/a file	Load file as AUTOEXEC.BAT in the NVM image. The file is renamed to "AUTOEXEC.BAT" in the image.
/na	No AUTOEXEC.BAT file. The default AUTOEXEC.BAT in EPROM will be used. Use this switch to suppress the prompt.
/u file	Load file as a user file. You can specify several user files, but only one per switch. For example: /u mydata.txt /u mydata2.txt
/nu	No user files. Use this switch to suppress the prompt.
/r file	Load file as a resident code file. You can specify several files, but only one per switch.
/rs file	Load file as a split resident code file. You can specify several files, but only one per switch.
/nr	No resident code files. Use this switch to suppress the prompts for /r and /rs files.
/l label	Label the NVM image as label. The label format is: #####-CCCCCCC or #####/CCCCCCC where ##### is a five digit part number, and CCCCCC is up to seven characters specifying the version. Ex: 12345-ver10
Output Switches	
Switch	Description
/h file	Output a .HEX file named file.HEX. The default filename is based on the image label.
/i file	Output a .IMG file named file.IMG. The default output filename is based on the image label.
	Note: If both the /i and /h switches are specified, supply only one filename, without an extension, for example: /i /h MYFILE.

Table 4-1. USRCFG Command Line Switches (Continued)

/in	Generate an Intel format .HEX file. (Used with /h switch only.)
/bi	Generate a binary format .HEX file. (Used with /h switch only.)
/co	Generate a compressed format .HEX file. (Used with /h switch only.) This saves disk space but may limit sendhex speed.
/tr	Generate a transparent format .HEX file (used with /h switch only.)
/256	Same as /j; use up to 256K of NVM. This switch is required to use more than 128K of NVM.
/128	Same as /nj; use only 128K of NVM.
/no	Do not generate .HEX or .IMG output files. Print files are still generated.
Print File Switches	
Switch	Description
/p file	Override the default print filename using the filename specified.
/np	Do not generate print files. Not recommended.

Response File Mode (Recommended Method)

Instead of entering the specifications for a desired NVM configuration information in response to prompts or as command arguments, you can provide this information in a response file that you include as the only command line argument when you invoke the User Configuration Tool (USRCFG.EXE). The response file is used as an input batch file that responds to USRCFG prompts. Besides being quicker, the response file allows you to specify more options than are allowed on the DOS command line, and also serves as documentation thereafter.



You specify the options and filenames using the same conventions as are required for the Command Line Mode that is described above in the *Command Line Mode* section. The response file can have any valid DOS filename, although it is recommended that you use the “.RSP” extension.

To build the NVM image file, execute USRCFG with the name of your response file, preceded by “@”:

```
USRCFG @filename (default extension is .RSP)
```

Building a Response File

Use any text editor that produces a plain ASCII file to enter lines in the response file.

You must specify switches in groups in this order:

1. Input switches
2. Output switches
3. Print switches.

To make your response file more readable, do the following:

- Place each switch argument on a separate line. A carriage return can occur anywhere a space would occur on the command line.
- Insert blank lines and comments into your file. USRCFG ignores the remaining portion of any line after an exclamation point (“!”).

A sample response file is provided at the top of the next page.

Sample Response File

```
! Load default system image
sys111111.hex
! Load shell and configuration, but no autoexec
/s c:\3000\files\command.com
/c c:config.sys !our custom config.sys
/na
! Load two resident code segments
!   one split, the other not split
/rs hellom
/r hellos
! Load one normal user .EXE file
/u c:\commands\beep.exe
! Specify the image label
/l 12345-x1.01-1
! Specify the output file as .HEX
/h /j sample.hex
```

Custom System Image

For most applications the standard operating system contained in system EPROM is a good operating system. However, it may be that your applications require a custom operating system.

A custom operating system, if needed, must be purchased from Symbol Technologies. The custom operating system should be included in the NVM image and identified as the system file instead of NULLSYS.HEX. The NVM is then configured to be both A and B disk drives, with the system image on disk A just as the default system is.

AUTOEXEC.BAT and CONFIG.SYS Files

If disk drive B contains a CONFIG.SYS file or an AUTOEXEC.BAT file, the following occurs:

- The AUTOEXEC.BAT in drive A will call the AUTOEXEC.BAT on drive B
- The CONFIG.SYS file on drive B will replace the CONFIG.SYS on drive A



Sample USRCFG Response Files

A Development Test System

A development system is used primarily as a test system for programs. To shorten load time, test programs are loaded and run directly from RAM. The following response file provides a useful environment:

```
c:\3000\files\nullsys.hex
/s c:\3000\files\command.com
/c test.sys
/a test.bat
/nr
/u c:\3000\files1.bat
/u c:\3000\files2.bat
/u c:\3000\tdrem.exe
/l 00012-0.0
/h /256 /test.hex
```

The file locations are on the development PC . Assume that the files were installed on the C drive using the default directories and subdirectories.

- | | |
|-----------------------------------|---|
| /c test.sys | Provides the system file “test.sys”. This system file becomes the terminal's CONFIG.SYS file. |
| /a test.bat | Provides the batch file “test.bat”. This batch file becomes the terminal's AUTOEXEC.BAT file. Since no program is run by this file, the COMMAND.COM program runs when the terminal is first turned on and prompts for further commands. |
| /nr | Indicates that no resident files of any kind (/r or /rs) will be loaded in this environment. |
| /u c:\3000\files\tdrem.exe | Provides the terminal segment of the Borland Turbo Debugger. When used with the host segment, TDREM allows you to load executable files directly to the terminal's RAM disk and to debug them remotely. |

/u c:\3000\files1.bat and /u c:\3000\files2.bat	Provide batch files to activate TDREM on different communications ports.
/l 00012-1.01	Specifies the image label, including part number and version.
/h /256 /test.hex	/h specifies the output file test.hex as hexadecimal format; /256 allows the use of 256K NVMs

A Production System

A production system is used primarily in the field to run programs. To protect the program from accidental corruption, it is loaded into NVM.

```
c:\3000\files\nullsys.hex
/s c:\3000\files\command.com
/c prod.sys
/a prod.bat
/nr
/u c:\3000\progs\invntry.exe
/l 00012-1.01
/h /256 /prod.hex
```

This configuration is similar to that for the development system except that it includes a different program and omits the removed portion of the Borland Turbo Debugger. The AUTOEXEC.BAT file (PROD.BAT) looks like this:

```
PATH b;;d:
cls
invntry
```

The last line runs the application upon system boot.

Make certain that any resident libraries and programs the application needs are in the terminal and activated before executing the application program.



Using the DOS CONFIG.SYS SHELL Command

The CONFIG.SYS shell command and SHELL.COM provide a means of activating programs once in the Series 3000 terminal. This command uses the form:

```
shell = a:shell.com <program 1><program 2><etc.>
```

Where each “program” is specified by a complete terminal path location.

When the system boots, SHELL activates each program in the order listed. The last program re-executes upon termination (as an infinite loop).

If no TSRs need to be loaded, only one program needs to be run and if the program never ends, then the form indicated below can be used instead:

```
shell = <program>
```

This saves the 1.5K used by the shell.com program.

Restoring the Default NVM Image

Series 3000 terminals are shipped from the factory with a default NVM image already loaded. Occasionally, you may need to restore the default image after you have replaced it — for instance, if you need a known good image.

The ADK includes several files, named “\3000\SAMPLE\DEFNVM.*”, which are needed to generate the image. Create the following file and name it DEFNVM.RSP:

```
/s C:\3000\files\command.com
/c C:\3000\sample\defnvm.sys
/a C:\3000\sample\defnvm.bat
/r C:\3000\adl\example\init.exe
/rs C:\3000\sample\scan3000.exe
/rs C:\3000\sample\order.exe
/u C:\3000\sample\scan.exe
/u C:\3000\sample\batcher.exe
/u C:\3000\sample\demo.bat
/u C:\3000\sample\demo.ini
/u C:\3000\files\tdrem.exe
/u C:\3000\scanparm.exe
```

```
/u C:\3000\files\eta3000.sys  
/l 00000-1.00 // Label  
/h /256 /co devnvm.hex
```

Then apply the User Configuration Tool (USRCFG.EXE) to create the .HEX file as follows:

```
c:\3000\sample>usrcfg @defnvm
```

If no errors occur, this command generates:

```
C:\3000\SAMPLE\DEFNVM.HEX.
```

For a description of the User Configuration Tool, refer to the *USRCFG.EXE* section in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual*.

NVM Image Size

The only restriction on the number of files you can load into NVM is the amount of memory available. Series 3000 terminals have either 128 KB or 256 KB of NVM.

There is no easy formula to provide the total NVM image size for a given configuration. The following formula, however, will give you a good approximation:

$$\text{NVM image size} = 1.02 X + 0.86 Y$$

where:

X = total size of files loaded with the **/s**, **/a**, **/c**, or **/u** USRCFG command switches, and

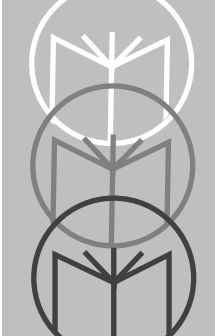
Y = total size of files loaded with **/r** or **/rs** USRCFG command switches.

Refer to Table 4-1 in this chapter for descriptions of these switches.



A number of considerations affect the amount of NVM required for any given file. One addition may require a variety of collateral additions. For example, if you add a 30-byte file, this adds an additional 30 bytes in the data area. If another sector is required, another sector map entry is also required, consuming another 4 bytes. Also needed is another entry in the FAT (2 bytes) and another entry in the directory (32 bytes). That may extend the directory into a second directory sector, which requires another sector map entry, etc.

The only way to find the exact size of the NVM image is to examine the configuration listing produced by the User Configuration Tool by generating the NVM image. Configuration listing files are named using the part number as the prefix and ".CFL" as the extension.



Chapter 5

Terminal Initialization

Chapter Contents

Introduction	5-3
Running BLDINIT	5-4
Font Table Selection.....	5-8
Files Linked with BLDINIT	5-8



Introduction

The Terminal Initialization Configuration Tool, BLDINIT.EXE, builds the INIT.EXE initialization program for Series 3000 terminals. INIT.EXE must then be downloaded to the NVM as NVM Resident Code. The warm and cold boot routines invoke INIT.EXE to initialize the terminal.

INIT.EXE initializes a number of system features, including:

- RAM Disk size
- TPA (Transient Program Area) size
- Communication queue sizes and default setup
- Console queue size
- Keyboard configuration
- Display font
- Cursor shapes
- Low-battery messages

INIT.EXE requires System BIOS version 3.01 or higher. To resize the TPA you must also include ETA3000.SYS.

INIT.EXE and ETA3000.SYS together replace the following drivers in previous versions of the ADK:

- CFG3000.SYS
- SCR3000.SYS
- SIZ3000.SYS
- TPA3000.SYS
- FONTRES.EXE



Running BLDINIT

To run BLDINIT, enter at the DOS prompt:

```
BLDINIT filename
```

The *filename* argument is optional and is used to specify a prefix for the output files. If **filename** is not entered, BLDINIT uses the default prefix (INIT) and generates:

```
INIT.C  
INIT.OBJ  
INIT.EXE  
INIT.RSP
```

If the .C file already exists, BLDINIT displays the following prompt:

```
Filename already exists. Load? [Y/N]
```

Enter "Y" to use the data from the existing C file. Otherwise, it is replaced with a new C file based on the new parameters.

The Main Menu is then displayed, as shown in Figure 5-1.

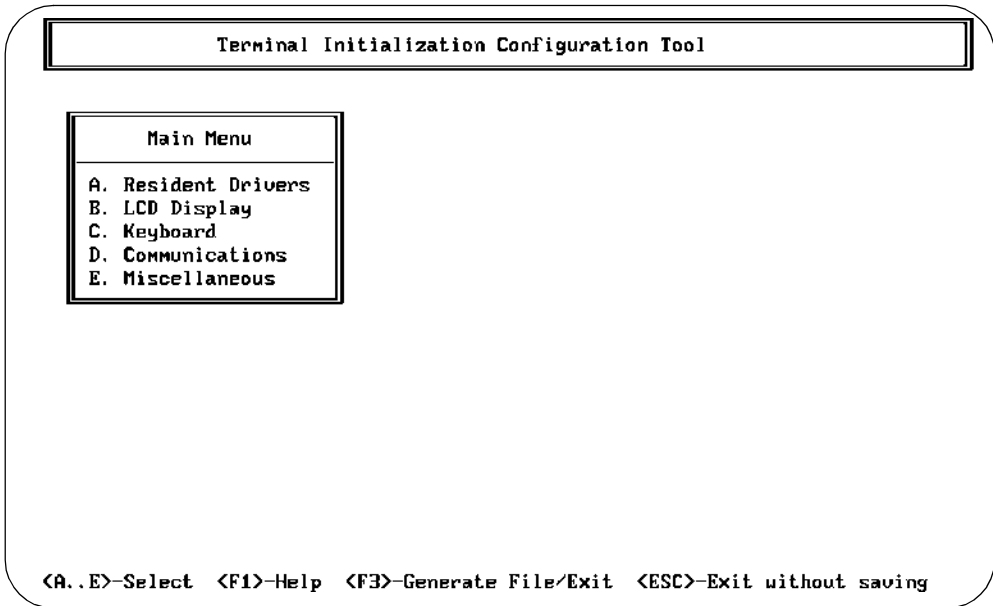


Figure 5-1. BLDINIT Main Menu

This menu contains a list of parameter categories. Select a category by pressing the letter displayed next to it or by moving the cursor with the arrow keys.

Once a category has been selected, an asterisk is displayed next to it and a submenu is displayed. The submenu contains all the available parameters under that category. This is illustrated for the "Resident Drivers" category in Figure 5-2.

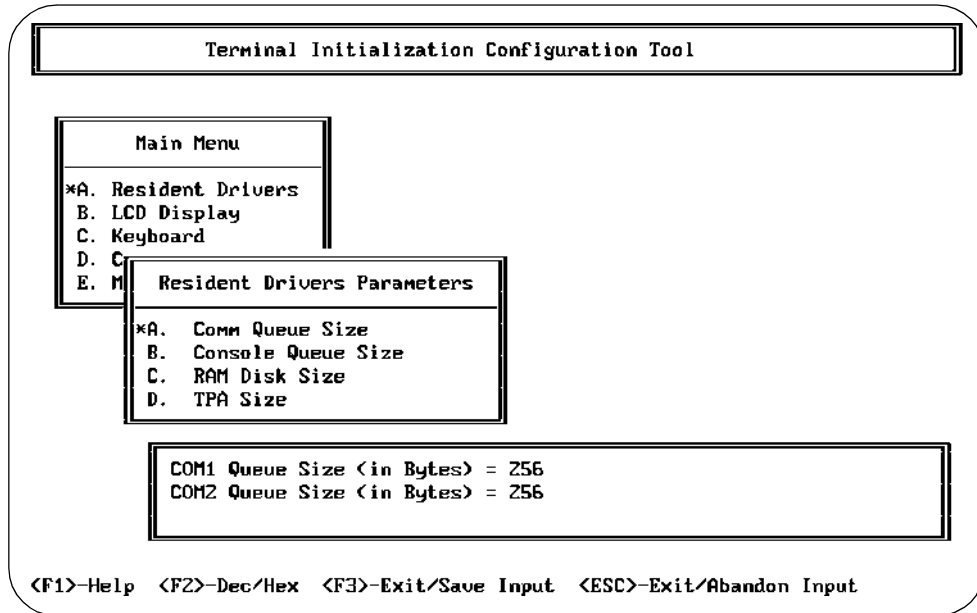


Figure 5-2. Resident Drivers Submenu

When you select a submenu item, you are prompted to enter the value(s) for that parameter.

If a valid file name has been entered at the command line, the values retrieved from the file are displayed. To keep these values simply press the **<ENTER>** key. Otherwise:

- For prompts that require a file name or function name as input, enter any string of up to 64 characters.
- For prompts that require decimal numerals, enter a maximum of 5 digits.
- You can enter a hex numeral by using the prefix "0x". A maximum of four hex digits will be accepted.
- Press the **<ENTER>** key to accept the data entered. If there is only one parameter value to enter, the **<ENTER>** key will bring you back to the submenu screen.

- Press <ESCAPE> to return to the previous menu without saving the values entered.
- Press <F1> for help information on any screen or prompt.
- Press <F2> to switch between decimal and hexadecimal mode.
- Press <F3> to save all the input parameters and return to the previous screen.

If no data is entered for a parameter, the default parameters set by the BIOS and the resident drivers remain in effect.

Loading INIT.EXE in NVM

To include INIT.EXE in the NVM image as resident code, you must include it in the image built by the User Configuration Tool (USRCFG.EXE). If you use the Response File Mode to enter NVM image file specifications, include the following line in the response file:

```
/r init.exe
```

This can also be done with a command line parameter if you use the Command Line Mode or entered in response to the resident code files prompt in the Prompt mode. For more detailed information on the User Configuration Tool and on the three methods of entering NVM file specifications, refer to:

The *Building an NVM Image* section of the chapter on *NVM Configuration* in this guide.

The *USRCFG.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

To resize TPA (Transient Program Area), include the ETA3000.SYS driver in the USRCFG response file (or as a command line parameter, or as an entry in response to the user files prompt) as follows:

```
/u eta3000.sys
```

Also, load the ETA3000.SYS driver as the *last* **device=** line in the NVM CONFIG.SYS file as follows:

```
device=b:eta3000.sys
```



Font Table Selection

For the font table selection, enter a font file name (a file with .FNT extension generated by the font builder program FONTBLD.EXE). For a detailed description of the font builder program, refer to the *Font Builder* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

When you enter a font file name, BLDINIT includes the font file in INIT.C. This font will be the default display font when the terminal is initialized.

Files Linked with BLDINIT

There are five parameters in BLDINIT which require an object file name for input. If you enter a file name without the extension, ".OBJ" will be assumed. The source file can be coded in either assembly language or C.

If used, the files shown in the following table must contain the predefined public labels specified in the right-hand column:

Object File	Label
Keyboard redefinition table	_kybd_tbl
Cursor translation table	_cursor_tbl
Replace cell message	_rep_cell_msg
RAM Disk configuration	_ram_cfg

BLDINIT will output an external reference to these labels and the specified object files will be linked with INIT.OBJ to form INIT.EXE.

Keyboard Redefinition Table

Keyboard Redefinition tables can be created using the Keyboard Mapping Utility (KBDMAKE.EXE); which is described in detail in:

The Using the Keyboard Mapping Utility section of the Keyboard Processing chapter in this guide

The KBDMAKE.EXE section of the Utilities chapter in the Series 3000 Application Programmer's Reference Manual.

The hypertext sample document (in the C:\3000\SAMPLE\INIT directory of the ADK) contains the following sample keyboard redefinition tables:

KYTBLC.C
KYTBL.ASM

The *Using the Keyboard Mapping Utility* section of the *Keyboard Processing* chapter in this guide describes keyboard redefinition. There is a complete sample program (KEYREDEF.C) in the C:\3000\SAMPLE\KEYBOARD directory of the ADK. KEYREDEF.C is a stand-alone executable program. KYTBLC.C, however, is a table in the CODE segment which is linked with INIT.OBJ.

Cursor Translation Table

The Cursor Translation Table sets the cursor character displayed for each keyboard state.

Refer to the description of the **Get/Set Cursor Character Translation Table** service (**INT A1h, Function 88h**) in the *Extended Video Services* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual* for a listing of the default Cursor Character Translation Table. This default table uses ASCII characters 80h to 8Fh for the cursor characters.

To change the cursor characters, you can:

- Use the default translation table, but use the FONTBLD utility to generate a font with different characters for 80h-8Fh. For a detailed description of this utility, refer to the *Font Builder* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.
- Use the default font, but create a translation table which points to a different set of cursor characters in the default font.
- Use a different translation table and a different font.

A cursor translation table is composed of the Normal Cursor Translation Table, followed by the Low Battery Cursor Translation Table. Each table is eight words long and contains entries for the following keyboard states:

Unshifted
Caps Lock



Momentary Shift
Momentary Unshifted
Num Lock
Control
Alt
Function

Each entry in a table contains the ASCII code for the cursor character in the lower byte and the attribute code for that character in the upper byte.

The hypertext sample document contains the following cursor translation tables:

CSRTBLC.C
CSRTBL.ASM

These files are in the C:\3000\SAMPLE\INIT directory of the ADK.

Replace Cells Message Table

The Replace Cells Message allows applications to replace the default Replace Cells Message "REPLACE BATTERY" with a message tailored for the application.

Replace Cells Messages are strings in the alternating character/attribute format (a string consisting of a character followed by its display attribute). The Replace Cells Message must be exactly 20 text characters long (40 bytes, including the attributes). Shorter messages must be padded with blanks.

The current Replace Cells Message is displayed on the first line of the display when the BIOS detects the Replace Cells Event. The message is displayed for 3 seconds before turning off the terminal. If the terminal is turned back on, this message will be displayed again for three seconds before the terminal is turned off again.

The hypertext sample document contains the following replace cells message tables:

REPCELLC.C
REPCELL.ASM

These files are in the C:\3000\SAMPLE\INIT directory of the ADK.

RAM Disk Configuration Routine

If you select the RAM Disk Configuration parameter ("Ram Disk Size") in the BIOS submenu of BLDINIT, you have the following choices:

1. Define the total size of RAM Disk and the amount of EMS to be used for the RAM Disk.
2. Use a customized routine to calculate these sizes.

If you decide to use a customized routine, you will be prompted to enter a file name. The file must contain a public routine named "**_ram_cfg**".

This routine will be called during the bootup process. Before this routine returns, it must set these registers:

BX =	size of RAM Disk in kilobytes
CX =	size of EMS used for RAM Disk in kilobytes
DX =	Maximum number of files and/or directories in the RAM Disk root directory.

These values will be passed to the RAM Disk driver and the RAM Disk will be configured accordingly.

The hypertext sample document contains the following RAM Disk configuration program:

_RAM_CFG.ASM

This routine cannot be written in the C language.



Initialization Parameters

Table 5-1 lists all of the BLDINIT initialization parameters with a range of valid values and a default value for each parameter. See Figures 5-1 and 5-2 in the *Running BLDINIT* section of this chapter for the Terminal Initialization Configuration Tool (BLDINIT.EXE) menu and submenu screens.

Table 5-1. BLDINIT (Terminal Initialization) Parameters

Parameter Group	Parameter	Valid Values and Default
Resident Drivers	Comm Queue Size	32 - 32,768 Bytes Default: 256 Bytes
	Console Queue Size	4 - 9999 Bytes Default: 100 Bytes
	RAM Disk Size EMS for RAM Disk	0 - 65,535 Kbytes 0 - 65,535 Kbytes
	Max Entries in Root Dir	0 - 65,535 Default: 64
	Can use custom routine to calculate and set sizes	Default: If EMS Use all of EMS Else Use 1/4 of Conventional RAM
	TPA Size EMS Reserved for EMM	0 - 65,535 Kbytes 0 - 65,535 Kbytes

Table 5-1. BLDINIT (Terminal Initialization) Parameters (Continued)

Parameter Group	Parameter	Valid Values and Default
LCD Parameters	Backlight Timeout	0 - 255 Seconds Default: 10 Seconds
	Cursor Mode	0 = Hardware 1 = Software Default: Software
	Cursor On/Off	0 = Off 1 = On Default: On
	Cursor Translation Table	Optional Default: BIOS default
	Font Selection	Optional Default: BIOS default
	Row Scale Column Scale	0 = Normal 1 = Double Wide/High Default: Normal
	LCD Viewing Angle	0 - 7 0 = Dimmest 7 = Brightest Default: 3
	Logical Screen Size: Number of Display Pages Number of Columns Number of Rows	0 - 255, Default: 8 0 - 255 0 - 255 Default: Physical Screen Size
	Screen Refresh Rate	0 - 3 0 = Fastest 3 = Slowest Default: 1
	Virtual Screen Size	Default: Physical Screen Size



Table 5-1. BLDINIT (Terminal Initialization) Parameters (Continued)

Parameter Group	Parameter	Valid Values and Default
Keyboard Parameters	Abort Key Scan Code	0 - 255 Default: 27 (Clear Key)
	Auto Repeat Key Rate Initial Delay Repeat Delay	0 - 65,535 ms 0 = Not Enabled Default: Not Enabled
	Key Click Duration	0 - 65,535 ms Default: 5 ms
	Key Click Enable\Disable	0 = Enabled 1 = Disabled Default: Disabled
	Keyboard Inactivity Timeout	0 - 65,535 Seconds Default: 30 Seconds
	Keyboard Operation Mode	0 = One-Finger 1 = PC Mode Default: One-Finger
	Keyboard Redefinition Table	Optional Default: BIOS Table
	Keyboard State	Bit 0 = Right Shift Bit 1 = Left Shift Bit 2 = Control Bit 3 = Alt Bit 4 = Scroll Lock Bit 5 = Num Lock Bit 6 = Caps Lock Bit 7 = Function

Table 5-1. BLDINIT (Terminal Initialization) Parameters (Continued)

Parameter Group	Parameter	Valid Values and Default
Communication Parameters Comm 1 or 2	Data Rate	0 = 150 bps 1 = 300 bps 2 = 600 bps 3 = 1200 bps 4 = 1350 bps 5 = 2400 bps 6 = 4800 bps 7 = 9600 bps 8 = 19200 bps 9 = 38400 bps Default: 9600 bps
	Data Size	2 = 7 bits 3 = 8 bits Default: 8 bits
	Parity	0 = Even 1 = Odd 3 = Space 4 = None Default: None
	Stop Bits	0 = 1 Stop Bit 1 = 2 Stop Bits Default: 1 Stop Bit
	Duplex	0 = Full Duplex 1 = Half Duplex 2 = Multi-Access Default: Full Duplex
	Modem Delay	0 - 65,535 ms Default: 200 ms
	Xmit Carrier Wait Time	0 - 65,535 ms Default: 100 ms
	Rcv Carrier Wait Time	0 - 65,535 ms Default: 0 sec
	Carrier Loss Detect Time	0 - 65,535 ms Default: 0 sec



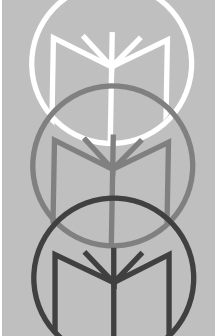
Table 5-1. BLDINIT (Terminal Initialization) Parameters (Continued)

Parameter Group	Parameter	Valid Values and Default
Communication Parameters Comm 1 or 2	Ctrl Stop Character	0 - 255 Default: 19
	Ctrl Start Character	0 - 255 Default: 17
	Ctrl Start Wait Time	0 - 255 Sec Default: 20 Sec
	CTS Loss Detect Time	0 - 255 Sec Default: 3 sec
	Recv Char Wait Time	0 - 255 Sec Default: 30 sec
	DSR Wait Time	0 - 255 Sec Default: 20 sec
	CD Wait Time	0 - 255 Sec Default: 0 sec
	SPACE Time	0 - 255 Sec Default: 0 sec
	MARK Time	0 - 255 Sec Default: 0 sec
	Line Conditioning Flags	Bit 2 = CTS Conditioning Bit 1 = CD Conditioning Bit 0 = DSR Conditioning Default: 5
	Operation Mode	0 = No Flow Control 1 = Software 2 = Hardware Default: 0

Table 5-1. BLDINIT (Terminal Initialization) Parameters (Continued)

Parameter Group	Parameter	Valid Values and Default
Communication Parameters Comm 1 or 2	Line Status Error Mask	Bit 4 = Break Detect Bit 3 = Framing Bit 2 = Parity Bit 1 = Overrun Default: 0
	Error Insertion Character	0 - 255 Default: 0
	DTR Settling Time	0 - 65,535 ms Default: 0 ms
	Connect Delay	0 - 255 Seconds Default: 0
Miscellaneous	Cradle Charging Rate	0 = Slow 1 = Normal Default: Normal if 3800 Slow if 3300/3100
	Replace Cell Message	"REPLACE BATTERY"
	Speaker Volume	0 = Low 1 = High Default: High
	Wake Up Events Power Off Events Mask Keyboard Timeout Mask	Bit 4 = Laser Trigger Bit 3 = Port 1 Ring Bit 2 = Port 0 Ring Bit 1 = RTC Alarm Bit 0 = Any Key Wake Up Defaults: After normal power off, all events except Any Key Wake Up After keyboard timeout, all events





Chapter 6

Managing Files

Chapter Contents

Introduction	6-3
UBASIC File Manager Features	6-4
Using the File Manager	6-8
Loading the File Manager	6-10
DOS File Management APIs	6-13
Microsoft Visual C Runtime Functions	6-14
Sample Programs	6-16



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>DR DOS</i> <i>UBASIC Record Manager</i>

There are two file management resources available to you for managing disk files on Series 3000 terminals. These are:

the DOS file management system

This file management system should be familiar to any one who has programmed in a DOS environment. This environment supports applications being ported from PCs as well as new applications. Any good book on programming for the PC will describe file management using DOS facilities. The *MS-DOS Programmer's Reference* and *The Peter Norton PC Programmer's Bible*, both published by Microsoft Press, are both useful and widely available resources.

- Symbol's UBASIC managers

The UBASIC Record Manager, File Manager, and Memory Manager form a high level file I/O system providing for simple and efficient use of disk space (RAM disk) on Series 3000 terminals. For applications being ported to the Series 3000 from earlier Symbol terminals, the UBASIC managers fully support the UBASIC data files. Refer to the chapter on the *UBASIC Record Manager* in the *Series 3000 Application Programmer's Reference Manual*.

This chapter we focuses primarily on the UBASIC managers.



UBASIC File Manager Features

The UBASIC File Manager provides high level file storage and access methods for fast and efficient file management. File Manager functions are designed especially to make use of the lower-level RAM disk driver routines that allocate conventional and expanded memory. Applications may build on these facilities, adding the semantics of data base management.

Record Types and Storage Methods

The File Manager supports both fixed and variable length ("variant") records, thereby freeing the application from the overhead associated with defining and managing these types. It also stores records as linked lists of individual records or as linked lists of groups of records, called clusters.

This scheme of record types and linking yields four file type choices:

- **Linked Fixed**
- **Linked Variant**
- **Cluster Fixed**
- **Cluster Variant.**

Each file type has advantages for different situations.

Use **Linked Fixed** when:

- The application's data structures require only fixed length records.
- The application frequently inserts and deletes records.

Use **Linked Variant** when:

- The application's data structures require variable length records and the records are of widely different sizes.
- The operator frequently inserts and deletes records.

Use **Cluster Fixed** when:

- The application's data structures require only fixed length records.
- The operator very seldom or never inserts and deletes records.

- You require very fast random and sequential access .

Cluster Fixed files are the most space-efficient file type and allow the fastest access to records. When the application inserts or deletes records, these Cluster Fixed files shuffle records to keep all but the last cluster full. As a result, inserts and deletes tend to reduce speed.

Use **Cluster Variant** when:

- The application's data structures require variable length records.
- You want to save data memory space and are seldom doing inserts or deletes.

When the application inserts and deletes, Cluster Variant files do *not* shuffle records to keep the clusters full. This keeps the speed fast but may leave a lot of unused space. Use the **mcrunch** function to crunch the clusters and free the unused space. For a description of this function, refer to the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Access to records is generally faster in the Cluster Variant than in the Linked Variant files.

Choose fixed length records when possible, but do not “pad” shorter records to fit the fixed length mold.

The following indicate advantages of using one or the other of the various record and storage types in certain conditions:

Variant Record Types.

When an application collects different types of data in a single file, it is usually better to use a file type with a variable length rather than one with a fixed length record format.

Using variable length records, the File Manager stores each record using only the space required for the data plus a one-byte record-type identifier. This is more efficient than padding smaller records to the length of the largest record in a fixed length record file.



Record Linking and Clustering.

The File Manager uses two methods for accessing records:

- linking
- clustering.

Linked record file types use three bytes of overhead per record to store the links. In an application using a large number of small records, this results in considerable overhead.

Clustering reduces the amount of overhead by grouping the records. In clustered file types, the clusters are linked together instead of the individual records. This requires only one link per cluster instead of one link per record. As the number of records per cluster increases, the overhead per record decreases.

In the U BASIC File Manager's implementation of this technique, the size of all clusters in the file are defined when the file is created and cannot be changed without deleting and recreating the file.

Record Type Information

When using variant records, you can add descriptive information to record types. For each record type, you can specify a record type information string when the Record Type Table is defined for a given variant record file. As the application reads and writes records in the file, it can get the record type information string for each record accessed.

The information strings can be comments or can be used to define the structure of the records (field definitions, etc.). This is an optional feature and requires only a small amount of overhead.

Sequential Access by Record Type

If a record type information string is defined for the record types in a variant record file, you can use the string to sequentially access records by type. The File Manager performs the function of skipping records of incorrect types automatically.

Record Overwrite Protection

The File Manager prevents one type of record from overwriting another type. Records in a file may be different lengths and there may not be enough memory allocated to write one type of record in the space allocated for another.

Therefore, to replace one type of record with another type, it is necessary to use **mdelete** and **minsert** functions. For descriptions of these functions, refer to the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Bounded Record Searching

In some cases, an application is interested in certain records in a specific area of a file. The File Manager provides a method for setting the lower and upper bounds in the search statement. This allows ranges within the file to be processed much more quickly, since records outside the bounds need never be searched.

Record Update Support

The File Manager makes available the record number of the record “just accessed”. You can use this as the record number when you perform a sequential update by record type.



Using the File Manager

File Manager Functions

Table 6-1 lists the functions that the UBASIC File Manager provides for file management. For detailed descriptions of these functions, refer to the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Table 6-1. UBASIC File Manager Functions

Function	Description
mclose	Close a File Manager data file
mcreate	Create a data file
mcrunch	Compress free space in a cluster variant file
mdelete	Delete a record from a data file
merase	Mark a data file as erased
mfree	Check the available free space in the file area
minit	Initialize the working Variables
minput	Read a record from a data file
minsert	Insert a record into a data file
mopen	Open a data file
mprint	Write a record to a data file
msearch	Search for a data record from a data file
mterminate	Set the status flag in all File Directory Blocks to 'close'

General Rules

The following rules must be observed by any application that uses the File Manager.

1. Declare all the File Manager routines as 'far' routines and all the pointer input parameters to the File Manager routines as 'far' type. This is handled for you if you **#include FMGR.H** in your application.

2. Declare and allocate space for a File Manager work buffer. The work buffer must be initialized in order to put it in the default data segment where the File Manager expects its to be.

This is handled for you if you `#include FMGR.H` in your application. FMGR.H declares the buffer type as:

```
WORKBUFT wrkbuf = { 0 }
```

3. Declare and allocate space for a File Control Block (FCB) for every file opened. This structure is used to pass information between the File Manager and the application. Before **mopen** is called, the File Control Block must be initialized with the data file name, the data buffer size, and the data buffer address. Before **msearch** is called, the **offset**, **keyadr** and **'keylen'** fields in the File Control Block must be set up with the desired value. In addition, the application is responsible to update the **lastop** field in the File Control Block after every File Manager function.

This is handled for you if you `#include FMGR.H` in your application. FMGR.H declares the File Control Block type, FCBT, as:

```
FCBT fcb1;
```

For a detailed description of the File Control Block structure, refer to the *Structure Definitions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

4. Call **minit** before any other File Manager function.
5. Call **mcreate** before **mopen** for linked variant, cluster fixed and cluster variant files.
6. If the application does its own mapping of logical EMS pages into the page frame, preserve the logical page number mapped into physical page 3. The File Manager always maps in the first data segment to page 3.
7. Split resident applications must use the Small or Medium Memory Models and the resident version of the File Manager. Non-split resident applications must use the Large Model. They may use either a resident version or a linked-in version of the File Manager.



File Manager Data Structures

The File Manager uses a number of data structures to pass parameters and data. Refer to the *Structure Definitions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual* for detailed descriptions of these structures.

As for all Symbol library functions, if you pass a pointer to a structure, that structure must be packed. Otherwise, the library function will access data in the structure incorrectly.

In order to access any File Manager data structures, e.g., the free list, the file directory blocks, etc., the user can use the information stored in the work buffer that is passed to **minit**.

Global Write Protection

If the application creates its own linked list and needs to write to the allocated block, the global write protection must be disabled before any write operation. To accomplish this, add the following lines to the application source file:

```
#include <3000\bios.h>
BiosSetGlobalWrtProt(0);
```

The write protection should be enabled with the following line when all write operations are finished:

```
BiosSetGlobalWrtProt(1);
```

Loading the File Manager

The UBASIC File Manager functions are provided by a library that you link with your application. There are two versions of the library:

- **fmgrtsr.exe**

This is the executable version. It can be loaded as NVM resident or executed as a TSR.

- **fmgr.lib**

This is the linkable version. Like any normal library function, it must be linked with your program.

The resident version keeps the application size small and allows several applications to share a single copy of the library, but it may be less time efficient.

When the executable version of the File Manager is used, either as resident or as a TSR, calls to File Manager functions are made by an interrupt call. The interrupt calls are handled for you by the File Manager Interface Library, **fmgrintf.lib**. Calling a resident function is done in the same way as calling a linked-in function.

Every function in **fmgr.lib** has a corresponding function in **fmgrintf.lib**. Accordingly, the library being used is transparent to the application.

Note: If you are writing your application as a split-resident program, use the resident version of the File Manager. The application should be compiled as a small or medium model and linked with the File Manager Interface Library.

Installing the File Manager Library

The following procedures show how to set up the File Manager.

Using the NVM Resident Version:

1. Load the resident version of the File Manager Library, **fmgrres.exe**, as a resident program in the NVM by entering the following command to the User Configuration Tool (USRCFG.EXE):

```
/r c:\3000\files\fmgrres.exe
```

(For a detailed description of the User Configuration Tool, refer to *Building an NVM Image* in the chapter on *NVM Configuration* in this guide.)

2. Load the image into the NVM.
3. Cold boot the terminal to load **fmgrres.exe** as a resident program.
4. In the application source file, add the following line:

```
#include <3000\fmgrintf.h>
```



5. Compile and link the application with the following command (for a non-split resident application):

```
cl /AL /Zp filename.c
```

For a split resident program, do one of the following:

```
cl /AS /Zp filename.c
```

```
cl /AM /Zp filename.c
```

6. Download the application program and execute.

Using the RAM Resident (TSR) Version:

1. Load **fmgrtsr.exe** into the terminal by either:
 - downloading it into the RAM disk, or
 - including it as a user file in the NVM image response file and load the image. Use this line in the USRCFG response file:

```
/u c:\3000\files\fmgrtsr.exe
```

(For a description of the USRCFG response file, refer to *Building an NVM Image* in the chapter on *NVM Configuration* in this guide.)

2. Execute **fmgrtsr.exe** once to make it resident in RAM.
3. In the application source file, add the following line:

```
#include <3000\fmgrintf.h>
```

4. Compile and link the application with the following command:

```
cl /AL/Zp filename.c
```

5. Download the application program and execute.

Using the Linked Version:

1. In the application source file, add the following line:

```
#include <3000\fmgr.h>
```

2. Compile and link the application with the following command:

```
cl /AL /Zp filename.c
```

3. Download the application program and execute.

File Manager Sample Programs

For a sample program that illustrates the use of the File Manager, refer to C:\3000\SAMPLE\FILE_API\FILE.C in the ADK.

DOS File Management APIs

In addition to the UBASIC File Manager, you can use the following two file management APIs in Series 3000 application programs:

- Microsoft Run-Time Library File Functions
- DR DOS Library File Functions



Microsoft Visual C Runtime Functions

The MSC Runtime Library File Functions are divided into three categories:

- Stream Routines
- Low-Level Routines
- MS-DOS Interface Routines

The stream routines are the highest level functions; the DOS interface routines are the lowest level functions. The stream routines produce more code than the low-level routines, but this disadvantage is generally made up for in convenience. A major convenience of the stream functions is their ability to handle data buffering and formatting.

Portability Issues

You lose some portability when you move from the stream routines to the low level routines. If you require portability, use the ANSI compatibility routines.

When you use Symbol Technologies' APIs, however, your application is limited to Series 3000 terminals. There is a trade off in selecting the level of API you use. A low level API is more efficient and has more control, but it is difficult to program and is not portable. A high level API has less control and is less efficient, but is easier to program and is portable.

MS DOS Interface Routines

The MS-DOS interface routines give you direct access to DOS calls. For the most part, there is not a big difference between the low level routines and the MS DOS interface routines. The low level routines do have more overhead, for example, to maintain a file pointer.

The MS DOS interface routines give you capabilities beyond simply reading and writing file data, such as:

- Finding the date and time a file was accessed and modifying that date and time.
- Searching through a directory to find a particular file name or a file name that matches a pattern.

Overhead is cut down to a minimum with these functions.

Do not mix the MS-DOS interface routines with the higher-level routines when reading and writing file data.

DR DOS Library File Functions

The DR DOS interface Library file functions are on the same level as the Microsoft Library low-level file functions but usually take parameters that are different from those taken by the MSC interface routines.

The DR DOS Library includes some functions not provided by the Microsoft Visual C Library.

These are Symbol functions, and have been carefully written to provide the absolute minimum of overhead; they are often more efficient than the Microsoft routines. Table 6-2 lists the DR DOS Library File functions. More detailed descriptions of the functions listed in their table can be found in the *DR DOS* chapter of the *Series 3000 Application Programmer's Reference Manual*.

Table 6-2. DR DOS Library File Functions

Function	Description
DosCreate	Creates a file via DOS int 21h (function 0x3C)
DosOpen	Opens a file or device via DOS int 21h (function 0X3D)
DosClose	Closes a DOS file or device associated with handle
DosRead	Reads from a file or device associated with handle
DosReadLine	Uses the DosRead function to read maxlen characters from the file associated with handle
DosWrite	Writes to the file or device associated with handle
DosWriteLine	Uses the DosWrite function to write the characters pointed to by bufptr to the file associated with handle



Sample Programs

The following sample programs are provided in the C:\3000\SAMPLE\FILE_API directory of the ADK:

FSTRMLVL.C	uses the Stream Routines
FLOWLVL.C	uses the Low-Level Routines
FDOSLVL.C	uses the MS-DOS Interface Routines
FDOSLIB.C	uses the DR DOS Library Routines

These programs perform typical file I/O operations. For ease of comparison, each sample program calls the same five functions which perform the API specific operations. The five primary functions are:

create_dataset()

Prepares the data storage mechanism so that data can be written to it.

store_into_dataset()

Takes the input data and stores it into the prepared data set. These examples use a data set that is compiled in as static data (from an #include file). Usually, the program would collect this data from the keyboard or scanner.

retrieve_from_dataset()

The supplied data set is created in chronological order. This function prints out the values in the data set, ordered by the primary key.

The File Manager provides an easy way of doing this.

The other sample programs scan through the entire list n times, where n is the number of items in the data set; n^2 operations are required. The first pass finds the smallest value in the list. Each succeeding pass finds the next smallest greater value. The result is printed at the end of each pass.

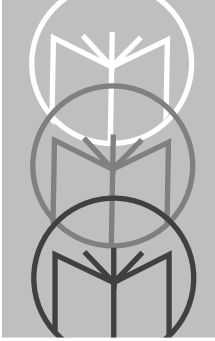
destroy_data_set()

Deletes all the data stored in the data set and reclaims the memory space it used.

print_freespace()

Prints the number of bytes currently available in the area in which the data set is being stored. This may not reflect writes that have been buffered until the buffer is actually stored into the data set.





Chapter 7

Managing Memory

Chapter Contents

Introduction	7-3
Program Memory Requirements.....	7-4
Maximizing Memory Use	7-6
Handling Low-Memory Situations	7-8
Managing Available Memory	7-9
EMS	7-13
Sample Programs.....	7-16



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>DR DOS</i> <i>UBASIC Record Manager</i>
<i>Series 3000 System Software Manual</i>	<i>Expanded Memory Manager</i>

Depending on the terminal configuration, you may have the following amounts of memory available for your application program:

- NVM (128K or 256K), used to store NVM Disk resident programs, split-resident programs, resident programs and libraries
- Conventional RAM (256K or 640K), used for the TPA (Transient Program Area) and RAM disk
- EMS RAM (0.5M, 1.5M, or 3.5M), used for the RAM disk or as available EMS pages

In any case, the amount of memory available for your application is limited. By carefully managing how you use the available memory, however, the Series 3000 terminals provide enough memory for just about any application.

As with any DOS application, it is important to monitor available memory in the terminal. If the application does not monitor memory, it is easy for an operator to spend all morning collecting data only to have the terminal lock up when it runs out of memory. Or, if you have an application full of data that you need to transmit to the host, it is important that you have memory reserved for a communications buffer.

This chapter focuses on methods of managing the way your applications use available memory . It also describes some ways of solving memory limitations .



Program Memory Requirements

The memory requirements of your application can be checked using several tools, depending on the area or the type of memory you are checking.

Determining RAM Requirements

Use the FREEMEM utility included with the Hypertext examples to find out how much TPA memory your program needs to run.

Run FREEMEM on the terminal as a TSR. Then run your application program and cause it to use the maximum amount of memory it needs for all its functions and features. Make sure you include data transmission and storage in your test.

During the application run, FREEMEM monitors the amount of TPA memory that is allocated to and by the program. When the program exits, FREEMEM displays the largest amount of TPA used during execution.

Note: FREEMEM does not remove itself from memory.
You must reboot to remove it.

Determining RAM Disk Requirements

Use the FREEDISK utility included with Hypertext examples to find out how much RAM disk space is used by your program.

Run FREEDISK on the terminal as a TSR. Then run your application program. Use all the features and functions of your program that allocate RAM disk space. Make sure to collect as much data as is required by the program specification.

When your program terminates, FREEDISK displays the largest amount of RAM disk space used by the application. It determines usage as the difference between the amount of free RAM Disk space available when the program is loaded and the smallest amount of free RAM Disk space available during the run. For this reason, begin with an empty RAM Disk. If the application begins by deleting a large file (e.g., old data), the FREEDISK report is unreliable.

Determining NVM Requirements

This depends on how big the programs (and libraries) are that you are going to store in the NVM and what drivers you are going to use.

The only way to determine accurately the amount of NVM your application requires is to create the program and build a NVM configuration file including all the components you would like in the NVM image. Include all the necessary libraries and drivers. The USRCFG utility (User Configuration Tool) generates a .CFL file that lists the sizes of the components loaded, the total amount of NVM memory used, and amount of NVM of memory still available.

If your program is too large to fit into the NVM you have available, adjustments are necessary. Utilities are available to compress the size of .EXE files and then expand them when copied into the TPA for execution. You can use the Microsoft **EXEPACK** program to do this, but there are other utilities which are more efficient.

In some cases, the program may be too large to fit into the NVM even after compression. In this case, you will have to load the program differently among the terminal memory areas. For instance, you can configure the terminal with a RAM disk and load the program there. This approach reduces the amount of NVM required, but increases the required amount of RAM and EMS for the RAM disk.

Refer to the *USRCFG.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual* and to the Building an NVM Image section of the NVM Configuration chapter in this guide for more information on the User Configuration Tool.

Determining EMS Requirements

EMS (Expanded Memory Standard) memory can be used for RAM disk and is available for EMS pages.

If your RAM disk and TPA requirement is greater than 640K, at least part of your RAM disk must be in EMS.

If your program uses the EMS driver to access EMS memory, you must add this to the EMS memory used by the RAM disk.



Distributing RAM between RAM Disk and TPA

If you configure a RAM disk (usually using INIT.EXE) and the terminal has no EMS memory, then one-fourth of the conventional RAM that would go to the TPA is used instead for the RAM Disk, by default.

In order to allocate as much memory as possible to the RAM disk and still have enough memory left in the TPA to run your program, use the FREEMEM program to determine the memory needed by the application. Then use the BLDINIT utility to allocate the rest of RAM to the RAM Disk.

Refer to the *Running BLDINIT* section of the chapter on *Terminal Initialization* in this guide for additional information on the BLDINIT.EXE and the INIT.EXE utilities.

Maximizing Memory Use

This section gives some suggestions for how to develop a program to fit in a terminal with limited resources. This requires fitting the most program into the least memory.

Using Program Residency Strategies

You have developed an application program on a development terminal. You used FREEMEM to find out how much TPA it uses and USRCFG to find out how much NVM it uses.

Your existing terminals aren't big enough. What do you do now? The solution is to use another type of program that requires less NVM or TPA.

If you don't have enough NVM, you can:

- Use a split-resident program, for a small gain
- Use a compressed program, for a large gain
- Use a RAM disk resident program.

If you don't have enough TPA, you can:

- Use a split-resident program, for a large gain
- Use chained programs, for a medium-to-large gain

- Use a resident library to reduce code duplication
- Use EMS for the RAM Disk to free up TPA.

NVM Disk

This is the easiest strategy. The programming is relatively simple, and the program loads quickly. In its basic form, it also uses the most TPA and the most NVM.

An NVM disk program can be compressed, however. It can also be split into smaller programs that are chained together. These programs can share a resident library to save additional space.

NVM Split Resident

A Split Resident program uses much less TPA than an NVM Disk program because the code stays in NVM. It also uses slightly less NVM because the header information pertaining to the code segment is removed when the program is split.

Two drawbacks in using a Split Resident Program are:

1. the complicated nature of the program
2. the inability to compress split programs, because the code relocation addresses are resolved at USRCFG time

Refer to the *Writing Resident Programs* chapter in this guide for information on writing split-resident programs.

Compressed Programs

The size of your .EXE file can be reduced by 20%-60% through the use of a compression utility. However, this is dependent on the compression program being used and the nature of the file being compressed.

Less NVM is required, but the same amount of TPA is needed.

Compressed programs take slightly longer to load, because they are decompressed when they are loaded into the TPA.



RAM Disk Resident

If your program is too large to fit into NVM, you can download the program onto the RAM disk. You will probably need EMS memory to increase the size of the RAM disk both for program and data storage, since the program must reside in the RAM disk and be copied to the TPA.

Handling Low-Memory Situations

There are a number of ways to handle low memory situations in an application. The best solution depends on the situation itself.

For example, the application may need to download data from the host but have too little memory to store the full amount. Depending on the application, loading some of the data may be a solution. If not, the transmission should never begin. In either case, the host must be notified of this before the transmission begins.

In an application that does data collection before doing a batch data transfer, a cutoff point for data should be established that will reserve enough memory for the communication session. When that cutoff point is reached, the application program should inform the operator that no more data can be collected until the current data have been transmitted to the host. It can then switch to the communications routine. Once the current data have been transferred to the host, the data files can be purged and data collection resumed.

It is the responsibility of the application programmer to anticipate and plan for low-memory situations that may arise and to prevent data loss as a result.

Managing Available Memory

A number of resources are available to help you monitor and manage the terminal RAM usage. The functions described below provide detailed control beyond the scope of most application programs. You may find them useful, however.

For detailed descriptions of the functions mentioned here, refer to the *Series 3000 Application Programmer's Reference Manual* or to the appropriate Microsoft Visual C, DR DOS, or MS DOS documentation.

Microsoft Visual C Library Functions

The Memory Management Microsoft Run-Time Library Functions are divided into two categories:

- Memory Allocation Routines
- MS-DOS Interface Routines

When DOS allocates memory, it allocates it in paragraph-size chunks (16 bytes); there is a one-paragraph overhead associated with each block of allocated paragraphs.

The Memory Allocation Routines call the MS-DOS Interface Routines to get a big block of memory. They then break this block into smaller blocks, and use pointers to link these small blocks together. If you allocate a small block of memory, you will only have two (or four) bytes of overhead per memory block. The Memory Allocation Routines themselves, however, will cost you some overhead.

If your program needs a large chunk of memory, which it will subdivide itself, the MS-DOS Interface Functions would be most useful.

DR DOS Library Memory Functions

These functions are similar to the Microsoft MS-DOS Interface routines `_dos_allocmem` and `_dos_freemem` and make the same DOS calls.

The DR DOS Library functions return a pointer to the allocated memory.



There are two DR DOS memory functions:

DosAllocMem	Allocates memory from the far heap
DosFreeMem	Frees a previously allocated block of memory

For more information on these functions, refer to the *DosAllocMem* and *DosFreeMem* sections of the DR DOS chapter in the *Series 3000 Application Programmer's Reference Manual*.

DR DOS File Management Functions

Get Available Disk Space

The Microsoft Visual C++ Run-Time Library contains `_dos_getdiskfree`. Pass this function the drive number (4 for drive D, the RAM disk) and pass a pointer to the diskspace structure. It fills the structure with `avail_clusters`, `sectors_per_cluster`, and `bytes_per_sector`. By multiplying these three values together, you'll get the number of available bytes on the disk. By multiplying `sectors_per_cluster` by `bytes_per_sector`, you can determine the minimum allocation unit size.

Flushing File Buffers

When you write data to a file, DOS puts the data you have written into a buffer. DOS always allocates full clusters, and never less than one cluster to a file. The stream routines have the function `fflush` which flushes the specified buffer. You should flush the buffer before calling `_dos_getdiskfree` on a file-by-file basis. The function `flushall` flushes all streams at once.

You will probably want to pair the flush functions up with some additional logic to make them fit into the way your program behaves with regard to free memory detection.

DR DOS Disk Management APIs

There are two DR DOS Disk Management APIs:

- IOCTL Functions
- Absolute Disk Functions

We only mention these to warn against them.

Instead of using the IOCTL functions, use the Memory Manager, which handles the functions for you.

Instead of the absolute disk functions, use the DOS file system.

If your application requires a large, linear, data space, and you want it to have high performance with a low overhead, you can use the Absolute Disk Functions to take over the entire RAM Disk. You will have to devise your own memory allocation scheme. You will also have to devise a way to monitor memory allocation.

The IOCTL functions allow you to use the RAM Disk as memory, but you will have all the problems of write protection and EMS access to deal with, and you will still have to provide your own memory allocation (and memory monitoring) schemes.

Memory Manager Routines

The UBASIC Memory Manager provides a set of functions used to manage the RAM disk. If your application is written to use the Memory Manager functions, it will run on any terminal regardless of how the RAM disk is configured.

Most of the Memory Manager functions are designed to manipulate data blocks in the form of a linked list. For every data block, there is a three byte overhead used for the link. If you anticipate a lot of random deletions and insertions of the data blocks in your application, and a linked list is the preferred data structure, the Memory Manager is most useful.

The Memory Manager allocates memory from the RAM disk, one segment at a time. The size of a segment is dependent on the value passed to the `minit` routine. During initialization, the Memory Manager allocates a segment of memory from the RAM disk as the internal working space. The remaining space in the segment is used later by the block allocation routines.



When the free space in a segment runs out, the Memory Manager allocates another segment from the RAM disk. If the available memory size is smaller than the segment size, the Memory Manager may generate an insufficient memory error, even if there is enough memory for the requested block.

The following memory management functions are described in greater detail in the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*:

minit	Initializes the Memory Manager working space
blk_alloc	Allocates a block
blk_delete	Deletes a block from a user linked list
blk_free	Frees a block to the free linked list
blk_insert	Allocates and inserts a block into a user linked list
blk_search	Searches a user linked list for a matching record, or for the next highest values compared to a search key
blk_traverse	Traverse a user-linked list to the desired block number

File Manager Functions

Each file directory entry in the UBASIC File Manager points to the blocks which contain the data for the records of that file, and these blocks are all linked together. Depending on the data file type, these blocks may contain either one record per block (Linked Fixed or Linked Variant), or a multiple number of records per block (Cluster Fixed or Cluster Variant).

If the file type is Cluster, the blocks may contain unused space owned by a file. For Cluster Fixed files, only the last block in the linked list may be partially filled as the other blocks should all be fully occupied with a fixed number of records. For Cluster Variant files, depending on how randomly the records are inserted or deleted, any memory block in the linked list may be partially occupied.

Get Available RAM Disk Space (**mfree**)

The File Manager **mfree** function allows an application to check for available space in the RAM disk. Depending on the value of the input parameters, **mfree** returns either the total number of free bytes or the maximum number of records that can be written to the file space.

The value returned by **mfree** should be taken as approximate rather than as exact.

The **mfree** function is described in greater detail in the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*

Reorganize RAM Disk Space (**mcrunch**)

The **mcrunch** function reorganizes file records and frees unused space so the Memory Manager can reallocate this space. This function can be used on a Cluster Variant file only. For more information on the **mcrunch** function, refer to the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

EMS

EMS is primarily used on Series 3000 terminals for RAM disk. When configured in this way, there is little you need to do beyond the managing the RAM disk as described earlier in this chapter.

Unless you are porting an existing application that uses EMS, we do not recommend using EMS directly. Use the File Manager instead because it automatically uses both RAM and EMS memory in the RAM disk interchangeably and homogeneously.

If file management is not what you need, the Memory Manager may meet your needs. It also uses both EMS and conventional memory interchangeably. By using the Memory Manager functions, your application will run on a terminal whether or not it has EMS memory.



Symbol Technologies does not provide an EMS API for C. The *Expanded Memory* chapter in the *Series 3000 System Software Manual* describes the Expanded Memory Manager (EMM3000.SYS) driver and how to load and access it. There are many off-the-shelf books on C Programming with EMS; for example, *Inside DOS: A Programmer's Guide*, by Michael Young. Refer to one of these books for instructions on using EMS from your program.

Monitoring EMS

An EMS driver call gives you the number of free EMS pages. It is up to your application to keep track of the pages used, and the memory available in those pages.

Why Use EMS

There are several reasons you might want to use EMS directly rather than as RAM disk:

- The ability to access two megabytes of storage, with four 16K blocks of memory directly accessible at a time, can be very valuable and is fast.
- If your application does a lot of data manipulation, operating on huge data sets, you can write an extremely efficient program by using EMS directly.
- If you have an application developed for EMS, you can port it to a Series 3000 terminal with EMS memory easily. The Symbol EMM driver is compatible with most C Language EMS API.

The RAM disk driver and the Symbol EMS driver cooperate in sharing EMS memory. The EMS driver finds out how many pages of EMS memory are reserved from the RAM disk and treats these pages as all the EMS there is. It then makes these pages available to the application through the standard EMS calls.

How to Use EMS

To use EMS:

1. Hold back some EMS pages from the RAM disk. The EMS memory not used by the RAM Disk can then be used by the EMS Driver.
2. Use the standard EMS operations.

By default, the RAM disk uses all EMS in the terminal unless some is reserved. To reserve EMS, use the BLDINIT utility and specify the amount to reserve. Refer to the chapter on *Terminal Initialization* in this guide for further information on BLDINIT.EXE, especially the section on *Running BLDINIT*.

Use Standard EMS Operations

The *Expanded Memory Manager* chapter in the *Series 3000 System Software Manual* describes how to control EMS through the EMM3000 driver. This control is available only through assembly language programming.

To use an EMS API for C:

1. Load and activate the EMM driver (EMM3000.SYS).
2. Determine the presence of EMM using an EMM call.
3. Find the page frame using an EMM call.
4. Allocate EMS page(s).
5. Map EMS page(s) into the page frame
6. Read and write to EMS pages
7. Keep track of allocated EMS page(s) while you are using them.
8. Unmap the EMS pages
9. Deallocate the EMS pages



Sample Programs

The following sample programs illustrate a number of memory management tasks using the tools described in this chapter. The programs are in the C:\3000\SAMPLE\MEM_API directory of the ADK.

Microsoft Run-Time Library Memory Functions

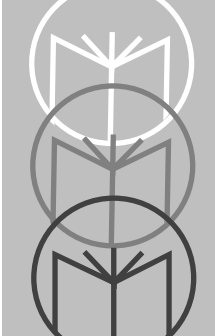
- | | |
|------------------|---|
| MMALLOC.C | Memory management using malloc, free, etc. |
| MDOSAL.C | Memory management using _dos_allocmem, etc. |

DR DOS Library Memory Functions

- | | |
|-------------------|---|
| MDRDOSAL.C | Memory management using DosAllocMem, etc. |
|-------------------|---|

Memory Manager Functions

- | | |
|--------------|---|
| MEM.C | Memory management using the Memory Manager. |
|--------------|---|



Chapter 8

Display Handling

Chapter Contents

Introduction	8-3
Improving Display Performance	8-12
Cursor Control	8-16
Miscellaneous Screen Operations	8-18
ANSI Support	8-19
Display Fonts	8-21
Font Builder	8-22
Making a Font TSR	8-31



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>Device Drivers</i> <i>BIOS Library Functions</i> <i>DR DOS</i>
<i>Series 3000 System Software Manual</i>	<i>ROM BIOS</i>

Due to their compact size, Series 3000 terminals use small display screens. Application programmers must take this small screen size into account in the design of their applications for the terminal. An application written for a full size screen (80 by 25 characters) will be hard for the operator to use, while an application that takes into account the small screen can be relatively easy to use.

The Series 3000 BIOS provides tools that use a scheme of *physical*, *virtual*, and *logical* display screens to manage effectively the small display on the terminal. A number of BIOS video services are provided to control the size and position of these screens.

Briefly, the three screens function as follows:

- The *physical* screen provides a fixed size view port to the virtual screen.
- The *virtual* screen actually contains the video data, stored in RAM.
- The *logical* screen defines the line wrap and screen scrolling boundaries.



Physical Screen

The *physical screen size* specifies the number of rows and columns supported by the display hardware. The physical screen sizes currently employed on Series 3000 terminals are:

- 8-line by 20-character display on Series 3300 and 3800 terminals
- 8-line by 40-character display on Series 3900 terminals
- 4-line by 20-character or 8-line by 20-character display on Series 3100 and Series 6100 terminals
- 16-line by 21-character display on Series 3500 and Series 6800 terminals

These sizes may differ if you use a font other than the default. The number of pixels remains the same.

The system BIOS determines the terminal screen size upon cold boot by a code associated with the display. An application can get the physical screen size by calling the **BiosGetPhyScrSize** function. This function is described in detail in the *Function Descriptions* section of the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Virtual Screen

The *virtual screen size* is determined by the video RAM, the amount of memory required to write data to the LCD display. Series 3000 terminals do not have a designated video RAM area, so a buffer is required to emulate video RAM.

At a system cold boot, the BIOS allocates enough RAM to emulate video RAM. The default is to allocate enough RAM to match the physical screen size. If a larger logical screen size is required, more RAM must be allocated at system cold boot, usually in INIT.EXE built by the BLDINIT utility. Refer to the *Terminal Initialization* chapter in this guide for a description of the BLDINIT utility, especially the section on *Running BLDINIT*.

The virtual screen size can be up to 80x25, the standard size of a PC screen. (See Figure 8-1.) This allows PC applications to be ported to a Series 3000 terminal. Provision for moving the physical screen around the virtual screen needs to be added to allow the full virtual screen to be viewed.

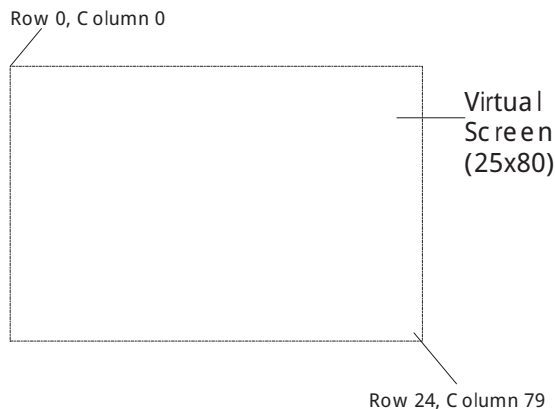


Figure 8-1. Virtual Screen, Maximum Configuration

An application can allocate more video RAM, and so specify a larger virtual screen area, by calling the **BiosSetVirScrSize()** function (described in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual*). The larger virtual screen can be used for multiple display pages. The virtual screen size cannot be defined smaller than the physical screen size. The maximum virtual screen size is 80 columns by 25 rows (Figure). You cannot increase the virtual screen size or the number of video pages to a combination that requires more video RAM than was allocated at boot-up time.

The upper left corner of the virtual screen is always positioned at row 0, column 0. Only data written to a row/column position within the virtual screen is retained in video RAM. Any data written outside the virtual screen boundaries is lost.



Displaying Virtual Screen Segments

Series 3000 terminals only display data from the virtual screen that is within the current boundaries of the physical screen size. However, the physical screen can be moved to display any part of the virtual screen. The physical screen is considered a moveable item in the context of Series 3000 operations (Figure 8-2)

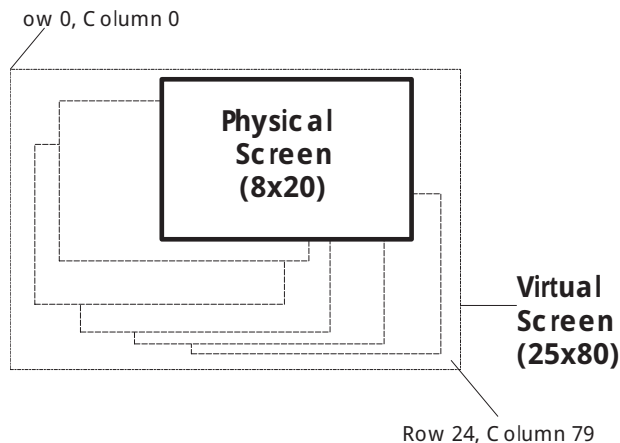


Figure 8-2. Physical Screen, Movable

Position the physical screen by calling the **BiosSetPhysicalPos** function, specifying the row and column coordinates of the upper left hand corner (origin) relative to the virtual screen. However, an edge of the physical screen *cannot* extend past the edge of the virtual screen. For example, if using a virtual screen size of 25 rows by 80 columns and an 8 by 20 physical screen, the upper left coordinates of the physical screen cannot be positioned past row 16 or column 59 (Figure 8-3). For a description of the **BiosSetPhysicalPos** function, refer to the *Function Descriptions* section of the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Manual*.

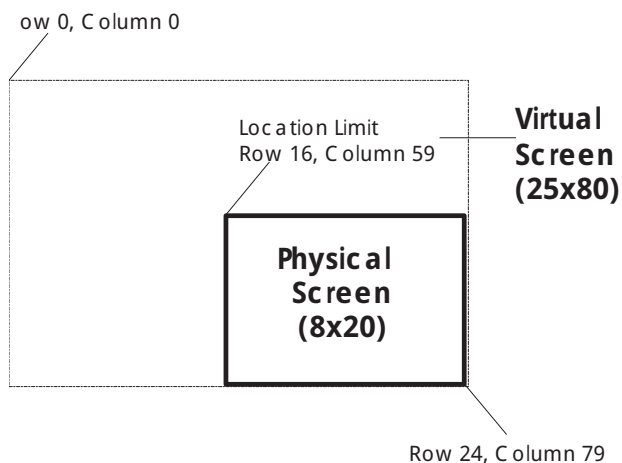


Figure 8-3. Physical Screen, Location Limit

Multiple Display Pages

Rather than use the virtual screen as a single space, an application can divide it into a number of display pages. The number of display pages is set using the **BiosSetVirScrSize** function (see the *Function Descriptions* section of the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Manual*). By default, only one display page is allocated at system cold start.

Each page is identical in size. A page can be selected and/or modified without changing the contents of other pages.

To determine the current display page use **BiosGetDisplayPage**. To display a different page use **BiosSetDisplayPage**. For descriptions of these functions, refer to the *Function Descriptions* section of the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Manual*.



Logical Screen

The *logical screen* defines the column at which a display line wraps and the row at which the screen scrolls. For example, when you write data to the display, data is written from left to right until it reaches the logical screen width. When the next character is written, the line wraps, displaying the character in the first column of the next line.

When the cursor reaches the limit of the logical screen length, the data wraps around to the first position of the next row of the virtual screen (Figure 8-4).

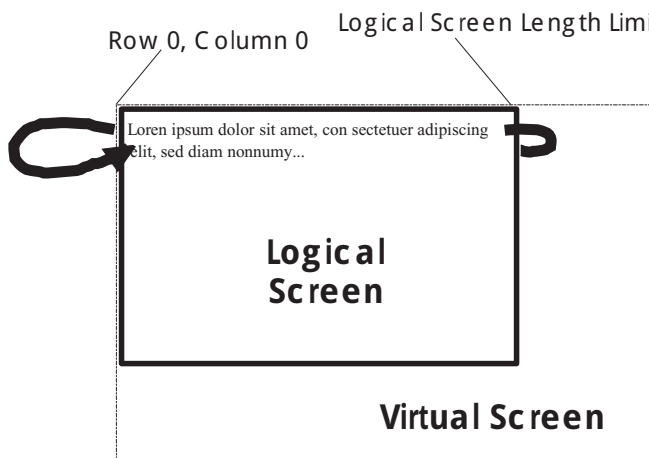


Figure 8-4. Cursor Wrap

If the next row is larger than the current logical screen length, the screen scrolls down (see Figure 8-5). The upper left corner of the logical screen is always positioned at row 0, column 0, of the virtual screen.

The default logical screen size is the physical screen size. To specify a different logical screen size, an application can make a call to the **BiosSetLogScrSize** function (see the *Function Descriptions* section of the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Manual*).

Note: The logical screen applies only when a program uses Series 3000 BIOS services that support these features (scrolling and autowrap), such as “Write

Character As TTY."

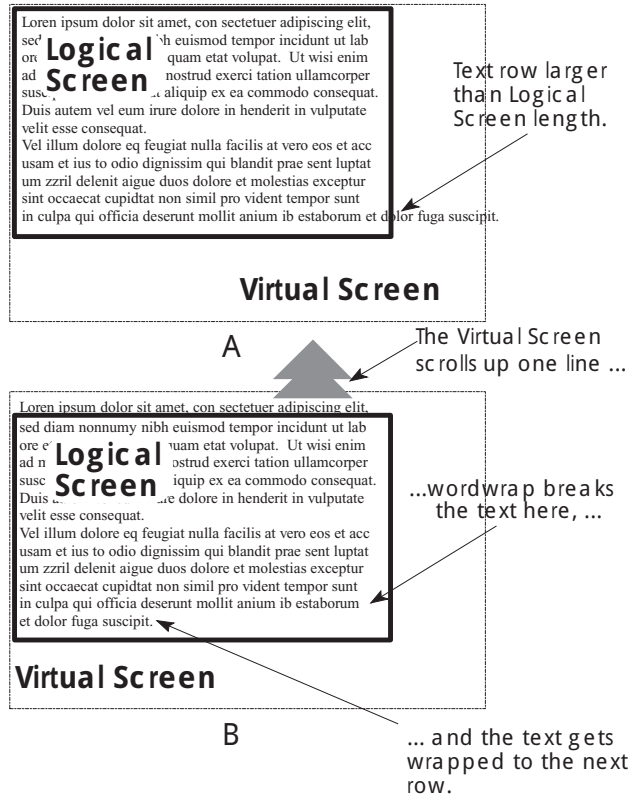


Figure 8-5. Logical Screen Scrolling



Using the Screens in an Application

Note that if the logical screen is larger than the virtual screen, a line of data will not wrap until it is past the boundary of the virtual screen. All data written past the edge of the virtual screen is lost (see Figure 8-6).

If the logical screen is larger than the physical screen, a line of data will not wrap until it is past the boundary of the physical screen. This data will not be displayed because the terminal only displays data within the boundaries of the physical screen. As long as the virtual screen is at least as large as the logical screen, you can display this data, but you will have to include in your application a method for repositioning the physical screen.

To maintain the standard 25x80 PC screen size in your application, set both the virtual screen and the logical screen to 25x80. Then include a method for moving the physical screen around on the virtual screen.

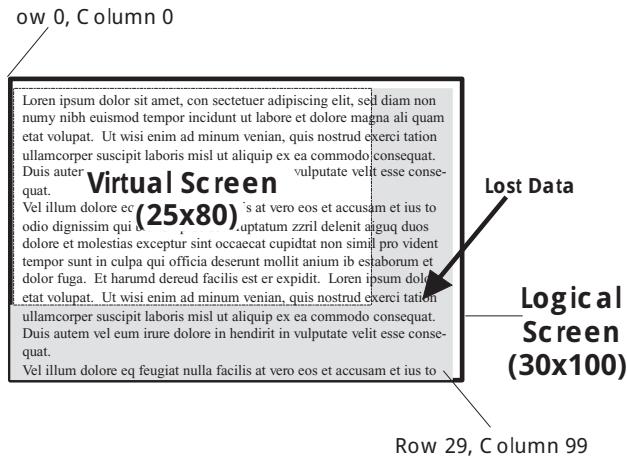


Figure 8-6. Screen Data Loss

It is generally better to write your application so that screens can be entirely displayed on the terminal screen at once. To do this, make all the virtual and logical screens the size of the physical screen. This is the default setting at cold boot. If your application will be run on Series 3000 models with different screen sizes, you will want to include a screen size detection routine and use it accordingly. You may wish to use a smaller logical screen to create a small window where text wraps and scrolls separately from the larger virtual screen.

To use every position on a Series 3000 screen, make the physical and virtual screen the same size and make the logical screen size one character wider. This prevents the screen from wrapping or scrolling after the last character is written to the lower right corner. You will have to use a line feed/carriage return at the end of each line of text to move the cursor to the next line.



Improving Display Performance

Applications written in C typically employ stream functions which include all the formatting capabilities provided by **printf**. These functions work well on large display screens, but they include a lot of code and run slowly. While the familiar stream functions are available, they are not well suited for output to the Series 3000 terminals.

Display performance is compromised in Series 3000 terminals by the use of LCD displays. This is especially evident when an application has to scroll an entire screen. It is generally better to use more efficient code and to selectively update the parts of the screen.

C and DOS Routines

The simplest improvement is to employ C and DOS routines that require less overhead.

The Microsoft Visual C++ console routines are similar to the stream routines, except that they go only to the console and they do not provide buffering. These routines are more efficient than the stream routines and so provide some improvement to Series 3000 display operations.

The following console routines, declared in **conio.h** are available for use on the Series 3000:

cprintf	Writes formatted data to the console
cputs	Writes a string to the console
putch	Writes a character to the console

If your application requires a large amount of linear output with very limited formatting, the following MS DOS functions also provide a performance improvement over the stream functions:

close	Closes a file
open	Opens a file

write	Writes data to a file
_dos_open	Opens an existing file
_dos_write	Sends output to a file
_dos_close	Closes a file

The following DR DOS file routines provide the same functionality as the Microsoft routines, but they are usually even more efficient:

DosOpen	Open a file to the console
DosClose	Close the console
DosWrite or DosWriteLine	Write to the display

For detailed descriptions of these DR DOS routines, refer to the *Introduction to the DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Symbol BIOS Routines

Because of the special demands and features of the Series 3000 display, a large number of routines are included in the ADK, providing easy, direct access to the system BIOS. These routines have the least amount of overhead, and so they produce fast, compact programs.

In addition to basic display functions, the Symbol BIOS library provides routines for:

- Cursor control
- Screen scrolling
- Windowing
- Display sizing and paging
- Viewing angle/contrast adjustment
- Display backlighting



You can mix the BIOS routines with the higher-level routines as desired for a combination of efficiency and convenience.

The following sections describe several common display control procedures and how to use the BIOS functions to perform them. Refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for detailed descriptions of each BIOS routine. The functions described there call ROM BIOS services that are described in the *ROM BIOS* chapter of the *Series 3000 System Software Manual*.

Windowing

Windowing is a technique for holding one or more whole display screens in memory and switching the screen displayed. Windowing provides a quick way to repeat a screen once it has been previously displayed, eliminating the need to redraw the screen.

Windowing involves two steps:

1. Saving the display contents as a window
2. Using rapid pop-up to display the window

To save a current screen as a window, use the **BiosTextRectToMem** function. You must define the text rectangle to select and supply a buffer.

BiosTextRectToMem then saves the character/attribute pairs from the rectangle into the buffer.

Once you have saved a rectangular area, you can erase it by scrolling it up zero lines.

To restore the window, use **BiosMemToTextRect**, specifying the buffer containing the window to display.

Windowing can also be used for special effects, such as displaying blinking messages. To do this, display a message on a known area of the screen, select the rectangle and save the text to the buffer. Make the message blink by alternately displaying and clearing the message.

For detailed descriptions of the **BiosTextRectToMem** and **BiosMemToTextRect** functions, refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual*. These functions call services that are described in the *Extended Video Services* section of the ROM BIOS chapter in the *Series 3000 System Software Manual*.

Display Pages

Display pages are multiple virtual screens. When you switch display pages, the physical screen will show that portion of the new virtual screen that is under the physical screen.

You can write only to the currently active page.

The BIOS Library function **BiosSetDisplayPage** selects the active display page. The **BiosSetVirScrSize** function sets the number of display pages (a maximum of 8). The functions are described in detail in the *BIOS Library Functions* chapter of the *Series 3000 Applications Programmer's Reference Manual*. They call services that are described in the *Extended Video Services* section of the ROM BIOS chapter in the *Series 3000 System Software Manual*.



Cursor Control

Applications usually turn the cursor on and off while they are running, especially when they are displaying menus and using a moving menu bar instead of the cursor.

Cursor Modes

BIOS control provides two cursor modes:

- *the standard hardware cursor*

This cursor blinks and is transparent.

- *a software cursor*

This cursor does *not* blink and is opaque.

The software cursor indicates keyboard shift states (unshifted, shifted, function, control, etc.) and battery condition. To set the cursor mode (i.e., hardware or software) use the **BiosSetCursorMode** function described in the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Cursor Shapes

The shape of the software cursor is configurable. The default software cursor for each shift state is shown below. Note especially the change for normal and low battery conditions.

The cursor shapes are all characters in the character set of the current font. You have two ways of modifying these shapes:

- Set up your own font. You can do this by using the Font Builder utility to modify the cursor characters in 68EU3000.FNT. Refer to the *Font Builder* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual* for detailed information on this utility.
- Modify the cursor translation table that defines which character will be used for each state. This requires some assembly language programming.

Keyboard State	Cursor Character	Low Battery
Unshifted	∨	▼
Shifted	∧	▲
Momentary Shifted	↑	⬆
Momentary Unshifted	↓	⬇
Control	⌂	Ⓒ
Alt	⌘	Ⓐ
Function	⌚	Ⓕ

Character Attributes

Series 3000 terminals support two character attributes:

- Normal Video 0x07 (dark pixels on a light background)
- Reverse Video 0x70 (light pixels on a dark background)

The video attributes are controlled by the following functions, which are described in detail in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual*:

BiosPutCharAttr

BiosGetCharAttr

BiosClrScr

BiosPutStrStay

BiosPutStrMove

These functions call services described in the *Standard Video Services* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*.



Miscellaneous Screen Operations

Clearing the Display

You can clear the current display screen in either of two ways.

- Use **BiosClrScr** to simply clear the entire screen.
- Use **BiosScrollUp**, specifying the window size to cover that portion of the virtual screen that you want cleared.

Refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for details on these functions.

Changing Viewing Angle

The optimal viewing angle of LCD displays is configurable by changing the contrast level of the display. The standard keyboard includes a key sequence allowing operators to change the viewing angle to adjust for lighting conditions. You may have reason to set the viewing angle or to provide a routine in your application for adjusting the angle. To do so, use:

- **BiosSetViewAngle** to set the LCD viewing angle
- **BiosGetViewAngle** to get the current viewing angle

Refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for details on these functions.

Backlighting

Terminals with LCD screens include a backlight to make the LCD display visible in low-light conditions. The standard keyboard includes a key sequence allowing the operator to turn the backlight on or off as required. You may have reason to control the lighting in your application. Use the **BiosSetBackLight** function to turn the LCD backlight on or off.

Because the backlight can quickly drain the battery if left on, you should set the backlight to turn off after a short period of time. Use **BiosSetBackLightTime** to set the backlight timeout. This function can also be used to disable the backlight and to enable or disable operator control.

For descriptions of these backlight control functions, refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual*.

ANSI Support

Series 3000 terminals can support ANSI display standards by loading the ANSI3000 driver. This driver is provided to enable applications to accept ANSI escape sequences in the data stream. Based on a captured escape sequence, the application can make appropriate BIOS calls to move the cursor, erase a portion of the screen, etc.

ANSI driver output is typically slow. ANSI3000 is provided primarily for applications that require an ANSI driver for maximum portability.

Refer to the *ANSI Compatibility Driver* section in the *Device Drivers* chapter of the *Series 3000 Application Programmer's Reference Manual* for a description of the ANSI3000 driver and the commands it supports.

Loading ANSI3000

ANSI3000 is provided both as a driver file (ANSI3000.SYS) and as a TSR (ANSI3000.EXE). Use only one version at a time.

To load the device driver version, include the following line in the terminal CONFIG.SYS file:

```
DEVICE=B:ANSI3000.SYS
```

The driver must be included as a user file in the NVM image.

The TSR version (ANSI3000.EXE) can be run from the command prompt or included in the terminal AUTOEXEC.BAT file to execute when DOS loads.

Small Screen Handling

ANSI3000 includes special features to support the small Series 3000 screens. For maximum flexibility, ANSI3000 checks the logical screen size of the terminal prior to each command. All screen size-specific operations then use this size. Changing logical screen sizes between operations works correctly.



Cursor Positioning

When an attempt is made to position the cursor to a location outside the logical screen of the terminal, ANSI3000 constrains the out of range row or column to either the highest or lowest acceptable value, as appropriate.

Screen Wrap

When the WRAP mode is set, ANSI3000 wraps a text line according to the logical screen size. If the WRAP mode is disabled, characters are discarded when they pass the logical screen boundary.

Invalid Command Handling

ANSI3000 screen control commands are a subset of the ANSI terminal escape sequence standard. If an unsupported command is entered, ANSI3000 prints all characters belonging to the command string just as if it were text. This provides a simple way for the programmer to see that the command was not executed.

Sample Programs

The ADK includes two demonstration programs for the ANSI3000, both in the C:\3000\SAMPLE\DISPLAY directory of the ADK:

ANSIDEMO.C	a simple program
DISPLAY.C	a more advanced demo program

Display Fonts

Series 3000 terminals come with a default font installed in hardware. The ADK also includes three standard soft fonts:

- 68PC3000.FNT - The standard PC font in a 6x8 format.
- 68EU3000.FNT - The European PC font
- 68ML3000.FNT - The International PC font

You can use these fonts or design a custom font using the Font Builder utility (FONTBLD.EXE) provided in the ADK. These fonts can then be loaded into the terminal either to replace the default font or as an application selectable font. Refer to the *Font Builder* section, below, in this chapter and to the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual* for more information on the Font Builder utility.

Font Formats

Series 3000 terminals support font files in three formats:

- .FNT - The .FNT file format is a font source format. This format has two uses. It is the editable format used by the Font Builder utility. It is also the format that is provided to BLDINIT utility for inclusion in the INIT.EXE file. Refer to the chapter on *Terminal Initialization* in this guide for more information on the BLDINIT utility.
- .EXE - The .EXE file format loads the font as a TSR. The file can be loaded onto a RAM disk or into NVM as either a user program or a resident program.



Font Builder

Font Builder (FONTBLD.EXE) is a utility that allows you to create and modify font files. It provides a graphical interface, including a bitmap representation of characters, that greatly simplifies the task of designing a font for Series 3000 terminals.

Note: The Font Builder is supported for CGA mode only. If problems occur when using an EGA or VGA monitor, check your graphics controller manual for information on how to switch to CGA emulation.

In addition to the description of the Font Builder utility provided in the following subsections, see the *Font Builder* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Running Font Builder:

1. Font Builder allows you to create a new font file from scratch or to edit an existing font (.FNT) file. If you are modifying an existing file, the safest procedure is to make a copy of the file before beginning. Use the DOS copy command to create a version to edit, for example:

```
COPY 68PC3000.FNT MYTEST.FNT
```

We will use MYTEST.FNT in our example.

2. Start Font Builder from the DOS prompt by entering:

```
FONTBLD
```

3. You are asked for the name of the file to edit. Enter the name of either a new or an existing file.

If you specified the name of a new (non-existent) file, you are asked if you want to create it. Answer "Yes" to create the file, "No" to exit and start over. If you choose to create the new file, you are asked for the controller type:

Font Format [0-61202/3 1-61830 2-82C425]: 0

Series 3000 terminals use the 61202/3 controller, so enter "0".

You are also asked for the character width and height. The acceptable character dimensions are determined by the controller as follows:

Controller	61202/3	61830/82C425
Width	6, 7	6, 8
Height	8	1 - 8

Select an appropriate dimension for the 61202/3 controller.

The Font Builder editing screen is then displayed.

Font Builder Editing Screen

The Font Builder main screen, shown in Figure 8-7, contains the four which are described below.

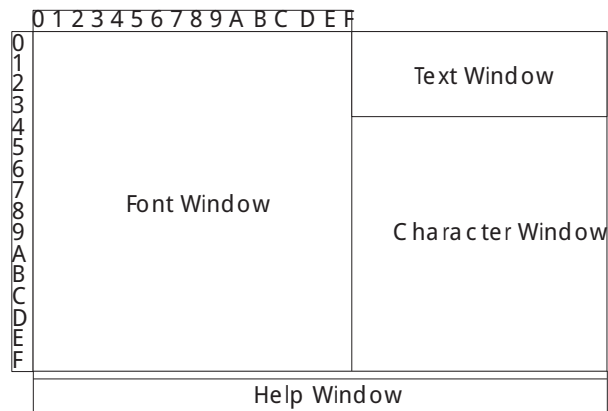


Figure 8-7. Font Builder Main Screen



The Font Window

The Font window displays the complete character table from the font file. The hexadecimal digits on the top and left of the window indicate the character code of each position in the table.

The Text Window

The Text window displays the font characters in string format. The characters can be entered from the keyboard or copied from the font window.

The Character Window

The Character window displays the character being edited.

The Help Window

The Help window displays available commands and methods to invoke them. Commands are entered as single key commands or as <ALT> or <SHIFT> key combinations.

Use the <TAB> key to cycle through the Font, Text, and Character windows. The commands displayed in the Help window change depending on the currently active window.

Basic Character Editing

The following is the procedure for editing a character:

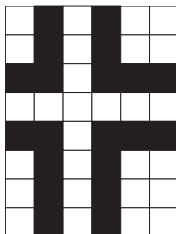
1. When the cursor is in the Font window, use the arrow keys on the keypad to move the cursor to the character (or character value) you want to edit. The current bit map for this character is shown in the Character window.
2. Press <TAB> twice to switch to the Character window.
3. In the Character window, use the arrow keys on the keypad to move the cursor to a pixel position. Press <+> or <Space> to toggle the current value of this pixel.
4. When you are finished modifying the character, press <ENTER>. This places the modified character in the Font window.
5. Repeat Steps 1-4 for as many characters as you want to create/edit.

6. To save the file when you are finished or periodically while editing characters, press **<ALT-S>**. You are prompted for the file format, font format, and the file name. When you have entered this information, the editing screen is repeated.

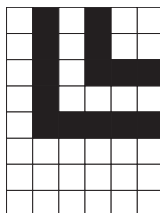
Using the Text Window

The Text window can be used to see how characters look when positioned next to each other on the screen. To position two characters, do the following:

1. In the Font window, place the cursor on the first character to display. For illustration purposes, we'll use the double-line intersection graphic character (Hex EC). Press **<F1>** to place the character in a temporary buffer.

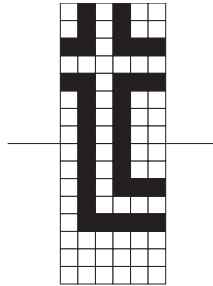


2. Press **<TAB>** once to move the cursor to the Text window. Use the arrow keys to move the cursor somewhere in the middle of the window.
3. Press **<F2>** to copy the character in the temporary buffer to the Text window.
4. Press **<TAB>** twice to return to the Font window.
5. Use the arrow keys to select another character and press **<F1>**. We'll use the double-line lower-left-corner graphic character (Hex 8C).





6. Press **<TAB>** once to return to the Text window and use the arrow keys to move the cursor next to the previous character and press **<F2>**. We'll place this character directly beneath the double-line intersection character.



You can use this method to place any font characters in any position relative to another character to check for appearance.

While a character is displayed in the Text window, you can edit it in the Character window. When you press **<ENTER>**, the character will be updated in the Text window as well as in the Font and Character windows.

General Commands

Table 8-1 lists commands that can be used at *any* of the four windows displayed on the Font Builder main screen.

Table 8-1. Font Builder General Commands

Command	Description
ALT-S	Saves the Font Window to a font file without exiting. Specify save options in response to the prompts as displayed.
ALT-Q	Quits without saving.
ALT-E	Exits and saves the Character Table to the same font file with the same format.
ALT-<i>n</i>	Moves cursor to window <i>n</i> , where <i>n</i> = 1, 2, or 3. 1 = Font Window 2 = Text Window 3 = Character Window.
ALT-F10	Displays the <Alt> key commands in the Help Window.

Table 8-1. Font Builder General Commands (Continued)

Shift-F10	Displays the <Shift> key commands in the Help Window. (Supported in the Character Window only.)
TAB	Moves the cursor to the next window.

Font Window Commands

Table 8-2 lists the keystroke commands that can be used in the *Font* window.

Table 8-2. Font Window Keystroke Commands

Keystroke Command	Description
ENTER	Selects the currently highlighted character for editing and places the cursor in the Character Window.
	Moves cursor <i>up</i> one character.
	Moves cursor <i>down</i> one character.
	Moves cursor <i>left</i> one character.
	Moves cursor <i>right</i> one character.
Home	Moves cursor to the leftmost character on the current row.
End	Moves cursor to the rightmost character on the current row.
PgUp	Moves cursor to the top character in the current column.
PgDn	Moves cursor to the bottom character in the current column.
F1	Copies the character under the cursor to a buffer. Use this command to copy characters between the Font and Text windows. Instead of creating a character from scratch, use this to modify an existing character.
F2	Copies the character in the buffer to the current character position. See [F1].
Del	Deletes the currently highlighted character.



Table 8-2. Font Window Keystroke Commands (Continued)

Ins	Undeletes - places the most recently deleted character (in the buffer) into the currently highlighted character position.
------------	---

Character Window Commands

Table 8-3 lists the keystroke commands that can be used in the *Character* window:

Table 8-3. Character Window Keystroke Commands

Keystroke Command	Description
↑	Moves highlight <i>up</i> one pixel.
↓	Moves highlight <i>down</i> one pixel.
←	Moves highlight <i>left</i> one pixel.
→	Moves highlight <i>right</i> one pixel.
Home	Moves highlight to the leftmost pixel on the current row.
End	Moves highlight to the rightmost pixel on the current row.
PgUp	Moves highlight to the top pixel in the current column.
PgDn	Moves highlight to the bottom pixel in the current column.
Enter	Saves the character to the Font Window.
Esc	Refreshes the Character Window to the character occupying the window before editing began.
Space	Toggles the currently highlighted pixel on or off.
+	Toggles the currently highlighted pixel on or off. Easy to use when using the keypad.
Ins	Turns the currently highlighted pixel ON.
<i>*To use on the PC with FONTBLD, turn off Numlock and use the key on the number pad.</i>	

Table 8-3. Character Window Keystroke Commands (Continued)

Del	Turns the currently highlighted pixel OFF.
Shift-▲ *	Toggles the highlighted pixel on or off while moving the cursor up one position.
Shift-▼ *	Toggles the highlighted pixel on or off while moving the cursor down one position.
Shift-◀ *	Toggles the highlighted pixel on or off while moving the cursor to the left one position.
Shift-▶ *	Toggles the highlighted pixel on or off while moving the cursor to the right one position.
Shift-Home *	Toggles all the pixels in a row on or off from the current cursor position to the leftmost position in the row while moving the cursor to the leftmost pixel position in the row.
Shift-End *	Toggles all the pixels in a row on or off from the current cursor position to the rightmost position in the row while moving the cursor to the rightmost pixel position in the row.
Shift-PgUp *	Toggles all the pixels in a column on or off from the current cursor position to the top of the column while moving the cursor to the top pixel position in the column.
Shift-PgDn *	Toggles all the pixels in a column on or off from the current cursor position to the bottom of the column while moving the cursor to the bottom pixel position in the column.
Shift-Ins *	Turns all the pixels in the Character Window ON.
Shift-Del *	Turns all the pixels in the Character Window OFF.
*To use on the PC with FONTBLD, turn off Numlock and use the key on the number pad.	



Text Window Commands

Table 8-4 lists the keystroke commands that can be used in the *Text* window:

Table 8-4. Text Window Keystroke Commands

Keystroke Command	Description
BackSpace	Moves the cursor to the left and clears the character.
Esc	Clears the window and places the cursor in the upper left-hand corner of the window.
↑	Moves cursor <i>up</i> one character.
↓	Moves cursor <i>down</i> one character.
←	Moves cursor <i>left</i> one character.
→	Moves cursor <i>right</i> one character.
Home	Moves cursor to the leftmost character on the current row.
End	Moves cursor to the rightmost character on the current row.
PgUp	Moves cursor to the top character in the current column.
PgDn	Moves cursor to the bottom character in the current column.
Del	Deletes the currently highlighted character.
Ins	Undeletes - places the most recently deleted character (in the buffer) into the currently highlighted character position.

Making a Font TSR

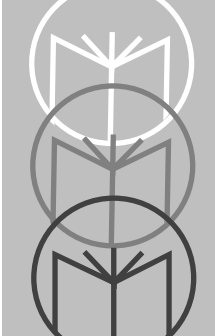
A font TSR program must include the code needed to select a font stored at the end of the file. To create this program:

1. Generate an executable file containing only the code to access the font data. The font data will be appended to this file.
2. Load the font definition into the Font Builder utility (FONTBLD.EXE).
3. Save the font as a .EXE filetype and specify the code only file as the file name. This appends the font information to the .EXE file.

The final file will contain the following information:

Reserved	32 bytes	Reserved by DOS
Header		DOS .EXE file header
Stack		Stack space used by the TSR program
TSR size	1 word	Size of TSR program in bytes
TSR code		TSR code to select soft font and to select the font stored in this TSR program, set logical screen size to the physical size, select soft cursor mode and set the character scale.
Font size	1 word	Size of the font data excluding the font header
Font scale	1 word	Scale for the character width and height (0 = single, 1 = double). First byte is for the width and the second byte is for the height.
Font data		The font header and data.





Chapter 9

Keyboard Processing

Chapter Contents

Introduction	9-3
Keyboard Operation	9-4
Spanish Language Support on Series 3000.	9-12
Processing Key Input.	9-14
Using DOS I/O Control Functions	9-21
Conserving Power	9-23
Using the Keyboard Mapping Utility	9-28
Using a Translation Table	9-36
Sample Programs	9-37



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>BIOS Library Functions</i> <i>DR DOS</i> <i>Appendix A</i>
<i>Series 3000 System Software Manual</i>	<i>ROM BIOS</i>

Series 3000 keyboard control is designed so that the keyboard can emulate the full 101-key IBM PC/XT extended keyboard. This involves an innovative method of mapping PC/XT compatible scan and character codes to key sequences on the Series 3000. With this mapping in place, input from the keyboard is handled on the terminal just as it is on a PC.

The keyboard mapping on a Series 3000 terminal can be modified by way of easy-to-build keyboard translation tables. This makes it easy to build a custom keyboard layout for your application, making the most used keyboard codes accessible by pressing a single key.

As of release 3.01 of the ADK, the Keyboard Mapper utility (KBDMAKE.EXE) has been provided to simplify building keyboard translation files. This utility provides a graphical interface with simple commands, allowing you to quickly and intuitively design a keyboard for your application. The Keyboard Mapper utility is described in detail in the *KBDMAKE.EXE* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.



Keyboard Operation

On a PC, each key generates a scan code. The character code generated by a given key is determined by the scan code and the current keyboard state (i.e., Unshifted, Shifted, Control, or Alternate). The scan code generated by each key is constant, independent of the keyboard state. (Scan code to character code mappings for the Unshifted, Shifted, Control, and Alternate states are shown in the *Appendix A* of the *Series 3000 Application Programmer Reference Manual*.)

The Series 3000 keyboards emulate the full PC / AT keyboard by using one or more modifier keys in sequence, followed by a character key. The modifier keys are:

- Shift
- Caps Lock (Alpha on some keyboards)
- Control (Ctrl)
- Function (Func)

The remaining keys (a through z, 0 through 9, special characters) are called “character keys.”

The character generated is a function of the key scan code and the keyboard state, as on a PC. The main difference is that the scan code generated by a key is also variable, determined by the keyboard state.

Scan Code Translation

Scan codes are determined as shown in Figure 9-1.

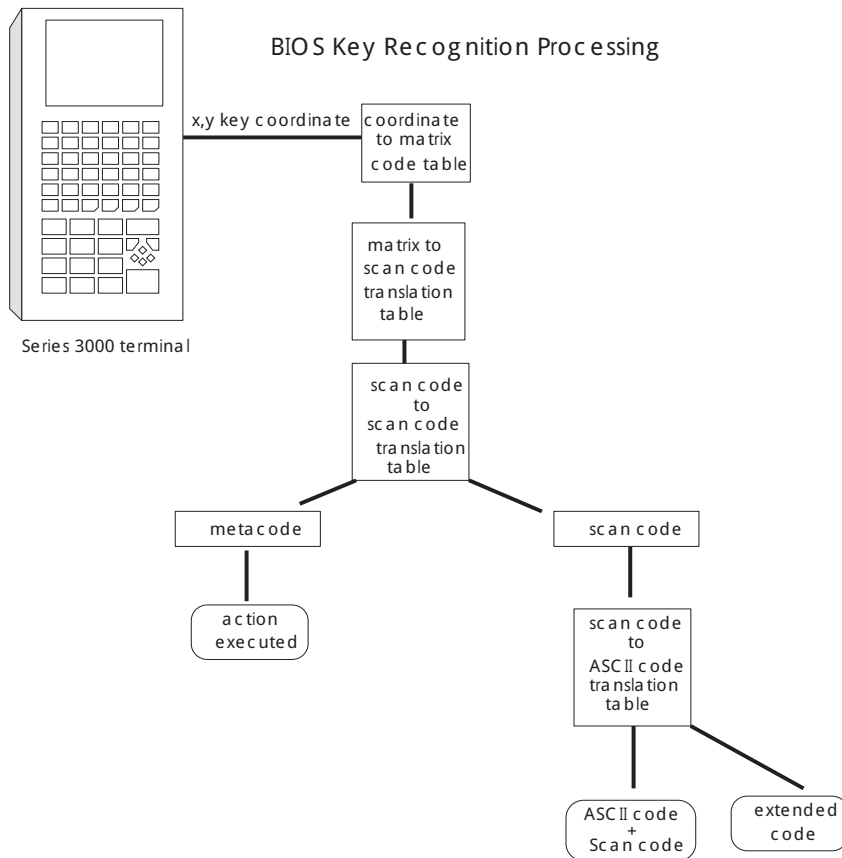


Figure 9-1. Key Processing Diagram



The terminal keyboard produces its own native scan code. On the 56-key keyboard, for instance, these are 00 through 55, as reported in the terminal self test. These native scan codes are mapped by the BIOS to a subset of PC/AT scan codes, the base scan codes (refer to Table 9-1). The subset is different for each keyboard.

Table 9-1. 56-key Scan Code Translation Table

Key Legend	Translations							
	Base Code	No Mod	Shift	Func	Ctrl	Alt	Cap Lock	Num Lock
F1/F6	59		84	64	94	104		
F2/F7	60		85	65	95	105		
F3/F8	61		86	66	96	106		
F4/F9	62		87	67	97	107		
F5/F10	63		88	68	98	108		
On/Off	98							
A	30							
B	48							
C	46							
D	32							
E	18							
F	33							
G	34							
H	35			59				
I	23							
J	36							
K	37							
L	38			100				
M	50			67				
N	49							
O	24							
P	25							
Q	16							

Table 9-1. 56-key Scan Code Translation Table (Continued)

Key Legend	Translations							
	Base Code	No Mod	Shift	Func	Ctrl	Alt	Cap Lock	Num Lock
R	19							
S	31			68				
T	20							
U	22			78				
V	47			74				
W	17			55				
X	45			53				
Y	21							
Z	44							
Caps Lock	58			69				
Func								
Ctrl	29			56				
Shift	42							
7	08			41		126		71
8	09			13		127		72
9	10			43		128		73
Clear	01							
4	05			26		123		75
5	06			27		124		76
6	07			39		125		77
Sp/dark	57			101				
uparrow	72			73	141			
Bksp/light	14			102	03			
1	2			40		120		71
2	3			51		121		80
3	4			53		122		81



Table 9-1. 56-key Scan Code Translation Table (Continued)

Key Legend	Translations							
	Base Code	No Mod	Shift	Func	Ctrl	Alt	Cap Lock	Num Lock
left arrow	75			71	115			
down arrow	80			81	145			
right arrow	77			79	116			
-/No	12					130		
0	11			82		129		82
./Yes	52			83				
Enter/=	28			13				

A keyboard translation table maps the base scan codes to other scan codes, based on the keyboard state. Translations can be provided for seven keyboard states: Unshifted, Shifted, CapLock, NumLock, Function, Control, and Alternate.

For example:

The default keyboard translation table for the 56-key keyboard specifies no translation for the Unshifted, Shifted, CapLock or Alternate states.

The NumLock state translates the number keys to numeric keypad keys.

The Control state translates the Backspace key to Scroll Lock.

The Function state translates several keys to special functions and miscellaneous PC keys.

Once the scan code is determined, the character code is determined by that scan code and the keyboard state, exactly as on a PC.

Meta Code Translation

In addition to translating between scan codes, the translation may be to meta codes. Table 9-2 list six meta codes available for special terminal operations:

Table 9-2. Meta Codes for Special Terminal Operations

Operation	Code	Description
Power	98	terminal On/Off
Keyboard timeout	99	powers off the terminal
Lamp	100	display back light
Dark	101	increase display contrast
Light	102	decrease display contrast
Null	104	generates unique key value

Note that meta code values are outside the range of the PC scan codes.

The Null meta code is available to generate a unique code that cannot be confused with any usual scan code. The code returned is two bytes, consisting of 68h in AH and the key scan code in AL (see the description of the **Return Next Character and Scan Code service ,Interrupt 16h Function 00h** in the *Standard Keyboard Services* section of the ROM BIOS chapter in the *Series 3000 System Software Manual*). Normal processing, which processes the scan code for a keyboard state, is bypassed, and the unique code can be trapped for special processing.

Keyboard States

When the operator presses a modifier key or a defined sequence of modifier keys, the keyboard enters a modified keyboard state. Refer to Table 9-3 for a listing of modifier key or sequence of modifier key and the associated keyboard states.

Table 9-3. Keyboard State Modifier Key/KeySequence

Modifier key or Key Sequence	Keyboard State
No modifier	Unshifted
Shift	All keys shifted
Caps Lock	Alphabetic keys shifted



Table 9-3. Keyboard State Modifier Key/KeySequence (Continued)

Modifier key or Key Sequence	Keyboard State
Ctrl	Control shifted
Func	Scan code translation only. Shift/control state determined by other modifiers in the sequence.
Func + Ctrl	Alternate (3100, 3300,3800,3900)
Func + Caps Lock	NumLock (3300 and 3800)
Func + N	NumLock (3900)

The Shift, Control and Alt (Func + Ctrl) states affect only the next character key pressed. The Caps Lock (Alpha) state remains in effect until Caps Lock is pressed again. On the 35-key keyboard, the Alpha key generates alphabetic characters (upper case only).

While there is no **<Alt>** key on the Series 3000, the Alt state is produced by pressing the **<Func>** key followed by the **<Ctrl>** key. In this sequence, the **<Func>** key maps the **<Ctrl>** key scan code to the PC **<Alt>** key scan code.

As shown by the Alt sequence, the **<Func>** key may be used individually or with other modifier keys. The **<Func>** key affects the keyboard for the next keystroke only. However, if the next key is another modifier key, the effect may be cumulative, setting another keyboard state.

Once a keyboard state has been set, the **<Func>** key may be pressed again to invoke the Function keyboard translation. If, for example, the **<Func>** key is pressed following the **<Ctrl>** key, the Control state remains in effect for the keyboard redefinition invoked by the **<Func>** key. In this cumulative state, the **<F1>** key produces Ctrl-F6 on the 56-key keyboard.

A more complex example is the Func/Ctrl/Func sequence. The first **<Func>** modifies the keyboard, mapping the **<Ctrl>** key into the PC **<ALT>** key. The second **<Func>** occurs in the ALT state, and simply redefines a few of the keys, producing, for example, ALT-F6.

Limits on Keyboard Redefinition

The keyboard state that selects a scan code translation also affects the character produced by that scan code. Only characters available in that keyboard state on a PC are available on the PDT. For example, only shifted characters are produced in the shift state, only control characters in the control state, and so on.

For example, it is not possible to assign the semicolon (;) to a shifted sequence (e.g., Shift + []). This sequence puts the keyboard in a shifted state, but semicolon is an unshifted character on the PC/AT keyboard.



Spanish Language Support on Series 3000

Spanish Character Set Issue

A law has been passed in Spain that requires electronic devices sold there to support those characters that are unique to printed (displayed) Spanish. These special characters are ñ, Ñ, ¿, ¡, ? and !. In order to display these characters they must 1) be available in the Series 3000 font set and 2) be accessible by an application program for display.

Series 3000 Font Set

The font set that is embedded in Series 3000 terminals already contain the six characters required by Spanish law. Applications can print these characters by using the following ASCII to character conversion table:

Table 9-1. ASCII to Character Conversion Table

Character	ASCII Code (decimal)
ñ	164
Ñ	165
¿	168
¡	173
?	63
!	33

As an example, to print the character 'Ñ' a 'C' language application would use the command:

```
printf("%c", 165);
```


The next step is to choose which characters on the standard keyboard the application writer wishes to replace with the Spanish characters. On keyboards that already contain the ? and !, only the remaining four special characters need be considered. Once the four characters to be sacrificed for the Spanish characters have been chosen, a simple routine can be used to translate the pressed keystroke into the corresponding Spanish character to be displayed.

Sample Terminal Application Code

In order to achieve typed keystroke to displayed Spanish character conversion, the following sample code has been provided. This sample translates the typed keystroke '{' into the displayed character 'ñ', the typed keystroke '}' into the displayed Spanish character 'Ñ', the typed keystroke '|' into the displayed Spanish character '¿' and the typed keystroke '~' into the displayed Spanish character '¡'.

```
char ch;
ch = getch();

switch (ch)

{

    case '{' : printf("%c", 164);
               break;
    case '}' : printf("%c", 165);
               break;
    case '|' : printf("%c", 168);
               break;
    case '~' : printf("%c", 173);
               break;
    default  : printf("%c", ch);
               break;

}
```



Processing Key Input

Both keyboard and scanned input are both by the console driver in essentially the same way. While the immediate focus in this chapter is on keyboard input, we will include scanned input processing in the discussion. For information on including scanning in your application, refer to the chapter on *Scanning* in this guide. Before your application can use keyed or scanned data, the keys and labels must be in a queue. The console driver handles the data queues. The application then obtains the queued data from the console driver using functions available in various libraries.

When a key is pressed, the BIOS interprets it and places it in the key queue. Further BIOS services are used by the console driver to extract keys from the queue to pass on to the application (see Figure 9-2).

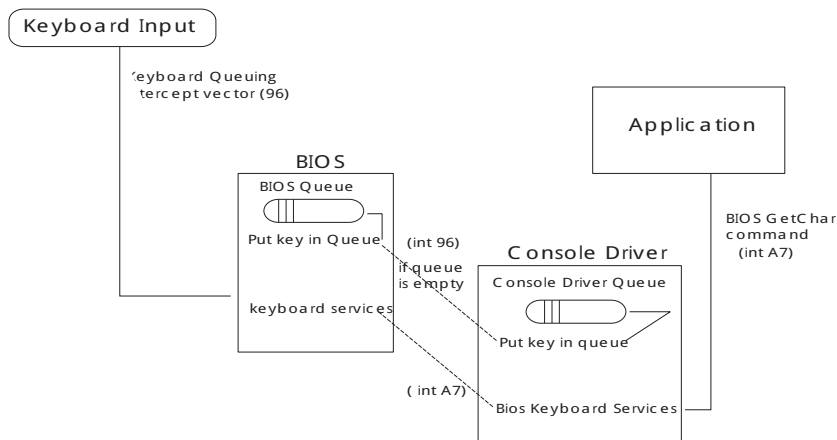


Figure 9-2. Key Placement in a Queue

Once the data is in the queue, there are basically five ways for an application to obtain them. The five levels of keyboard API are, in order from highest to lowest level, the following:

1. Use the C runtime library console I/O functions (**getch**, **printf**, etc.).
2. Access CON: device driver using DOS library file I/O routines (**DosOpen**, **DosRead**, etc.). Refer to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual* for descriptions of these routines.

3. Call DOS library routine to get keys from the queue and check if keys are in the queue (**DosGetCh**). Refer to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual* for a descriptions of this routine.
4. Call BIOS library routines to get keys from the queue and check if keys are in the queue (**BiosGetChar**, **BiosCheckKey**). Refer to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for descriptions of these routines
5. Use assembly language invoking the BIOS by interrupt.

A further consideration is the input mode to use. Table 9-4 lists the three input modes for Series 3000 terminals.

Table 9-4. Series 3000 Input Modes

Mode	Action
Keys Only	Returns key values only. Labels in the queue are skipped until the selected input mode can use them
Keys and Labels	Returns both keys and labels
Labels only	Returns label values only. Keys in the queue are skipped until the selected input mode can use them

Scanning interface is provided using DR DOS IOCTL functions, so if you are including scanning in your application you will have to use a Level 2 or lower interface.

Note: Before using scanning, an application must enable scanning by using I/O control functions. Lower API levels, such as BIOS calls, will not support scanning until this has been done.

The following subsections describe how to use each level of API in all of the applicable modes.



Using C Runtime Library Functions

Information on using the C runtime libraries is provided in the Microsoft C Runtime Library manual and other detailed C programming books.

Using the C runtime libraries is convenient if you are converting a C program to run on a series 3000 terminal. You will only have to change the interface if you plan to use scanning.

Refer to the C:\3000\SAMPLE\KEYBOARD\KEYC.C sample program in the ADK for an example of using this method. Note that the sample is in keys only mode, the system default.

Using DOS File I/O Commands

One of the more flexible methods of accessing keyed and scanned data is through the DOS file I/O and DOS I/O control (IOCTL) commands. This is the only method that allows you to distinguish between key and label data in the queue. None of the other methods (such as using the C libraries, BIOS calls or DOS calls) can distinguish between the two.

The IOCTL commands communicate directly with the SCAN3000.EXE TSR program via the console driver (see Figure 9-3).

To use the DOS File I/O commands effectively, you must know:

- how to establish a DOS handle
- how to close the DOS handle
- how to read keys and labels
- which modes to use
- how and when to use IOCTL commands

All but the last two of these steps are illustrated in the sample program provided in C:\3000\SAMPLE\KEYBOARD\KEYFILE.C of the ADK.

Note that the sample is in Keys Only input mode (see Table 9-4, above). To use scanning, you need to use the Keys and Labels mode or the Labels Only input mode.

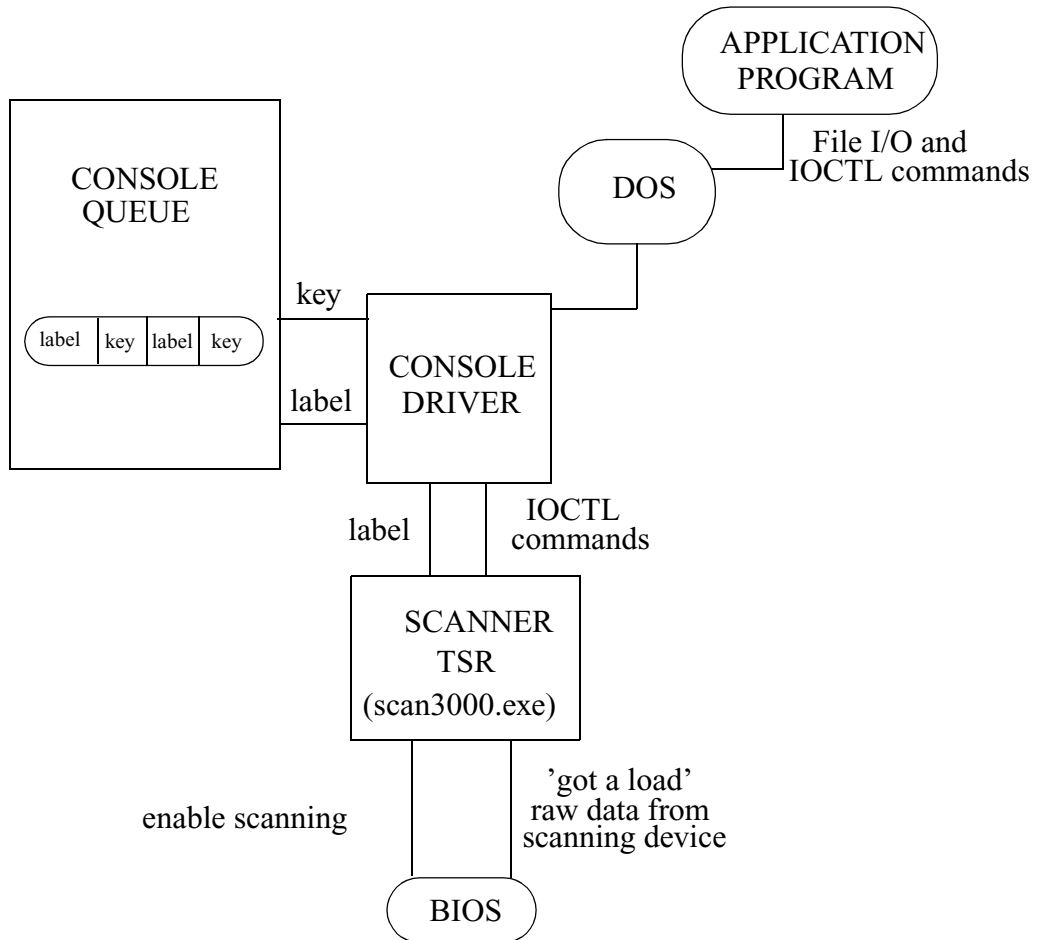


Figure 9-3. Application to SCAN3000.EXE Interface



Using DOS Routines

DOS routines provide yet a lower level of API. The advantage of using this method is that the resulting code is smaller than that produced by the higher level APIs. It cannot, however, distinguish between key and label data in the queue. You still have to use DOS IOCTL commands to perform scanning.

The commonly used DOS command for obtaining keys from the queue is **DosGetCh**. **DosGetCh** returns one character from the queue. You can set one of two option flags:

1. Return to the application if no character is in the queue
2. If the queue is empty, wait for a character before returning to the application.

The following two lines show how you might use **DosGetCh**:

```
int keyhit;  
keyhit = DosGetCh (1);
```

The KEYDOS.C sample program provided in the ADK directory C:\3000\SAMPLE\KEYBOARD illustrates how to use this method. Refer to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual* for a detailed description of **DosGetCh**.

Using BIOS Routines

BIOS calls are the lowest level API available. An advantage to using them is that you receive a single word value for each key, providing both the ASCII code and the scan code. With this information, you can take the word values, define names for them, and create a comparison table. A disadvantage is that BIOS calls cannot distinguish between keys and labels in the queue.

The two most important BIOS commands are **BiosCheckKey** and **BiosGetChar**.

BiosCheckKey

This command checks the BIOS queue for a key. If a key is in the queue, it reports the key's value, but does not remove it from the queue. If no keys

are in the queue, this function returns to the application without waiting for a key. This is referred to as a “non-destructive get.”

BiosGetChar

This command also checks the BIOS queue for a key. However, when a key is in the queue, it reports the key and removes it from the queue. If there are no keys in the queue, it will wait until there is one, powering down the terminal if necessary. This is referred to as a “destructive get.”

For more detailed descriptions of **BiosCheckKey** and **BiosGetChar**, refer to the *Function Descriptions* section of the chapter on *BIOS Library Functions* in the *Series 3000 Application Programmer's Reference Manual*.

There are several other BIOS commands that are useful. They are listed and described briefly below. More detailed descriptions can be found in the reference cited above.

BiosGetKeyboardState

Similar to the standard personal computer's get shift status command. The difference is that the keyboard state command replaces information about the insert key with information about the function key.

BiosSetKeyboardState

Sets the same bits the previous command returns.

With this command, you can start the terminal out in the appropriate keyboard mode. To do this, first use the **BiosGetKeyboardState** command, and then use **BiosSetKeyboardState** to set the keyboard to the state the application requires.

This is especially useful on the 3800, which has a small keyboard and uses modes to access different sets of characters and functions. In a numeric-type field, you could start the terminal out in numeric mode, while an alpha-type field could start the terminal in alpha mode.



BiosClick

Enables, disables audio feedback (clicks) from the keyboard. To enable key clicks, use 0; to disable key clicks, use 1.

BiosAutoRepeat

Enables, disables and sets the initial and repeat delay times for automatic key repeat.

BiosGetKybdTimeout

Gets the amount of time the terminal will wait for keyed input before powering off. This command is used mainly with the **BiosGetChar** command.

BiosSetKybdTimeout

Sets the amount of time the terminal will wait for keyed input before powering off.

BiosSetAbortKey

Selects the scan code to use for the [ABORT] key. The [ABORT] key on series 3000 terminals has the following functions:

- it terminates time delays
- it aborts communications on serial ports
- it stops processing the current application program

BiosGetAbortStatus

Gets the current status of the [ABORT] key. Applications can periodically check the status of the [ABORT] key so they can properly react to it.

A return status of 0 indicates the key hasn't been pressed and is not pressed. A return status other than 0 indicates the [ABORT] key has been pressed and may be still pressed.

The sample program KEYBIOS.C, provided in the ADK directory C:\3000\SAMPLE\KEYBOARD, illustrates the use of the BIOS API.

Using DOS I/O Control Functions

If you want an application to scan data, you must use DOS I/O control functions. These functions are the only means of communicating with the scanning TSR (SCAN3000.EXE). In addition, this is the only method an application can use to distinguish between keys and labels as they are taken from the queue.

In this section we will concentrate on the common aspects of using IOCTL commands for both keyboard and scanner input. For more detailed information about scanning specifically, refer to the *Scanning* chapter in this guide.

IOCTL Parameter Structure

The console driver process controls information in block form. The structure of the block depends on the command being issued.

There are two basic types of DOS I/O control functions. They are:

DosIoCtrlRdData (IOCTL Read)

This function reads blocks of control information sent by the driver.

DosIoCtrlWrData (IOCTL Write)

This function writes blocks of control information to the driver.

In either case, the first two bytes of the block are the command code and the error code, respectively. The structure of the rest of the block depends entirely on the command. Each command and its structure are described in the *DR DOS* chapter of the *Series 3000 Application Programmer's Reference Manual*.

The possible error codes are indicated by a number that corresponds to an error condition. For error codes, refer to the *IOCTL Commands and Error Codes* section of the above reference.



Opening and Closing a Handle

When using DOS IOCTL commands, you must use the **DosOpen** command to establish a handle for communicating with the driver. To do this, use the **DosOpen** function as follows:

```
DosOpen ("CON", READWRITE, &scanhandle) ;
```

Once the application has finished communicating with the driver, terminate the handle by using the **DosClose** function as follows:

```
DosClose (scanhandle) ;
```

Refer to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual* for descriptions of **DosOpen** and **DosClose**.

Selecting a Function

Depending on the requirements of your application, you must select one of the DOS I/O read or write commands. Next, set the function code to the appropriate value. If the command is a write-type command, figure out the format the command requires and supply the application's data in that format. If the command is a read-type command, the application must be able to provide space for the incoming information in the command's block format.

Each IOCTL command falls into one of the following categories:

- Read Only: Used to check status (**DosIoCtrlRdData** only)
- Write Only: "Do this" activities (**DosIoCtrlWrData** only)
- Read and Write: Configuration commands. These commands are used to check and change status. (Both **IoCtrl** commands allowed)

For read and write commands, you should *always read before writing*. If you do not follow this sequence, you can accidentally place invalid information within the structure.

Refer to the *IOCTL Commands and Error Codes* section of the *DR DOS* chapter in the *Series 3000 Applications Programmer's Reference Manual* for a complete listing of the keyboard IOCTL commands.

Conserving Power

There are a number of ways that keyboard controls can be used to reduce the power consumed by the terminal. Two important keyboard related methods of conserving power are:

- The power off terminal function (**BiosPowerOff**)
- The keyboard timeout functions (**BiosSetKeybdTimeout** and **BiosGetKeybdTimeout**)

The following are brief descriptions of these power conserving functions. All of the input and output requirements for these functions are documented in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual*. The services called by these functions are described in the *ROM BIOS* chapter of the *Series 3000 System Software Manual*.

Power Off Terminal Service

An application program can power off the terminal by calling the BIOS function **BiosPowerOff**. The terminal is powered back on by an event previously selected by **BiosSetWakeReason**. The **BiosGetWakeReason** function returns the reason for the power on.

Keyboard Timeout Services

There are two keyboard timeout BIOS services:

BiosSetKeybdTimeout()
BiosGetKeybdTimeout()

You can use these timeout services with **BiosGetChar()** and the DOS file I/O commands. The keyboard timeout services, however, do not apply to **getch()**, **DosGetCh()**, or **BiosCheckKey()**

There are three types of keyboard input routines:

1. Routines that wait for keyboard input and then timeout.

If your program has no other tasks to perform while it is waiting on keyboard input, you can set the keyboard timeout with



BiosSetKeybdTimeout and then call **BiosGetChar** (or use the DOS File I/O commands).

The terminal will wait for keyboard input during the time set by **BiosSetKeybdTimeout**. After the keyboard timeout elapses, the terminal will power off.

2. Routines that check for input, but don't wait. **BiosCheckKey()** and **DosGetCh()** are in this group.

Your program could use these routines to check for keyboard input, perform some other tasks, and then return to check for keyboard input again.

3. Routines that wait for input and never timeout. **getch()** and **DosGetCh(1)** fall into this group.

Refer to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual* for a description of the **DosGetChar** function. Descriptions of the other functions referred to above can be found in the *BIOS Library Functions* chapter of the same manual.

Waking Up the Terminal

After using BIOS services to power the terminal off, there are conditions that power the terminal back on. There are also BIOS services that report the event that powered the terminal back on. All of the following functions are documented in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual* and the services they call are described in the *Power Management* section of the *ROM BIOS* chapter of the *Series 3000 System Software Manual*.

Wakeup Events

The BIOS function **BiosSetWakeReason** lets you select the events that will power the terminal on when it is off. A value of **1** (i.e., the associated bit is **set**) enables the event, a value of **0** (i.e., the associated bit is **clear**) disables the event. Table 9-5 lists the wakeup events and their bit assignments.

Table 9-5. Wake Up Event Bit Assignments

Wake Up Event	Bit	Default Value
Laser trigger	4	1 (Enabled)
Port 1 ring	3	1 (Enabled)
Port 0 ring	2	1 (Enabled)
RTC alarm	1	1 (Enabled)
Any key wake up	0	0 (Disabled)

Note that using “Any Key Wake Up” may not be the most practical way to reactivate the terminal under certain circumstances. Use it only for short-term situations such as when an application is awaiting key or scanned input. Note that, by default, every method is enabled except “Any Key Wake Up.”

Real Time Clock (RTC) Alarm Wakeup

The real time alarm function can power on the terminal if the terminal is off when the preset alarm time arrives. You must, however, use the **BiosSetAlarm** function to set the real time clock alarm and enable the RTC alarm wakeup event (see Table 9-5) before the application powers off the terminal.

Get the Wake Up Cause

Once a terminal has been reactivated from sleep mode, you may want to know which event reactivated it. For this, use the ROM BIOS power management **Get Last Wake Up Cause** service (**INT B1h Function 02h**). Based on the value returned by a call to this service, you can determine which event reactivated the terminal. See the *Power Management* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual* for a description of this service. Table 9-6 lists possible return values and the associated wake up causes.



Table 9-6. Wake Up Causes

Return Value	Wake Up Cause
0	Port 0 ring (COM1)
1	Port 1 ring (COM2)
2	Laser trigger
3	Alarm
4	Power keys
5	Other key
6	Operating system boot
7	System cold start
8	Command mode entry
9	Diagnostics mode

A drawback to this method is that it returns information about the last time the terminal was powered on. Depending on how long the terminal has been up, the information delivered by this service might be quite old. For example, if the terminal is reactivated by a ring and is then placed in a cradle or charger, it will continue to report that a ring reactivated it. This is only a problem when the terminal is not powered off before the check is made.

Disabling the Power Key

During certain critical operations, you may want to disable the power key so that an operator cannot possibly turn the terminal off. To do this, use the BIOS **Disable/Enable Power Key** service (INT B1h, Function 0Ch), described in detail in the *Power Management* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*. This function disables the power key, until you call the service again to reactivate the key.

Note that this command doesn't really disable the power key. Instead, it merely delays the power key's function. If the power key is pressed while disabled, the terminal will not power off until the enable command is issued.

Planning Your Application

Due to the small size of the keyboard, it is a good idea to plan your keyboard definition. This will allow the operator to issue the most common commands using as few keystrokes as possible. Unless you are also providing a keyboard overlay template, you should also attempt to use the keyboard legend printed on the terminal.

For example, the standard keyboard legend on Series 3000 terminals is already mapped and labeled for functions such as “menu”, “help”, and “send”. Try to use this legend as much as possible. Not only will you save the effort of remapping keys, but your application will be easier to use.

It is also generally easier for the operator to press a single key at a time, especially on hand held units, rather than to try to press two or more keys simultaneously. Single-key operating mode is the usually the keyboard mode of choice for applications. It is a good idea to use as few keystrokes as possible for any operation. For example, on a 56-key 3300 terminal, design your application to use function keys F1 through F5 for common functions rather than F6 through F10, which require the **func** modifier.



Using the Keyboard Mapping Utility

The easiest way to create a keyboard redefinition file is to use the Keyboard Mapping utility (KBDMAKE.EXE) that is included in versions 3.01 and later of the ADK. If you are using an earlier version of the ADK, you must create the keyboard definitions “by hand.” The Keyboard Mapping utility is a graphic program that simplifies the process of designing a custom keyboard layout.

KBDMAKE can produce two types of keyboard definition files:

- *Resident file*
The resident file can be included in the terminal NVM image to define the default keyboard.
- *KBD3000 data files*
The KBD3000 format is used as a data file for KBD3000.EXE, making keyboard definition changes dynamic and eliminating the need to reboot the terminal.

These files can be used separately or together, depending on the needs of the application.

The interface is intuitive, using the window, menu, button, and dialog box methods familiar to users of Microsoft Windows™. The following description highlights the procedure only, and does not attempt to be a complete explanation of how to use the program. In addition to what follows in this section, there are detailed descriptions of KBDMAKE.EXE and KBD3000.EXE in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual*.

Since KBDMAKE employs a graphic user interface, it requires a CGA, EGA or VGA display on the development PC. A mouse is required for making selections on the screen.

The “FG_Display” environment variable must be set. Use VGA12 as the default, but look at C:\3000\DOCS\KBDMAKE.DOC for a list of settings for special equipment.

Keyboard Mapping (Map Definition) Utility (KBDMAKE.EXE)

KBDMAKE.EXE requires a mouse and a VGA monitor capable of displaying 640 x 480 resolution. To set-up the appropriate display type for your system, use the FG_DISPLAY environment variable. The list of display adapter settings in Table 9-7 will help you determine the appropriate setting for your system. Please note that some mouse drivers can not operate at resolutions above 640 x 480; therefore, you may have to use a 640 x 480 resolution to see your mouse cursor.

Table 9-7. Display Adapter Settings

Display Types	Adapter Settings
ATI62	ATI mode, 640 x 480, 256 colors
ATI63	ATI mode, 800 x 600, 256 colors
DFIHIRES	Diamond Flower Instruments, 800 x 600, 256 colors
EVGAHIRES	Everex EVGA, 800 x 600, 16 colors
ORCHIDPROHIRES	Orchid ProDesigner VGA, 800 x 600, 16 colors
PARADISEHIRES	Paradise VGA, 800 x 600, 16 colors
TRIDENTHIRES	Trident VGA, 800 x 600, 16 colors
TSENGHIRES	Tseng Labs ET4000 Super VGA, 800 x 600, 16 colors
VEGAVGAHIRES	Video 7 VEGA VGA board in 800 x 600, 16 colors
VESA1	VESA mode 640 x 480, 256 colors
VESA2	VESA mode 800 x 600, 16 colors
VESA3	VESA mode 800 x 600, 256 colors
VESA5	VESA mode 1024 x 768, 256 colors
VESA6A	VESA mode 800 x 600, 16 colors
VESA7	VESA mode 1280 x 1024, 256 colors
VGA11	IBM VGA mode 640 x 480, monochrome
VGA12	IBM VGA mode 640 x 480, 16 colors
VGA13	IBM VGA mode 640 x 480, 256 colors

The recommended setting for FG_DISPLAY is VGA12. Here is an example of how to set the adapter type in your autoexec.bat file:

```
SET FG_DISPLAY=VGA12
```



Caution

Running KDBMAKE with the incorrect adapter type will cause unpredictable results.

Starting the Keyboard Mapper

Start KDBMAKE by entering the following at the DOS command prompt:

```
> cd \3000  
> kbdmake
```

A window and menu bar is displayed with the About Keyboard Mapper dialog box. Click on OK to clear the dialog box.

Creating and Editing a Keyboard Layout

To edit an existing keyboard definition file, select Open under the File menu, then select the file to edit. The keyboard screen for this definition is then displayed (Figure 9-4).

To create a new keyboard definition based on a standard keyboard, select New under the File menu. A dialogue box is displayed listing the currently available terminal types. Terminals are listed by general model number (e.g. 3310 is included in the 3300 type) and number of keys. Select the item that matches the terminal you are programming and click OK. The program then displays a keyboard screen.

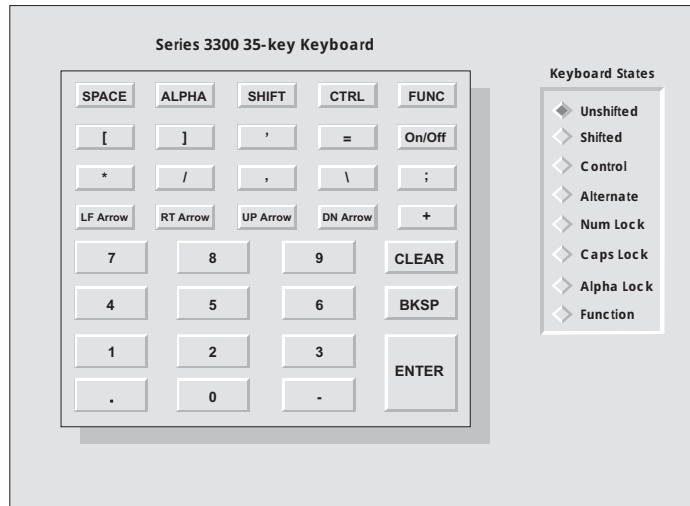


Figure 9-4. Series 3300 35-key Keyboard

As shown in Figure 9-4, the keyboard display screen has two sections: a keypad on the left and a keyboard state panel on the right. The current keyboard state is indicated by the darkened radio button in the keyboard state panel. The keypad indicates the characters assigned to each key in the current keyboard state. Click on the state button to change the state displayed.

To edit the characters assigned to a key, click on that key. The *Key Definition* dialog box for this key is then displayed (Figure 9-5).



	DEFAULT	LABELS DISPLAYED	OVERLAY	Scan Code	ASCII Code
Unshifted	a	a		30	97
Shifted	A	A		30	65
Control	^A	^A		30	1
Alternate	Alt-A	Alt-A		30	0
Num Lock	a	a		30	97
Caps Lock	A	A		30	65
Alpha Lock	A	A		30	65
Function	A	A		30	65

Key ID: 6

Next Prev Default OK Cancel

Figure 9-5. Key Definition Screen

The key is described in five columns:

Default - The characters assigned to this key in the standard (default) keyboard mapping. These characters never change.

Displayed - The legend shown on the key indicating the character assigned to this key and keyboard state. This field can be edited, though it should reflect the character assignment.

Overlay - This legend is included for the key when the keyboard overlay specification is printed. Unless a legend is entered, this field is left blank. It is useful when designing a keyboard overlay template.

Scan Code - The keyboard scan code generated by the key, in decimal. This value is determined from the value in the Default column and cannot be edited.

ASCII Code - The ASCII value of the character generated by the key, in decimal. This value is determined from the value in the Default column and cannot be edited.

To change the character assigned to the key for a keyboard state, click on the button in the DEFAULT column. The Key Selection dialog box is displayed (Figure 9-6).

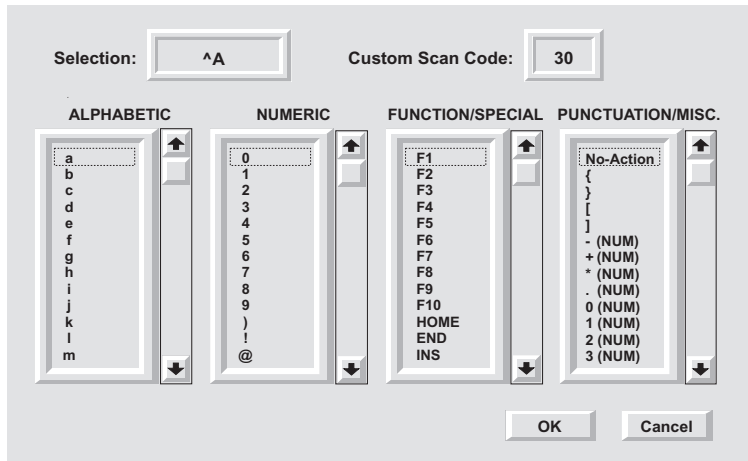


Figure 9-6. Key Selection Screen

To select a character, scroll through the four lists until the character is displayed, then click on it. The character is displayed in the Selection box and its scan code is shown in the Custom Scan Code box. Then click OK.

Note that not all characters can be generated in all keyboard states. For example, only shifted characters, such as upper case letters, can be generated in the shifted state. So, in the shifted state, even if you assign the character “a”, the terminal will generate “A”, the shifted variant.

Repeat the above process for each key and state you need to define. Then use the Save or Save As option under the File menu to save your definition.



Generating Keyboard Files

Once the keyboard definition has been created, generate the definition file. You can generate either a .KBD file or a .C source file or both. The .KBD file is used by KBD3000.EXE as keyboard definition data. The .C file is used in the source for an NVM image file, used by BLDINIT.EXE (see the *Terminal Initialization* chapter in this guide for information on BLDINIT.EXE).

To generate the keyboard definition file or files, select Generate under the Tools menu. Select the file type to generate and click OK.

The Generate screen includes an Emulation field. This is a text field in which you can enter the name of an emulation, such as "VT100" or "5250." The emulation field is used by KBD3000 to restrict its file search to keyboard definitions matching the emulation. Refer to the description of *KBD3000.EXE* in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Manual* for information on this feature.

Writing a Keyboard Translation Program

If you do not use the Keyboard Mapping utility, you must write your own translation table. The keyboard translation table is a small program usually written in C or assembly. The sample program KEYREDEF.C, provided in C:\3000\SAMPLE\KEYBOARD directory of the ADK, illustrates this method.

Note that the SCANCODE.H, METACODE.H and AVAILST.H files are **#included** in the sample program. These provide definitions for the scan codes, meta codes, and keyboard states that you are redefining.

If you are only changing one or two keys in the same keyboard state, you can include the redefinitions at the beginning of the program. However, if you are redefining a number of keys in various keyboard states, you probably need to use the three separate include files.

The keyboard redefinition table consists of a listing of the keyboard states affected by the redefined keys, along with a table listing the number of keys redefined in each keyboard state. Even if the state does not have any redefined keys, you must still list it in this table.

Redefining the Keyboard

Depending on the needs of your application, the keyboard scan code mapping may need to be partially or completely redefined. The sample program `KEYREDEF.C`, provided in `C:\3000\SAMPLE\KEYBOARD` directory of the ADK, is a simple keyboard redefinition program, using the include files `SCANCODE.H`, `METACODE.H` and `AVAILST.H`. The rest of this subsection describes the method used by this program.

The logic used by the sample program is as follows:

- Include all of the necessary include files.
- Define the keyboard redefinition table. This table is composed of a listing of the modified keyboard states followed by a break down listing the number of keys affected in each state.
- Specify the keys to redefine and their replacement values.
- Save interrupt vector 82 and change it to point to the redefined table. This way, the BIOS will use the new redefinition table. If you include your translation file in `INIT.EXE` using `BLDINIT`, this is done for you.

If you are loading the translation table in an application program, remember to save the location of the starting keyboard table and restore it before the program terminates. If you fail to do so, the system will most likely crash.

Another method of redesigning your keyboard is through the use of the `KBDMAKE.EXE` utility, which is a graphic program that simplifies the process of designing a custom keyboard layout. For more information on using this utility, see the earlier section in this chapter on *Using the Keyboard Mapping Utility*, especially the subsection on *Starting the Keyboard Mapper*.



Using a Translation Table

The following are three methods which can be used to remap a Series 3000 keyboard.

Method 1: Changing the Default Keyboard

You can specify a keyboard translation file in the terminal initialization file (INIT.EXE) when you run the BLDINIT utility. Refer to the *Terminal Initialization* chapter in this guide for instructions.

Method 2: Translating From an Application

To initiate keyboard translation from a program, you must set interrupt vector 0X 82 to point to the table. This is demonstrated in the sample program provided in C:\3000\SAMPLE\KEYBOARD\KEYREDEF.C on the ADK. To change the auxiliary keyboard definition, use INT 0X85.

Method 3: Using KBD3000

KBD3000.EXE is a keyboard redefinition program that runs on the Symbol Series 3000 terminal as a TSR. KBD3000 redefines the keyboard according to data provided in a file generated by the Keyboard Mapper utility (KBDMAKE.EXE). It is used in conjunction with a .KBD file you have generated with KBDMAKE. The TSR registration program, TSRREG.EXE, must be loaded before KBD3000 is loaded, or KBD3000 will exit and return an error to the DOS shell. For detailed descriptions on KBD3000, KBDMAKE, and TSRREG, refer to the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*. There is also additional information on KBDMAKE in the *Using the Keyboard Mapping Utility* section of this chapter.

Invoke KBD3000 in a batch file or from the terminal command line as a command with arguments. For example:

```
C:> KBD3000 -STD=f B:S3356V1
```

This command loads the file S3356V1.KBD from the B: drive:

Note: Arguments used with KBD3000 are case-sensitive.

For further information on using KBD3000.EXE refer to C:\3000\DOCS\KBD3000.DOC in the ADK, or to the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.

KBD3000 can be accessed from an application program through C interface routines. Refer to *Series 3000 Application Programmer's Reference Manual* for descriptions of these functions.

Note: TSRREG.EXE must be loaded before loading KBD3000.

For example, in your response file:

```
/u c:\3000\files\tsrreg.exe  
/u c:\3000\files\kbd3000.exe
```

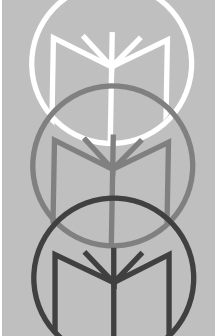
In the AUTOEXEC.BAT file to be included in your terminal, invoke **tsrreg** *before* **kbd3000**.

Sample Programs

The following programs are included in the C:\3000\SAMPLE\KEYBOARD directory of the ADK to illustrate methods of incorporating keyboard and scanner input:

KEYREDEF.C	Redefines the keyboard
KEYC.C	Uses the C Runtime Library routines
KEYFILE.C	Uses the DOS File I/O commands
KEYDOS.C	Uses the DOS routines
KEYBIOS.C	Uses the BIOS routines





Chapter 10

Scanning

Chapter Contents

Introduction	10-3
Changes in This Release	10-3
The Scanning Interface	10-6
Loading SCAN3000	10-9
Loading SCAN3500	10-11
Adding Scanning to an Application	10-12
BLDSCAN.EXE.	10-14
Parameter Descriptions.	10-32
Parameter Menu Scanning	10-39
Modifying the Scanner, Reader, and Decoders	10-77
Creating the Initialization File	10-88
Updating the Scanner, Readers, and Decoders	10-89



Introduction

As you read this chapter, we recommend that you have available for reference the following Series 3000 ADK document:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>DR DOS</i>
<i>Series 3000 System Software Manual</i>	<i>Console/Scanner Device Driver</i>

Bar code scanning as an alternative to keyboard data entry is central to most applications that run on Series 3000 terminals. Series 3000 terminals support a wide range of bar code scanning input devices and symbologies.

While the tools provided in the ADK and Scan Kit attempt to simplify the task of incorporating bar code scanning, you need to be familiar with the environment in which your application programs will be employed. For equipment configuration requirements, refer to the documentation that has been provided for the scanning device. For an introduction to bar code scanning and symbology, refer to a book on the subject, such as *The Bar Code Book* by Roger C. Palmer (Helmerts Publishing Co., Inc., 2nd edition, 1991).

Changes in This Release

There are many new features and enhancements in this version of the Series 3000 Scan Kit. In this release, one API (Set/Get Laser Timeout) has been added to SCAN3000.EXE to take advantage of the PDT 68XX terminal's programmable laser-on time feature. The BLDSCAN utility has been updated to include the option menu for UCC/EAN128 code type. The default parameter settings for the SCAN3000.EXE program in PDF1000 mode differ from settings in previous releases; the default return code type has been changed from Code39 to PDF and the default block size for the PDF 1000 mode has been increased from 20 bytes to 100 bytes.



Table 10-1. Differences Between SCAN3000.EXE and SCAN3500.EXE

Item	SCAN3000.EXE	SCAN3500.EXE
DOS variables that must to be set in order to run BLDSCAN	FILES3000 3000INC LIB3000 Example: FILES3000=c:\3000\files 3000INC=c:\3000\3000 LIB3000=c:\3000\lib	FILES3500PDF 3500PDFINC LIB3000 Example: FILES3500PDF=c:\3000\3500pdf\files 3500PDFINC=c:\3000\3500pdf\3500 LIB3000=c:\3000\lib
Platforms	All other Series 3000 terminals	PDT35XX-P terminals only
Loading	SCAN3000 [xxxx][yyyy]	SCAN3500
Running	None	The terminal must not be seated in the cradle

Table 10-1. Differences Between SCAN3000.EXE and SCAN3500.EXE (Continued)

Item	SCAN3000.EXE	SCAN3500.EXE
IOCTL commands		The following commands are not available in this release: ConsIoctlGetReaderChar ConsIoctlSetReaderChar ConsIoctlGetReaderParms ConsIoctlSetReaderParms ConsIoctlSetPDFComParms ConsIoctlGetPDFComParms ConsIoctlGetScanParms* ConsIoctlSetScanParms* * The options for BeepFreq, beepTime, beepDecode, and FeedbackTime are still available. The following commands must be executed only when the terminal is not in the cradle: ConsIoctlSetScanState ConsIoctlSetDecoderParms ConsIoctlSetRedundancy ConsIoctlSetCheckDigits ConsIoctlSetUPCParms ConsIoctlSetReturnFmt ConsIoctlSetScanExtensions ConsIoctlSetPDFcontrol ConsIoctlGetPDFcontrol ConsIoctlSetPDFdata_access ConsIoctlGetPDFdata_access ConsIoctlResetPDFdata_status ConsIoctlGetPDFdata_status
System Information	None	NR and NG When NR is read from the DosRead(), there is no read in the last trigger pull. When NG is read from the DosRead(), data is damaged, and rescan is required.



The Scanning Interface

Scanning on Series 3000 terminals is handled as an extension of the console. A scanning driver, SCAN3000.EXE (or SCAN3500.EXE on PDT 35XX-P terminals), is loaded into the terminal, usually into NVM, and run as a TSR. As data comes into the console in need of decoding, the data is passed to SCAN3000 (or SCAN3500). The decoded data is then returned to the console driver for processing. Refer to the chapter on the *Console/Scanner Device Driver* in the *Series 3000 System Software Manual* for a detailed description of scanning operations and commands that can be used in applications to control scanning functions.

SCAN3000 (or SCAN3500) is easily configured to include only the decoders required by your application. Use the BLDSCAN utility (BLDSCAN.EXE), described later in this chapter, to build a custom SCAN3000 (or SCAN3500). Control of the scanning process, enabling and disabling decoders, setting scanner characteristics, setting the scan mode, and so on, is handled by the application program using console IOCTL commands. These commands are described in the *IOCTL Commands and Error Codes* section of the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Foreground and Background Scanning

There are two basic ways to incorporate scanning in an application:

- As a *foreground process*, scanning can be enabled only when needed ("scan on demand"). Once scanning is enabled, the application waits until data is scanned-in before proceeding.

Use this method when both scanned and keyed-in operator input are required, but only at specific fields. For example, in an electronic order entry system you may want to scan in the product number but key in the quantity. The application should disable scanning when prompting for keyed information.

- As a *background process*, scanning is enabled at all times. If the application is not ready for scanned input, the label is placed in the scan ahead buffer.

Use this method in applications that scan and increment. For example, an inventory program that increments after receiving a scanned label is such that there are bursts of scanned input and periods of inactivity. Since the program never knows when to expect scanned input, scanning should be on at all times.

To enable background scanning, use the **ConsIoctlScanState** command setting the *scan_state* field to **SCANSTATE_ON**.

PDF 417 Support

Versions 3.01 and higher of SCAN3000.EXE provide support for PDF 417 two-dimensional bar readers. PDF 417 is an interface between the PDF1000 laser scanner/decoder and the console driver. The PDF1000 sends serial data to the terminal through the laser scanner port. If the scanner driver has PDF 417 linked in, the scanner driver takes the serial data and converts it to a format that is specified via the BLDSCAN utility (BLDSCAN.EXE), described later in this chapter. Three data format modes are supported by BLDSCAN: *contiguous mode*, *separator mode*, and *template mode*. These are all described later in this chapter in the *PDF Submenu* subsection of the *BLDSCAN.EXE* section.

Note: When using the PDT 35XX-P terminal, PDF support is built into the terminal. No external connection of a PDF1000 scanner is necessary.

Sample Programs for PDT 35XX-P Terminals

The following sample program can be found in the C:\3000\SAMPLE directory of the ADK:

SCANDISP.C A simple scanning application

Run NMAKE 35XXPNVM to create an NVM image for the PDT 35XX-P terminal. The output file created is called 35XXPNVM.HEX.



Sample Programs for all other Series 3000 Terminals)

The default NVM **SCAN.C** and **ORDER.C** programs are now distributed on the ADK 3.01 and above, and can be found in the C:\3000\SAMPLE directory.

Two sample programs for scanning are provided in C:\3000\SAMPLE\KEYBOARD:

SCAN.C	A simple scanning application
SCANNKEY.C	Integrates scanned and keyboard data entry

Two other useful sample programs are also included:

SCANLIST.C	Prints a list of all available decoders and their characteristics
SCANMENU.C	Allows users to change the characteristics of the available decoders

Run NMAKE DEFNVM to create an NVM image for the other Series 3000 terminals. The output file created is called DEFNVM.HEX.

Loading SCAN3000

SCAN3000.EXE is a configurable TSR that interfaces with the console driver to perform bar code decoding. To configure SCAN3000, i.e., to select the bar code symbologies and parameters you want to support in your application, use the BLDSCAN utility, which is described in the *BLDSCAN.EXE* section of this chapter.

SCAN3000 can be executed on the terminal command line, included in the terminal AUTOEXEC.BAT, or executed by any batch file. The format is:

```
SCAN3000 [xxxx] [yyyy]
```

where:

- xxxx* *Raw data buffer size*, specified in bytes. This buffer holds the data from the bar code reader. In general, the terminal requires 2 bytes of storage for each bar and space (transition) in the bar code. The default value is 0800 hex bytes, or enough for 1024 transitions.
- yyyy* *Label buffer size*, specified in bytes. This buffer holds the decoded label data being passed from the scanner driver to the console driver input queue. The default value is 0024 hex (36 dec) bytes. There are 4 bytes of overhead associated with the buffer, so this allows for labels up to 32 characters. The maximum buffer size is 3B hex (59 dec) for 55 characters.

The parameters are position sensitive.

Version Numbers and CRC

The scanner driver reports up to three version numbers when it loads. It always reports its own version number. It may also report a version number for Klasse Eins and PDF, if they are present.

Following the version number(s), the driver displays the CRC code. It is always the same for a version of SCAN3000 and can be used to track iterations of the driver and to verify that the driver has loaded properly.



Error Conditions

The following errors may occur when loading SCAN3000:

Not enough memory

There is not enough memory for SCAN3000 to load. To remedy this condition, remove unneeded items from memory, increase the transient program area (TPA), or rebuild SCAN3000 with fewer decoders.

Already loaded

The scanner is already loaded. Press ENTER to abort loading.

Scanner error

The following error numbers can occur:

Table 10-1. Scanner Error Codes

Code	Description
1	This indicates that the TSR could not find the console driver. This scanner error is <i>not</i> likely to occur.
2	SCAN3000 cannot load because the raw data buffer plus the scanner data space exceeds 64K.
3	SCAN3000 cannot load because it exceeds the address space of the current data segment.
4	SCAN3000 cannot load because the Klasse Eins module was unsuccessful in allocating a timer for itself from the system timer pool.

Loading SCAN3500

SCAN3500.EXE is a configurable TSR that interfaces with the console driver to perform bar code decoding. To configure SCAN3500, i.e., to select the bar code symbologies and parameters you want to support in your application, use the BLDSCAN utility, which is described in the *BLDSCAN.EXE* section of this chapter.

SCAN3500 can be executed on the terminal command line, included in the terminal AUTOEXEC.BAT, or executed by any batch file. The format is:

SCAN3500

For more detailed information on Series 3000 scanning operations and scanner driver commands for use in application programs, refer to the *Console/Scanner Driver Device Driver* chapter of the *Series 3000 System Software Manual*.



Adding Scanning to an Application

There are three basic methods of incorporating scanning into an application:

- The *wedge* approach. Label and key input are accepted.
- The *UBASIC WANDIN%* approach. Only label input is accepted.
- The *UBASIC KEYIN* approach. Input can be set to accept key-only, label-only, or both key and label.

Using the Wedge Approach

The wedge scanning approach is the easiest way to incorporate scanning into an application. Both keyed and scanned data can be entered.

For *foreground* scanning, set keys and labels mode. When the application requests keyboard input, scanning is temporarily enabled as well as the keyboard. As a foreground process, this method accepts keyed ahead data, but not scanned ahead data.

As a *background* process, this method accepts both keyed ahead and scanned ahead data. Scanning is always enabled. Labels scanned ahead fill the buffer in chronological order.

Once the necessary IOCTL commands have been issued to set up the wedge approach, any API can be used to get the keys and labels from the queue.

See the SCAN.C program in the C:\3000\SAMPLE directory in the ADK for an example of using the wedge approach.

Using the UBASIC WANDIN% Approach

The UBASIC WANDIN% approach provides the same functionality as the UBASIC Plus WANDIN% command. This method only lets the operator scan input. No keyed input is accepted. With this approach, both foreground and background scanning use labels-only mode.

For *foreground* scanning, set labels-only mode. Scanning is enabled when the application requests keyboard input.

For *background* scanning, scanning is always on so the operator can scan ahead. The next time the application requests keyboard input, it can only read a label.

Labels-only mode is active for a single read request. After the read is complete, whether successfully or unsuccessfully, the mode reverts to its previous setting. As a result, an application must set labels-only mode for each read. This prevents the keyboard from being locked out for more than one scan.

Using the UBASIC KEYIN Approach

The UBASIC KEYIN approach provides the same function as the UBASIC KEYIN command. This command allows scanning on an empty field, but doesn't allow scanning after one or more keys have been entered for a field. However, if the operator enters and then erases all data from the field, scanning is temporarily re-enabled.

Only foreground scanning can be used in this approach. This method enables or disables scanning based on data in the field. Background scanning would defeat the purpose of the KEYIN approach.

See the SCANNKEY.C program in the C:\3000\SAMPLE \KEYBOARD directory of the ADK for an example of this approach.



BLDSCAN.EXE

The BLDSCAN.EXE utility builds a customized scanning driver TSR called SCAN3000.EXE (or SCAN3500.EXE on PDT 35XX-P terminals). BLDSCAN enables you to optimize the scanning driver by selecting only the decoders that your application requires.

If you do not need or want to optimize the default decoder TSR, you may use the default decoder TSR (SCAN3000.EXE or SCAN3500.EXE). Most decoders occupy 1K or 2K, so you can usually save a good deal of space by using BLDSCAN.EXE.

Running BLDSCAN

1. To invoke the BLDSCAN utility enter the following at the DOS prompt on the development PC:

```
BLDSCAN [filename]
```

where *filename* is an optional parameter specifying the output file. BLDSCAN appends a .EXE extension. The default output filename is SCAN3500.EXE for PDT35XX-P terminals, and SCAN3000.EXE for all other Series 3000 terminals.

The Terminal Selection Menu screen is then displayed (Figure 10-1).

Please select the type of terminal: _____

1. PDT35XX-P
2. All others

Figure 10-1. Terminal Selection Menu Screen

2. Select the appropriate terminal type from the menu. If you select PDT 35XX-P from this screen, the PDT 35XX-P PDF 417 submenu is displayed (Figure 10-6). If you select "2" (i.e., All others), the program displays the

Decoder Selection Menu Screen (see Figure 10-2).

Decoder	Data Length
A. UPC-A	12
B. UPC-E0	6
C. EAN-13	13
D. EAN-8	8
E. UPC-E1	6
F. Supplementals	2/5-OFF
G. D 2 of 5	12
H. I 2 of 5	14
I. Code 39	Variable
J. Codabar	5 - 55
* K. Code 128	Variable
L. Code 93	4 - 55
M. Code 11 1 cd	4 - 55
N. MSI 1 cd	4 - 55
O. PDF 417	--
P. UCC/EAN 128	Variable

SPECIAL OPTIONS LIST

BAR CODE MENUS

F1	HELP
F3	SET DEFAULTS
F5	SAVE SETTINGS
F9	RESTORE SETTINGS
F10	RESTART BLDSKAN

Select Decoders
By Letter

X. Expanded Parameter Summary Menu
S. Special Options
Q. Quit (lose changes)
Z. Execute

Driver: scan3500
Copyright (c) 1990-1998, Symbol Technologies, Inc.

Figure 10-2. Decoder Selection Menu Screen

3. Select the decoders you want to include in the scanning driver from the Decoder Selection Menu Screen. To select a decoder, press the letter key of the letter shown beside the decoder name. An asterisk (*) is displayed next to selected decoders. To deselect a decoder, press the letter key until the asterisk is removed.
4. Some decoders display one or more submenus and prompts to select special properties. These are described below.
5. Press <Z> to build the scanner TSR. You can specify the name of the output file on the command line or accept the default filename, i.e., SCAN3500.EXE or SCAN3000.EXE.

The scan program is ready to be downloaded to your terminal. Make sure it is invoked before your scanning application.



To modify decoder parameters, deselect the decoder then select it again set the appropriate parameters.

Data Length Submenu

Some decoders support several data lengths. For these decoders, the Data Length submenu is displayed (see Figure 10-3).

To select a data length from the Data Length submenu, press the letter key of the letter beside the correct data length as follows:

- | | |
|--------------------|---|
| A. Single | Only bar codes of a single length are accepted. You will be prompted for the length. |
| B. Dual | Only bar codes of either of two lengths are accepted. You will be prompted for the two lengths. |
| C. Range | Only bar codes within a range are accepted, inclusive of the extremes. You will be prompted for the minimum and maximum acceptable lengths. |
| D. Variable | This accepts bar codes of any length. No additional information is requested. |

When you have made your selection, press <ENTER> to return to the decoder selection menu.

(decoder name)	
Data Length	
*	A. Single B. Dual C. Range D. Variable
Enter to Accept	

Figure 10-3. Data Length Submenu

Supplementals

The Supplementals option on the decoder selection menu displays the Supplemental submenu, shown in Figure 10-4. UPC supplementals are short code sections placed immediately to the right of the main bar code.

Supp Mode	
*	A. Off B. Supp only C. Auto

Supp Mode	
	D. No Supps E. Supps - 2 F. Supps - 5 * G. Supps - 2 - 5

Enter to Accept

Figure 10-4. Supplemental Submenu



The selections on the Supplementary Submenu (Figure 10-4) are:

- | | |
|----------------------|--|
| A. Off. | No supplementals are decoded. Any bar code will be decoded, but the supplementals will not. |
| B. Supp Only. | Only UPC bar codes with supplementals matching a specified supplemental length are decoded. |
| C. Auto. | Bar codes with or without supplementals, and supplementals of length 2 or 5 are decoded. Other supplemental lengths are not permitted. |
| D. No Supps. | No supplementals are decoded. This option forces Supp Mode to Off (Option A). |
| E. Supps-2. | Supplementals of length 2 are decoded, but only if Supp Only is also selected. |
| F. Supps-5. | Supplementals of length 5 are decoded, but only if Supp Only is also selected. |
| G. Supps-2-5. | Both supplementals of length 2 and 5 are decoded, but only if Supp Only is also selected. |

PDF Submenu

PDF 417 is not actually a decoder but an interface between the PDF1000 laser scanner/decoder and the console driver. The PDF1000 sends serial data to the terminal through the laser scanner port. If the scanner driver has PDF 417 linked in, the scanner driver takes the serial data and converts it to a format that can be specified through the BLDSCAN utility. The format is specified using parameters on the Series 3000 PDF 417 submenu (Figure 10-5).

Note: When the PDT 35XX-P terminal type has been specified on the Terminal Selection Menu screen (see Figure 10-1), a different submenu appears (See Figure 10-6).

PDF 417 Interface Parameters				
Baud Rate		A. 300 B. 600 C. 1200	*	D. 2400 E. 4800 F. 19200
Parity	*	G. Even F. Odd		I. None
Stop	*	J. One		K. Two
Data	*	L. 7Bit		M. 8Bit
Dir	*	N. Fwd		O. Rvs
Source		P. Wand	*	Q. Lsr
Data Mode	*	R. Contiguous Mode S. Separator Mode T. Template Mode		
		Press Enter to Accept		

Figure 10-5. Series 3000 PDF 417 Submenu

Most of the parameters displayed on the Series 3000 PDF 417 submenu in Figure 10-5 are communications parameters. These should be set to match the PDF laser scanner setup.

The remaining parameters are set up in the serial data to appear to the application as either wand or laser scanner data. In this way, no special programming is required for the PDF 417 scanner. If PDF 417 is enabled, only the PDF 1000 laser scanner/decoder may be used. Options for I D Symbologies (i.e., code 39 length, disabling 12 of 5) must be parameterized on the PDF 1000 system.



PDF 417 Interface Parameters		
Data Mode	*	A. Contiguous Mode B. Separator Mode C. Template Mode

Press Enter to Accept

Figure 10-6. PDT 35XX-P PDF Submenu

PDF Data Mode

The Data Mode field on the PDF submenus in Figures 10-5 and 10-6 determines how data is packaged when sent to the application from the scanner driver. The three data mode options each use one additional prompt.

Contiguous data mode passes the entire decoded PDF 417 label to the application as the selected bar code type. The maximum size of the individual blocks is specified by the block size parameter. If a terminating character is specified, a label information buffer packet containing only the terminating character is sent as the last block.

Contiguous Format Parameters	
0	A-O. Returned Type - - -PDF 417 - - -
100	P. Block Size
---	Q. Terminate Character

Enter to Accept

Separator data mode allows a special character to be embedded in the bar code to delimit data fields. Data between separator characters is output as individual scans, each with the bar code type defined in the Returned Type field. If any data field is larger than the specified block size, it is broken into individual blocks. The separator character is not reported as part of the data.

Separator Format Parameters	
1	A-0. Returned Type ---Code 39 ---
20	P. Block Size
^	Q. Separator Character
Enter to Accept	

Template mode allows selection of one or more templates. Templates are files that describe how to handle data. Templates can have an ID field that matches an ID field in the PDF label allowing use of Auto Template Detect. Templates are detected by matching the ID field. Alternatively, several templates can be available, with the current template being selected by the application.

Template Format Parameters	
A.	Auto Template Detect
B.	Default Template
C.	User Template 1
D.	User Template 2
E.	User Template 3
F.	User Template 4
G.	User Template 5
Enter to Accept	

The *PDF Templates* section that follows in this chapter describes valid PDF template file commands and their parameters.

PDF Templates

The PDF template file mode allows you to define a template for parsing the encoded PDF data as a series of simulated scans. The template source file is an ASCII text file with the .PDF extension.

Each line in the file contains a template directive (command). The following are valid template directives:



-APPLEN *aaa*

Initial Default: *aaa* = *ppp* set by **-PDFLEN** (see below in this section)

Number of bytes in decimal that the application program is expecting for the current data field. It may not be less than the value of *ppp* set by the **-PDFLEN** directive.

-BEEP

Initial Default : Beep on last

Beep control value. The terminal will beep on the current parsed label from the PDF label buffer when passed to the application program; otherwise, it will beep on the last simulated scan from the PDF label buffer.

-COMMENT *text*

Initial Default : Not Applicable

Comments for the template file. When used, **-Comment** must be the only directive on the line.

-DIR *FWD* | *RVS*

Initial Default : Forward (1)

Returned scan direction as follows:

1 = Forward

2 = Reverse.

-ID *char* | *ascii value*

Initial Default : None

Template ID value may be specified explicitly or by its ASCII value from 00 to 255 in decimal. ASCII values less than 10 must be preceded by a leading 0. Up to 9 characters, each separated by the space character, may be used to define the template's ID. If used, **-ID** must be the first directive defined in the template and must be the only directive on the line. The first data bytes in the PDF bar code must match the specified ID. If they do not match, the

bar code is not parsed by the rest of the template. If auto discriminate templates is enabled, all templates will be searched for an ID match.

-KEYSTROKE *char* | *ascii value* | *specialname* | *extendedname*

Initial Default : None

-KEYSTROKE allows keyed data to be entered directly into the application program. For example, the user may have to hit the “Y” and then **<ENTER>** to input another item. **-KEYSTROKE** must be the only directive on the line.

Keystrokes may be specified explicitly or by their ASCII values, 00 to 255 decimal. ASCII values less than 10 must be preceded by a leading 0. Up to 8 characters, each separated by a space, may be used to specify a keystroke value. Keys that cannot be expressed as a character or ASCII value may be specified by their *specialname* or *extendedname*. Valid special key names and extended key names are listed in Tables 10-2 and 10-3, respectively. Note that special names take up 3 keystroke values and extended names take up 2 keystroke values.

Table 10-2. Special Key Names

CTRLA	CTRLQ	KPAD3
CTRLB	CTRLR	KPAD4
CTRLC	CTRLS	KPAD6
CTRLD	CTRLT	KPAD7
CTRLE	CTRLU	KPAD8
CTRLF	CTRLV	KPAD9
CTRLG	CTRLW	CTRLLEFT
CTRLH	CTRLX	CTRLRIGHT
CTRLI	CTRLY	CTRLUP
CTRLJ	CTRLZ	CTRLDOWN
CTRLK	ESC	GREYPLUS
CTRLL	SPACE	GREYMINUS
CTRLM	ENTER	GREYSTAR
CTRLN	BKSP	GREYSLASH



Table 10-2. Special Key Names (Continued)

CTRLO	KPAD1	
CTRLP	KPAD2	

Table 10-3. Extended Key Names

ALTA	ALTY	ALTF8	CTRLF1
ALTB	ALTZ	ALTF9	CTRLF2
ALTC	ALT1	ALTF10	CTRLF3
ALTD	ALT2	F1	CTRLF4
ALTE	ALT3	F2	CTRLF5
ALTF	ALT4	F3	CTRLF6
ALTG	ALT5	F4	CTRLF7
ALTH	ALT6	F5	CTRLF8
ALTI	ALT7	F6	CTRLF9
ALTJ	ALT8	F7	CTRLF10
ALTK	ALT9	F8	CTRLBRK
ALTL	ALT0	F8	PGUP
ALTM	ALTMINUS	F9	PGDN
ALTN	ALTFUNCEQU	F10	HOME
ALTO	ALTFUNCH	SHIFTF1	END
ALTP	ALTFUNCM	SHIFTF2	INS
ALTQ	ALTFUNCS	SHIFTF3	DEL
ALTR	ALTF1	SHIFTF4	CTRLPGUP
ALTS	ALTF2	SHIFTF5	CTRLPGDN
ALTT	ALTF3	SHIFTF6	CTRLHOME
ALTU	ALTF4	SHIFTF7	CTRLEND
ALTV	ALTF5	SHIFTF8	
ALTW	ALTF6	SHIFTF9	
ALTX	ALTF7	SHIFTF10	

-LOOP

Initial Default : Not Applicable

Indicates the end of a repeat loop. Matches with previous **-REPEAT** directive (see below). **-REPEAT...-LOOP** pairs may be nested up to 100 levels. Directive lines enclosed will be repeated until the embedded data character defined by the matching **-REPEAT** directive is reached. The embedded character need not match with the definition line prior to the **-LOOP** directive.

-PDFLEN *ppp*

Initial Default : None. Must be defined by user.

Number of bytes in decimal to get from the PDF label buffer. If the length is set to variable (*ppp* = 0), then the default or selected value for the **-SEP** directive (see below) is used and must be placed in the encoded data.

-POST *char* | *ascii value*

Initial Default : NULL = 0

Post-fill character may be specified explicitly or by its ASCII value from 00 to 255 in decimal. ASCII values less than 10 must be preceded by a leading 0. Used only if the application program expects more characters than requested from the PDF label buffer (*aaa ppp*). See the **-APPLEN** and **-PDLEN** directives, above. Only a Pre (see below) or a Post character may be used within an individual simulated scan, not both.

-PRE *char* | *ascii value*

Initial Default : NULL = 0

Pre-fill character may be specified explicitly or by its ASCII value from 00 to 255 in decimal. ASCII values less than 10 must be preceded by a leading 0. Used only if the application program expects more characters than requested from the PDF label buffer (*aaa ppp*). See the **-APPLEN** and **-PDLEN** directives, above. Only a Pre or a Post character may be used within an individual simulated scan, not both.



-REPEAT *char* | *ascii value*

Initial Default : None. Must be defined by user.

Indicates the start of a repeat loop. Loop terminator character may be specified explicitly or by its ASCII value from 00 to 255 in decimal. ASCII values less than 10 must be preceded by a leading 0. The **-REPEAT** body must be terminated by a **-LOOP** directive (see above).

-SEP *char* | *ascii value*

Initial Default : ^ = 94

Separator character may be specified explicitly or by its ASCII value from 00 to 255 in decimal. ASCII values less than 10 must be preceded by a leading 0. This character must terminate the data for the current field; if the character is not found, processing of the template is terminated.

-TYPE *typename* | *typenumber*

Initial Default : 55 or CODE_39

The data field's returned bar code type in decimal or by symbology name. Table 10-4 lists valid *typename* and *typenumber* parameter values.

Table 10-4. Valid Values for *typenumber* and *typename* Parameters

<i>typenumber</i>	<i>typename</i>
48	UPC_E0
49	UPC_E1
50	UPC_A
51	MSI
52	EAN_13
53	EAN_8
54	CODABAR
55	CODE_39
56	CODE_D25
57	CODE_I25
58	CODE_11
59	CODE_93
60	CODE_128
61	PDF_417
--	KEY

The *typename* value KEY causes the data to be passed on to the application as though it was data entered from the keyboard.

Directives for each simulated scan are placed on one line in the template file, delimited by spaces and terminated by a carriage return. Lowercase letters are allowed for the directive fields. If a directive is not specified on a definition line, its default is used. Blank lines are ignored. At a minimum, the **-PDFLEN** directive must be declared on each definition line of the template file, excluding the **-ID**, **-KEYSTROKE**, **-COMMENT**, **-REPEAT** and **- LOOP** lines.

The following example illustrates the template file mode. (Bold faced type delineates data fields for this example, actual data is standard ASCII characters.)



Encoded Data:

**BOXTMPL1324178075021532120271^ATypeWidgets*07599237512219^
BTypeWidgets*0746452758201456^CTypeWidgets*0422828158280431^
D Type Widgets*#1422^**

Template file:

```
-COMMENT Example template file 3/12/93
-COMMENT Ensure that the template ID BOXTMPL carriage return
-COMMENT is found (ASCII code 13 decimal = CR)
-ID B O X T M P L 13
-COMMENT Get Box ID - Code 39, Encoded Length,
-COMMENT Pre-fill with 0)
-PDFLEN 5 -APPLEN 10 -PRE 0

-COMMENT Get data sequence until # is reached in data
-REPEAT #
  -COMMENT Get Code field - UPC A, Encoded Length 12
  -TYPE 50 -PDFLEN 12
  -COMMENT Get Quantity field - Keyboard entry,
  -COMMENT Variable length, separated by ^
  -TYPE KEY -PDFLEN 0
  -COMMENT Get Type field - Code 39, Variable length,
  -COMMENT separated by * (i.e., ASCII code 42 dec), and Beep
  -PDFLEN 0 -SEP 42 -BEEP
  -KEYSTROKE y ENTER
-LOOP

-KEYSTROKE n ENTER

-COMMENT Get Aisle number - Code 39, Encoded length 2
-PDFLEN 2
-COMMENT Get Shelf number - Interleaved 2 of 5,
-COMMENT Variable length, Post-fill with "0" (i.e., ASCII 48 dec)
-TYPE CODE_I25 -PDFLEN 0 -APPLEN 4 -POST 48
```

After scanning, the PDF label user views:

		Type	Beep
Scan Box ID	: 0000024178	Code 39	None
Scan Code	: 075021532120	UPC A	None
Key Quantity	: 271	Keyboard	None
Enter Type	: A Type Widgets	Code 39	Beep
Add Item?	: y(enter)	Keyboard	None
Scan Code	: 075992375122	UPC A	None
Key Quantity	: 19	Keyboard	None
Enter Type	: B Type Widgets	Code 39	Beep
Add Item?	: y(enter)	Keyboard	None
Scan Code	: 074645275820	UPC A	None
Key Quantity	: 1456	Keyboard	None
Enter Type	: C Type Widgets	Code 39	Beep
Add Item?	: y(enter)	Keyboard	None
Scan Code	: 042282815828	UPC A	None
Key Quantity	: 0431	Keyboard	None
Enter Type	: D Type Widgets	Code 39	Beep
Add Item?	: n(enter)	Keyboard	None
Enter Isle	: 14	Code 39	None
Enter Shelf	: 2200	I 2 of 5	Beep

Starting with the first byte of incoming PDF 417 decoded data, the template file defines separate label information buffer packets. These packets are sent individually, in order, to the application program. Errors in the template file will be detected during the BLDSCAN process and are shown in the file **TEMPLx.ERR**. Bounds and data validity checking are not performed on the outgoing packets. For example, if the PDF 417 decoded data is 212 bytes long and the template defines only 200 bytes, the remaining 12 bytes are discarded. If the user specifies that a field be treated as Interleaved 2 of 5 and an illegal character such as "T" is in the PDF 417 decoded data buffer, the illegal character will be passed to the application program. If application programs require more than 6 templates, templates may be compiled with **BLDTMPL.EXE** and linked into the application program. IOCTL calls must be made to switch between templates within the application program. Template file names must be in the format *fname*.PDF. Generated template files are named *fname*.H. These header



files may be added to user applications; in addition, the header file **PDFTEMPL.H** must be included before user template files.

Special Options

The Special Options (S) on the Decoder Selection Menu (see Figure 10-2 above) provide:

- IEC825 Class 1 Restrictions
- Bar Code Menus

IEC825 Class 1 Restrictions

IEC825 Class 1 is an international laser safety standard that limits the amount of energy emitted by the laser over a period of time. This option by itself does not constitute a complete IEC825 Class 1 system. The terminal must also have a BIOS supporting IEC825 Class 1. A terminal containing an IEC825 Class 1 BIOS will *not* respond to a SCAN3000 driver which is not configured for IEC825 Class 1.

IEC825 Class 1 limits the duration of laser scanner on-time, using an emission accumulator mechanism. The first time you use the laser scanner after the correct SCAN3000 driver is installed, you only have 2 seconds of scan time available. In essence, your emissions accumulator is nearly empty. If you wait 1000 seconds (approximately 17 minutes), you acquire the full 60 seconds of scan time. At that time, your emissions accumulator will be full. The laser cannot be on for more than 60 seconds in any 1000-second time period. One second of available laser scan time is earned back 1000 seconds after it is used. In other words, if you use one second of scan time, it takes 1000 seconds (approximately 17 minutes) to regain that one second of scan time.

When you have run out of scan time, the system emits one long low frequency beep. Once you have regained two seconds of scan time, the system emits another long high frequency beep to indicate that you have regained two seconds of scan time.

Note: Scan time is *not* expiring while you are scanning bar codes. Scan time only decrements if the laser beam is being emitted, but no bar codes are being read. So you may scan as many bar codes as you want, and no scan time is used.

Additionally, while scanning bar codes, your available scan time continues to increment toward the maximum 60 seconds.

See also IOCTL subcommands 2 and 6. These subcommands return the status of the scan time (subcommand 2) and scan time used and remaining (subcommand 6) values. For more information about IEC825 Class 1, contact Symbol Technical Support (phone number listed in the front of this manual).

Bar Code Menus

Bar Code Menus allow you to enable and disable decoders and parameters built into SCAN3000 by scanning special bar code menu labels. To include menus, Code 128 with variable length data must be included in SCAN3000. If you have not already selected it, you are asked if you want to include it. Answering No disables bar code menus.

Note: If you are using the PDT35XX-P terminal, Bar Code Menu is set to on, and IEC825 Class 1 is set to off. These options cannot be changed.

Refer to the *Parameter Descriptions* and the *Parameter Menu Scanning* sections of this chapter for descriptions of bar code menus and the *Bar Code Menu Labels* subsection for a list of bar code menu labels.

Scanning Sequence Examples

In most cases you need only to scan one bar code to set a specific parameter. For example, if you want to add Code 39, simply scan the **ADD CODE 39** bar code in the *CODE TYPES* subsection of the *Parameter Menu Scanning* section below.

If you want to set or change code type lengths, you may have to scan several bar codes. This procedure is described in the *Parameter Descriptions* section of this chapter.

Errors While Scanning

Don't worry if you make an error during a scanning sequence. Merely reenter the correct parameter.



Parameter Descriptions

Set Parameter Defaults

Scanning the **SET DEFAULT** bar code returns all parameters to the parameters that were specified when BLDSCAN was run.

Add/Delete Code Types

The bar code menu selections enable the scanner to decode any or all of the following symbologies:

- UPC Versions A and E (EAN 8 and 13)
- UPC-E1
- Interleaved 2 of 5
- Code 128
- Code 11
- Codabar
- Code 39
- Discrete 2 of 5
- MSI/Plessey
- Code 93

If you try to decode a symbol that the terminal cannot recognize, the symbol will be decoded, but not transmitted to the terminal. When selecting MSI/Plessey or Code 11, you must also select a number of check digits. In addition, you may choose to enable **ALL CODE TYPES**, or **COMMON CODE TYPES ONLY**. When common code types (Code 39, UPC/EAN, I 2 of 5 and Code 128) are enabled, all other code types are disabled.

Code Lengths

Code lengths for certain code types (i.e., Code 39, Codabar, etc.) may be set for any length, for one or two discrete lengths, or for lengths within a specific range. The length of a code refers to the number of characters (i.e., human readable characters) the code contains. *When calculating lengths for **Codabar**, the start and stop characters must be included.* For calculating MSI Plessey and Code 11 code lengths, see the **Transmit MSI Check Digits** and **Transmit Code 11 Check Digits** parameters.

Length Within Range

This option allows you to decode a code type within a specified range. For example, to decode Code 39 characters containing between 4 and 12 characters, first scan **Code 39 Length Within Range**. Then scan **0, 4, 1** and **2** (single digit numbers must always be preceded by a leading zero).

One Discrete Length

This option will allow you to decode only those codes containing a selected length. For example, if you select **I 2 of 5 One Discrete Length** and then scan **1, 4**, the only I 2 of 5 codes decoded will be those containing 14 characters.

Two Discrete Lengths

This option will allow you to decode only those codes containing two selected lengths. For example, if you select **I 2 of 5 Two Discrete Lengths** and then scan **0, 2, 1, 4**, the only I 2 of 5 codes decoded will be those containing 2 or 14 characters.

Any Length

Scanning this option allows you to decode the selected code type containing any number of characters. For example, if you scan **Codabar Any Length**, you will be able to decode a Codabar symbol containing any number of characters.

Code 39 Full ASCII

The ASCII character set assigns a code to letters, punctuation marks, numerals, and most control keystrokes on the keyboard. The bar code menu for this option is shown below with the *Decode Options* bar codes.



The first 32 codes are non-printable and are assigned to keyboard control characters such as BACKSPACE and RETURN. The other 96 are called printable codes because all but SPACE and DELETE produce visible characters.

Code 39 Full ASCII interprets the bar code control character (\$ + % /) preceding a Code 39 symbol and assigns an ASCII character value. For example, when Code 39 Full ASCII is enabled and a **+B** is scanned, it will be interpreted as **b**, **%J** will be interpreted as **?**, and **\$H** will emulate the keystroke **BACKSPACE**. Scanning **ABC\$M** will output the keystroke equivalent of **ABC ENTER**.

The scanner will not autodiscriminate between Code 39 and Code 39 Full ASCII.

Decode Options

Convert UPC-E0 to UPC-A

Use this parameter to convert UPC-E (zero suppressed) decoded data to UPC-A format before transmission. After conversion, data will follow UPC format and be affected by UPC-A programming selections (e.g., Preamble, Check Digit).

Convert UPC-E1 to UPC-A

Use this parameter to convert UPC-E1 decoded data to UPC-A format before transmission. After conversion, data will follow UPC format and be affected by UPC-A programming selections (e.g., Preamble, Check Digit).

EAN Zero Extend

This parameter adds five leading zeros to decoded EAN-8 symbols to make them compatible in format to EAN-13 symbols.

Decode UPC/EAN Supplemental

Select whether UPC/EAN is decoded with or without supplemental characters. Supplementals are additionally appended characters (2 or 5) according to specific code format conventions (e.g., UPC A+2, UPC E+2, EAN 8+2). If UPC/EAN with supplemental characters is selected, UPC/EAN symbols without supplemental characters won't be decoded. If UPC/EAN without supplemental characters is selected and the scanner is presented with a UPC/EAN plus supplemental symbol, the UPC/EAN will be decoded and the supplemental characters ignored. If autodiscrimination is chosen, the LS 2502 will, after additional processing to ensure a good decode, transmit either.

Note: In order to minimize the risk of invalid data transmission, it is recommended that you select whether to read or ignore supplemental characters.

NOTIS Editing

This option strips the start and stop characters from decoded Codabar symbol.

CLSI Editing

Use this parameter to strip the start and stop characters and then insert a space after the 1st, 5th, and 10th characters of a 14-character Codabar symbol. Note that the symbol length does not include start and stop characters.

Verify Code 39 Check Digit

When enabled, this parameter checks the integrity of a Code 39 symbol to ensure it complies with a modulo 43 check digit algorithm. This check digit is transmitted with the data.

MSI Plessey Check Digit

One or two digits at the end of the bar code that check the integrity of the data. At least one check digit (default) is always required. Check digits are not transmitted with the data, unless selected (see **Transmit MSI Plessey Check Digit** below).



Transmit MSI Plessey Check Digit

Selecting this parameter will transmit the chosen number of check digits with the data. When disabled, the code length is calculated the same as other variable length codes. (Check digit(s) not counted.) When enabled, include the number of check digits in calculating the length. For example, to transmit a 10 element MSI Plessey bar code with two check digits, select a length of 12 (10 elements + 2 check digits).

Code 11 Check Digit

Select whether to decode Code 11 bar codes with none, one or two check digits.

Transmit Code 11 Check Digit

Selecting this parameter will transmit the chosen number of check digits with the data. When disabled, the code length is calculated the same as other variable length codes (Check digit(s) not counted). When enabled, include the number of check digits in calculating the length. For example, to transmit a 10 element Code 11 bar code with two check digits, select a length of 12 (10 elements + 2 check digits).

Transmit UPC-E/UPC-A/UPC-E1 Check Digit

Select if decoded UPC symbols are transmitted with or without a check digit.

Decode Redundancy for UPC/EAN Without Supplementals

With **Autodiscriminate UPC/EAN Supplementals** enabled, this option adjusts the number of times a symbol without a supplemental must be decoded before transmission. The range is from two to ten times. Five or above is recommended when decoding a mix of UPC/EAN symbols with and without supplementals.

UPC/EAN Security Level

The SCANKIT offers four levels of decode security for UPC/EAN bar codes. Increasing levels of security are provided for decreasing levels of bar code quality. There is an inverse relationship between security and scanner aggressiveness, so be sure to choose only that level of security necessary for any given application.

- ***Security Level 0***

This is the default setting which allows the scanner to operate in its most aggressive state, while providing sufficient security in decoding “in spec” UPC/EAN bar codes.

- ***Security Level 1***

As bar code quality levels diminish, certain characters become prone to mis-decodes before others (i.e., 1, 2, 7, 8). If you are experiencing mis-decodes of poorly printed bar codes and the mis-decodes are limited to these characters, select this security level.

- ***Security Level 2***

If you are experiencing mis-decodes of poorly printed bar codes and the mis-decodes are not limited to characters 1, 2, 7 and 8, select this security level.

- ***Security Level 3***

If you have tried Security Level 2 and are still experiencing mis-decodes, select this security level.

Caution

Be advised that selecting this option is an extreme measure against mis-decoding severely out of spec bar codes. Selection of this level will significantly impair the decoding ability of the scanner.

Special Decode Options

These options are provided for enhanced decode security on certain poor quality bar codes (e.g., poorly printed or low contrast), and for insecure symbologies such as Interleaved 2 of 5. Following the descriptions of these decode options is a default table, and a matrix defining which options apply to specific types of hosts.

Simple Redundancy

When enabled, a bar code will be transmitted only when there are two decodes with the same result. This option can be enabled for selected code types on an individual basis.



Bi-directional Redundancy

When enabled, a bar code must be decoded in both directions before being accepted as a successful decode. The option only applies to those code types which have **Simple Redundancy** already enabled.

Linear UPC/EAN Decode

This option applies to those code types containing two adjacent blocks (e.g. UPC-A, EAN-8 and EAN-13). When enabled, a bar code will be transmitted only when both the left and right blocks are successfully decoded within one laser scan. This option should be enabled when bar codes are in proximity to each other.

Transmit Code ID Character

A code ID character identifies the code type of a scanned bar code. This may be useful when the scanner is decoding more than one code type. If a prefix is selected, the code ID character is sent after the prefix. Code ID characters are: A = UPC-A, UPC-E, EAN-13, or EAN-8; B = Code 39; C = Codabar; D = Code 128; F = Interleaved 2 of 5.

UPC A, E, and E1 Preambles

Three options are given for the lead-in characters of decoded UPC-A, UPC-E or UPC-E1 symbols transmitted to the host device. Select one preamble for each code type. These lead-in characters are considered part of the symbol itself. The three options are:

- a system character only
- the country code and system character
- no preamble

The system character is the digit printed to the extreme left of a UPC symbol. For UPC-A and UPC-E, the country code is always 0. For UPC-E1, the country code is 0, and the system character is 1. The system character is always transmitted with the decoded data.

Parameter Menu Scanning

Bar Code Menus allow you enable and disable decoders and parameters built into the scanner driver by scanning special bar code menu labels. This section contains the bar code labels that can be used for this purpose.

The following chart shows the beeper sequence and the associated indication for barcode scans in this section.

BEEPER SEQUENCE	INDICATION
3 Beeps	Correct entry scanned.
2 Beeps	Value entered (additional entries are expected).
1 Long Beep	Input error, or "CANCEL" is scanned.

Tables 10-5 through 10-7 list bar code menu parameter defaults. The associated bar code menu labels are displayed with captions on the pages that follow. These are the defaults in the SCAN3000.EXE supplied on the SCANKIT release disk. They are the defaults selected in BLDSCAN by the **Select Defaults (F3)** command.

Table 10-5. Bar Code Menu Parameters: Code Types

Code Type	Length	Default Value
UPC-A	12	Enabled
UPC-E0	6	Enabled
UPC-E1	6	Disabled
EAN-8	8	Enabled
EAN-13	13	Enabled
D 2 of 5	12	Enabled
I 2 of 5	14	Enabled
Code 39	Variable	Enabled
Codabar	5 to 55	Enabled
Code 128	Variable	Enabled
Code 93	4 to 55	Disabled



Table 10-5. Bar Code Menu Parameters: Code Types (Continued)

Code Type	Length	Default Value
Code 11	4 to 55	Disabled
MSI Plessey	4 to 55	Disabled
PDF417	Variable	Enabled (for PDT 35XX terminal)

Table 10-6. Bar Code Menu Parameters: Decode Options

Decode Option	Default Value
Transmit UPC-E0 Check Digit	Disabled
Transmit UPC-E1 Check Digit	Disabled
Transmit UPC-A Check Digit	Enabled
Convert UPC-E0 to UPC-A	Disabled
Convert UPC-E1 to UPC-A	Disabled
EAN Zero Extend	Disabled
Code 39 Full ASCII	Disabled
CLSI Editing	Disabled
NOTIS Editing	Disabled
MSI Plessey Check Digit	One
Transmit MSI Plessey Check Digit	Disabled
Code 11 Check Digit	One
Transmit Code 11 Check Digit	Disabled
Verify Code 39 Check Digit	Disabled
Decode Redundancy for UPC/EAN without Supplementals	5
UPC/EAN Security level	0

Table 10-7. Bar Code Menu Parameters: Special Decode Options

Special Decode Option	Default Value
Simple Redundancy:	
Code 39	Disabled
D 2 of 5	Disabled
I 2 of 5	Disabled
Code 128	Disabled
Codabar	Enabled
Code 11	Disabled
MSI Plessey	Disabled
Code 93	Disabled
Bi-Directional Redundancy	Disabled
Linear UPC/EAN Decode	Enabled
UPC-E0 Preamble	None
UPC-E1 Preamble	None
UPC-A Preamble	System Character
Transmit Code ID Character	Disabled

Bar Code Menu labels begin on the next page.



Bar Code Menu Labels

Set Default Parameter

Defaults are those specified when BLDSCAN was run. Refer to the *Running BLDINIT* section of the *Terminal Initialization* chapter in this guide for a description of the BLDINIT utility.



SET ALL DEFAULTS
Not for use with
SCAN3500

Note: In order to use the following bar codes, you must first ENABLE BAR CODE MENUS in BLDSCAN.

PDF 417 Scanning Mode Options

When using a PDT 35XX-P terminal, there are two main scanning options:

- **SLAB RASTER:**

A trigger pull creates the slab raster pattern. If the target is a 1-D bar code, the pattern never gets beyond a slab raster. However, if the target bar code is PDF417, the pattern opens up to an optimized raster pattern as soon as the scanner is properly aligned over the bar code.

- **AIMING DOT:**

A trigger pull creates a single dot aiming pattern, which lasts for a fixed interval. This dot can be seen easily in outdoor or high ambient light environments. A slab raster pattern appears next, depending on the programmed scanning option.



SLAB RASTER
Not for use with SCAN3000



AIMING DOT
(Normal Timeout)
Not for use with SCAN3000



Code Types

Add or delete specific code types by scanning the appropriate bar code(s).



ENABLE ALL CODE TYPES
Not for use with SCAN3500



DELETE ALL CODE TYPES
Not for use with SCAN3500



**ENABLE COMMON CODE
TYPES ONLY**
Not for use with SCAN3500



ADD CODE 39



DELETE CODE 39



ADD UPC-A



DELETE UPC-A



ADD UPC-E0



DELETE UPC-E0

Code Types (Continued)



ADD UPC-E1



DELETE UPC-E1



ADD EAN-8



DELETE EAN-8



ADD EAN-13



DELETE EAN-13



ADD I 2 OF 5



DELETE I 2 OF 5



ADD CODABAR



DELETE CODABAR



Code Types (Continued)



ADD CODE 128



DELETE CODE



ADD MSI/Plessey*



DELETE MSI/Plessey



ADD CODE 11**



DELETE CODE 11



ADD CODE 93



DELETE CODE 93



ADD D 2 OF 5



DELETE D 2 OF 5

*After adding MSI/Plessey you must select either one or two check digits from the *Decode Options* (see below).

**After adding Code 11, you must select none, one, or two check digits from the *Decode Options* (see below).

Code Types (Continued)



ADD PDF417
Not for use with SCAN3000



ADD UCC/EAN-128
Not for use with SCAN3000



DELETE PDF417
Not for use with SCAN3000



DELETE UCC/EAN-128
Not for use with SCAN3000



Code Lengths

To select two lengths for each code type:

1. Scan the desired option.
2. Scan two bar codes from page 10-56 for each desired length. For example, for a length of "12", scan "1" and then "2". For a length of "3", scan "0" and then "3". **You must always scan two bar codes for each length.**
3. If you make an error or wish to change your selection, scan **CANCEL**



CODE 39 ANY LENGTH



CODE 39 LENGTH
WITHIN RANGE



CODE 39 1 DISCRETE
LENGTH



CODE 39 2 DISCRETE
LENGTHS

Code Lengths (Continued)



CODABAR ANY LENGTH



**CODABAR LENGTH
WITHIN RANGE**



**CODABAR 1 DISCRETE
LENGTH**



**CODABAR 2 DISCRETE
LENGTHS**



Code Lengths (Continued)



CODE 128 ANY LENGTH



**CODE 128 LENGTH
WITHIN RANGE**



**CODE 128
1 DISCRETE LENGTH**



**CODE 128
2 DISCRETE LENGTHS**

Code Lengths (Continued)



D 2 OF 5 ANY LENGTH



D 2 OF 5 LENGTH
WITHIN RANGE



D 2 OF 5 1 DISCRETE
LENGTH



D 2 OF 5 2 DISCRETE
LENGTHS



Code Lengths (Continued)



CODE 93 ANY LENGTH



**CODE 93 LENGTH
WITHIN RANGE**



**CODE 93
1 DISCRETE LENGTH**



**CODE 93
2 DISCRETE LENGTHS**

Code Lengths (Continued)

Note: Choosing I 2 of 5 ANY LENGTH may lead to misread codes.



I 2 OF 5 ANY LENGTH*



**I 2 OF 5 LENGTH
WITHIN RANGE**



**I 2 OF 5 1 DISCRETE
LENGTH**



**I 2 OF 5 2 DISCRETE
LENGTHS**



Code Lengths (Continued)



**MSI/Plessey
ANY LENGTH**



**MSI/Plessey
LENGTH WITHIN RANGE**



**MSI/Plessey 1 DISCRETE
LENGTH**



**MSI/Plessey 2 DISCRETE
LENGTHS**

Code Lengths (Continued)



CODE 11
ANY LENGTH



CODE 11 LENGTH WITHIN
RANGE



CODE 11 1 DISCRETE
LENGTH



CODE 11 2 DISCRETE
LENGTHS



Code Lengths (Continued)



0



1



2



3



4



5



6



7



8



9



CANCEL

Decode Options



TRANSMIT UPC-E0 CHECK DIGIT



DO NOT
TRANSMIT UPC-E0 CHECK DIGIT



TRANSMIT UPC-E1 CHECK DIGIT



DO NOT
TRANSMIT UPC-E1 CHECK DIGIT



TRANSMIT UPC-A CHECK DIGIT



DO NOT
TRANSMIT UPC-A CHECK DIGIT



Decode Options (Continued)



CONVERT UPC-E0 TO UPC-A



**DO NOT
CONVERT UPC-E0 TO UPC-A**



CONVERT UPC-E1 TO UPC-A



**DO NOT
CONVERT UPC-E1 TO UPC-A**



ENABLE EAN ZERO EXTEND



DISABLE EAN ZERO EXTEND



**ENABLE CODE 39
FULL ASCII**



**DISABLE CODE 39
FULL ASCII**

Decode Options (Continued)



**DECODE CODE 11 WITH
NO CHECK DIGITS**
(Not for use with SCAN3500)



**DECODE CODE 11 WITH
1 CHECK DIGIT**
(Not for use with SCAN3500)



**DECODE CODE 11 WITH
2 CHECK DIGITS**
(Not for use with SCAN3500)



**TRANSMIT CODE 11
CHECK DIGIT(S)**
(Not for use with SCAN3500)



**DO NOT TRANSMIT CODE 11
CHECK DIGIT(S)**
(Not for use with SCAN3500)



Decode Options (Continued)



decode Code 11 with no check digits
(For use with SCAN3500)



decode code 11 with 1 check digit
(For use with SCAN3500)



decode code 11 with 2 check digits
(For use with SCAN3500)



Transmit code 11 check digits
(For use with SCAN3500)



Do not transmit code 11 check digits
(For use with SCAN3500)

Decode Options (Continued)



**DECODE MSI/Plessey WITH
1 CHECK DIGIT
(Not for use with SCAN3500)**



**DECODE MSI/Plessey WITH
2 CHECK DIGITS
(Not for use with SCAN3500)**



**TRANSMIT
MSI/Plessey CHECK DIGITS
(Not for use with SCAN3500)**



**DO NOT TRANSMIT
MSI/Plessey CHECK DIGITS
(Not for use with SCAN3500)**



**DECODE UPC/EAN
SUPPLEMENTALS**



**IGNORE UPC/EAN
SUPPLEMENTALS**



**AUTODISCRIMINATE UPC/EAN
WITH SUPPLEMENTALS**



Decode Options (Continued)



**Decode MSI with 1 check digit
(For use with SCAN3500)**



**Decode MSI with 2 check digits
(For use with SCAN3500)**



**Transmit MSI check digits
(For use with SCAN3500)**



**Do not transmit msi check digits
(For use with SCAN3500)**

Decode Options (Continued)

Select the type of supplemental when DECODE UPC/EAN SUPPLEMENTALS is ENABLED.



Enable 2 Characters Supplemental



Disable 2 Characters Supplemental



Enable 5 Characters Supplemental



Disable 5 Characters Supplemental



Decode Options (Continued)



ENABLE CLSI EDITING



DISABLE CLSI EDITING



ENABLE NOTIS EDITING



DISABLE NOTIS EDITING



**VERIFY CODE 39
CHECK DIGIT**



**DO NOT VERIFY CODE 39
CHECK DIGIT**



TRANSMIT CODE ID CHARACTER
Not for use with SCAN3500



**DO NOT TRANSMIT CODE
ID CHARACTER**
Not for use with SCAN3500



**TRANSMIT SYMBOL CODE ID CHARAC-
TER**



TRANSMIT AIM CODE ID CHARACTER
Not for use with SCAN3000

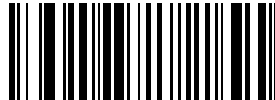


DO NOT TRANSMIT CODE ID CHARACTER
Not for use with SCAN3000

Decode Options (Continued)



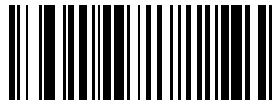
UPC/EAN SECURITY LEVEL 0
Not for use with SCAN3500



UPC/EAN SECURITY LEVEL 0
Not for use with SCAN3000



UPC/EAN SECURITY LEVEL 1
Not for use with SCAN3500



UPC/EAN SECURITY LEVEL 1
Not for use with SCAN3000



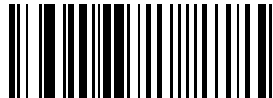
UPC/EAN SECURITY LEVEL 2
Not for use with SCAN3500



UPC/EAN SECURITY LEVEL 2
Not for use with SCAN3000



UPC/EAN SECURITY LEVEL 3
Not for use with SCAN3500



UPC/EAN SECURITY LEVEL 3
Not for use with SCAN3000



Decode Options (Continued)

Decode Redundancy for UPC/EAN without Supplementals

To set a decode redundancy value:

1. Scan the DECODE REDUNDANCY bar code below.
2. Scan two bar codes on the next page that represent the desired number of times. For single digit numbers, include a leading zero.
3. If you make an error, or wish to change your selection, scan CANCEL.



**DECODE REDUNDANCY for
UPC/EAN without
SUPPLEMENTALS
(Not for use with SCAN3500)**



**DECODE REDUNDANCY for
UPC/EAN without
SUPPLEMENTALS
(For use with SCAN3500)**

Decode Options (Continued)

Decode Redundancy for UPC/EAN without Supplementals



0



1



2



3



4



5



6



7



8



9



CANCEL



UPC-A Preamble

Select one option for UPC-A preamble by scanning the appropriate bar code.



NONE



SYSTEM CHARACTER



**SYSTEM CHARACTER
&
COUNTRY CODE**

UPC-E0 Preamble

Select one option for UPC-E0 preamble by scanning the appropriate bar code.



NONE



SYSTEM CHARACTER



**SYSTEM CHARACTER
&
COUNTRY CODE**



UPC-E1 Preamble

Select one option for UPC-E1 preamble by scanning the appropriate bar code.



NONE



SYSTEM CHARACTER



**SYSTEM CHARACTER
&
COUNTRY CODE**

Special Decode Options

Simple Redundancy



ENABLE CODE 39
SIMPLE REDUNDANCY



DISABLE CODE 39
SIMPLE REDUNDANCY



ENABLE CODE 128
SIMPLE REDUNDANCY



DISABLE CODE 128
SIMPLE REDUNDANCY



ENABLE I 2 OF 5
SIMPLE REDUNDANCY



DISABLE I 2 OF 5
SIMPLE REDUNDANCY



ENABLE CODABAR
SIMPLE REDUNDANCY



DISABLE CODABAR
SIMPLE REDUNDANCY



Special Decode Options (Continued)

Simple Redundancy



**ENABLE MSI/ Plessey
SIMPLE REDUNDANCY**



**DISABLE MSI/Plessey
SIMPLE REDUNDANCY**



**ENABLE CODE 11
SIMPLE REDUNDANCY**



**DISABLE CODE 11
SIMPLE REDUNDANCY**



**ENABLE D 2 of 5
SIMPLE REDUNDANCY**



**DISABLE D 2 of 5
SIMPLE REDUNDANCY**



**ENABLE CODE 93
SIMPLE REDUNDANCY**



**DISABLE CODE 93
SIMPLE REDUNDANCY**

Special Decode Options (Continued)

Bi-Directional Redundancy

Enable or disable bi-directional redundancy for all those codes with *Simple Redundancy* enabled.



ENABLE BIDIRECTIONAL
REDUNDANCY



DISABLE BIDIRECTIONAL
REDUNDANCY



Special Decode Options (Continued)

Linear UPC/EAN Decode



**ENABLE LINEAR
UPC DECODE**



**DISABLE LINEAR
UPC DECODE**

Check Digits, Code 11 and MSI

Code 11 and MSI decoders use check digits: 0, 1, or 2 for Code 11, 1 or 2 for MSI. Select the number of check digits in response to the Check Digits prompt.

Code 11 Check Digits		
A.	ZERO	
B.	ONE	
C.	TWO	
Enter to Accept		

Figure 10-7. Check Digits Prompt

After selecting the number of check digits, you are asked whether or not the check digit character should be returned to the label information buffer:

Report Check Digit(s)? (Y/N) [N] ?

Answering “Yes” returns the character to the buffer; answering “No” suppresses the character. Report the check digit only if the application needs the character.

Check Digits, Code 39

Code 39 check digits, if used, are part of the regular data characters and are reported to the application whether they are checked or not. Answering “Yes” to the Show/use C39 Check prompt causes the decoder to verify as well as report the check digit. If the check digit fails, the label is not returned as data.

Label Expansion Prompt

The UPC-E0 and UPC-E1 decoders are compressed UPCA decoders, with data compressed from 12 to 6 characters. EAN8 (8 characters) is compressed EAN13 (13 characters). Enabling expansion decompresses the data. Labels are decompressed by padding with zeros.



When the Enable Label Expansion prompt is displayed, press Y to enable or N to disable expansion:

Enable UPC Expansion?
(Y/N) [N] ?

Code 39 ASCII

Code 39 permits combinations of Code 39 characters to represent any character in the ASCII character set. The Enable Full ASCII prompt allows you to enable or disable this feature:

Enable Full ASCII?
(Y/N) [N] ?

Be aware that enabling Full ASCII may cause the encoded data length (two characters) to be longer than the decoded data length (one character). The data length specified in the Decoder Selection Screen is the encoded data length.

Modifying the Scanner, Reader, and Decoders

The SCANPARM Utility

SCANPARM.EXE comes as part of the default NVM (non-volatile memory) configuration in the Series 3000 terminals. It can be used to modify the scanner, reader, and decoder. For effective operation this utility must be loaded on a Series 3000 terminal on which SCAN3000.EXE has also been loaded.

Note: SCAN3000.EXE must be loaded *before* SCANPARM.EXE.

To run SCANPARM from the command line or from a batch file, use the following format:

```
SCANPARM [/C] [filename.ini]
```

When executed, SCANPARM checks for command line arguments. If the /C argument is found, it loads the file specified in the *filename.ini* parameter into an internal data structure, merges it with the current scanner data structure, updates the scanner, and then exits. If only the *filename.ini* parameter is provided, the initialization file specified is loaded into a temporary data structure and merged with the current scanner settings, and the Main menu is presented for further actions. If no command line arguments are given with **SCANPARM**, the current scanner data is read and the Main menu is presented.

From the Main menu you may select the following:

- Save to disk
- Update Scanner
- Edit menu
- Exit



Selecting Edit Menu will present the next menu, from which you may edit the DECODERS, the CHARACTERISTICS, the PARAMETERS, the MODES or MISCELLANEOUS features of the terminal scan functions. You may select an item for editing by either using the UP/DOWN arrows or pressing the character of the first word of the item you desire to edit. Press the ENTER key and you are presented with each item to change.

Decoders

Selecting DECODERS from the Edit menu allows the user to configure the parameters for many types of industry standard bar code labels. The configuration parameters are listed on the screen by Label Type, Enable/Disable, Min, Max, Mode, and Dep.

Navigate and change the values for Label Types by using:

<ENTER>:	to <i>accept</i> and <i>move</i> to the next label
<UP Arrow>:	to <i>increase</i> the <i>value</i> of the field
<DOWN Arrow>:	to <i>decrease</i> the <i>value</i> of the field
<SPACE>:	to <i>move</i> to the next field
<BACKSPACE>:	to <i>toggle Enable/Disable</i> .
<ESC>:	to <i>return</i> to the previous menu

Enable/Disable enables or disables the scanner's ability to recognize the Label Type when scanned.

The Min value is the minimum length of a label in the particular label type.

The Max value is the maximum length of a label for the particular label type.

Mode is used only when selecting parameters associated with SUPPS (supplementals).

The Dep (Decoder Dependent) setting sets extended options for certain decoders. (See individual Label Types).

The Min and Max are fixed for UPC label types; entering other values does not change UPC settings.

LABEL TYPE	ENABLE/ DISABLE	MIN	MAX	MODE	DEP
CODE 39	Enable	1	32	N/A	N/A
UPC A	Enable	12 Fixed	12 Fixed	N/A	N/A
UPC E0/ UPC E1	Enable	6 Fixed	12 Fixed	N/A	*See DepNote 3
EAN 13	Enable	13 Fixed	13 Fixed	N/A	N/A
EAN 8	Enable	8 Fixed	8 Fixed	N/A	N/A
CODE D2 OF 5	Enable	1	32	N/A	N/A
CODE I 2 OF 5	Enable	2	32 even	N/A	N/A
Codabar	Enable	1	32	N/A	N/A
Code 128	Enable	1	32	N/A	N/A
Code 93	Enable	1	32	N/A	N/A
Code 11	Enable	1	32	N/A	*See DepNote 1
MSI	Enable	1	32	N/A	*See DepNote 2

Figure 10-8. DECODERS from SCANPARM Edit Menu

**DepNote1*

Value	Meaning
0	No Check Digit
1	1 Check Digit
2	2 Check Digits

**DepNote2*

Value	Meaning
0	1 Check Digit
1	2 Check Digits



*DepNote3

Value	Meaning
0	Labels are 6 characters in length fixed
1	Labels are 6 characters (compressed). Expand to 12 when read.

There is also a SUPPS (Supplemental) Label Type which has different parameters from those listed above. They are:

Disable/Enable Set by the value of the Mode parameter below:

Mode Parameter	Value	Meaning
Two	0	Don't Scan supplemental labels of length two.
	1	Scan supplemental labels of length two.
Five	0	Don't Scan supplemental labels of length five.
	1	Scan supplemental labels of length five.
Mode	0	Don't scan supplementals (Disable Set).
	1	Scan supplementals (Enable Set).
	2	Auto discriminate. (If a supplemental is found scan it - Enable Set).

Characteristics, Parameters, Modes, and Miscellaneous Navigation / Modification

Each item of the selected edit function is presented with its present value. You may then use the following keys to navigate or to modify the presented fields:

<ENTER>	Accept value and present next field. If last field, return to Edit Menu.
<ESC>	Return to Edit Menu.
<UP Arrow> or <DOWN Arrow>	Present next or previous item in enumerated set. Wrap to top or bottom as required.
	Integer (numeric) fields: Increment or decrement field.
<BACKSPACE>	Integer (numeric) fields: Move cursor to left and erase character. Enumerated fields: No effect.
<LEFT Arrow>	Same as BACKSPACE.
<SPACE>	No effect.



Characteristics

When Characteristics is selected from the Edit Menu, the screen blanks and three lines of data are presented. The first line is a title line indicating the section being edited, the second line is the field being edited, and the third line is the current value of the field. All of the fields for Characteristics are Boolean and are presented with default values of "True" or "False." You may change the current value of the field by using the UP or DOWN arrows to toggle the value for the field.

```
[Characteristics]
```

```
Example:Trigger
```

```
true
```

The following are Characteristic fields and their associated default values:

Characteristic Field	Default Value
Trigger	True
Multi_Scan	True
Direction	True
Feedback	True
Enable_Used	False

Parameters

The Parameters for the scanner are a combination of Boolean, enumerated set, and integer fields. Integers are displayed as 4-digit numbers and may be changed by backspacing or using the left arrow key and then entering the number desired, or by using the UP/DOWN arrows to increment /decrement the value. Boolean and enumerated sets are modified by using the UP and DOWN arrows.

Examples:

Integer (numeric) field

[PARAMETERS]

Enable_Set_Time

1000

Boolean field

[PARAMETERS]

ClkSpeedToggle

false

Enumerated sets

[PARAMETERS]

DecodeFailAct

STOP

The Parameter fields and defaults are defined below. Note that numeric fields are depicted by a vertical bar, the least possible range of the numeric value, an ellipsis (...), the maximum possible range of the value, and a vertical bar. Enumerated sets and Boolean fields are depicted with the available selections listed within {} braces.



Parameter Field	Default Value	Integer (Numeric) Field/ Boolean Field/ Enumerated Set
Enable_Set_Time	1000	0...65535
Power_Set_Time	3000	0...65535
Feedback_Log_Level	high	{high,low}
White_Data_Log_Lev	low	{high,low}
BeepOnDecode	false	{true,false}
BeepTime	100	0...255
ScansPerLabel	1	0...255
ClkSpeedToggle	false	{true,false}
TransRes	8	{2,4,8,16}
LabelTermChar	0	0...255
InactiveTime	30	0...65535
QuietZoneRatio	8	0...255
InitScanTime	0	0...255
PulseDelay	45	0...255
SubsScanTime	3	0...255
NoDataTime	30	0...255
PostDecodeAct	PULSE	{PULSE,ACQUIRE,STOP}
DecodeFailAct	PULSE	{PULSE,ACQUIRE,STOP}

Extensions

Extensions are Parameters for the scanner which are supported for SCAN3000 version 3.XX and higher. Extensions are either Boolean or integer (numeric) fields. Integers are displayed as 4-digit numbers and may be changed by backspacing or using the left arrow key and then entering the number desired, or by using the UP/DOWN arrows to increment /decrement the value. Boolean fields are modified by using the UP and DOWN arrows.

Note: If you are using a SCAN3000 version below 3.XX, these parameters will have no effect.

Ranges of valid integer field values and default values for integer and Boolean fields are listed below:

		Default	Range
D2 of 5	Redundancy	false	{true,false}
I2 of 5	Redundancy	false	{true,false}
Code39	Redundancy	false	{true,false}
Codabar	Redundancy	true	{true,false}
Code128	Redundancy	false	{true,false}
Code93	Redundancy	false	{true,false}
Code11	Redundancy	false	{true,false}
MSI	Redundancy	false	{true,false}
BiDir	Redundancy	false	{true,false}
Code39	CheckByte	false	{true,false}
Code11	CheckDigit	1	0...2
ReportCode11	ChkD	false	{true,false}
MSI	CheckDigit	1	0...1
ReportMSI	ChkD	false	{true,false}
UPC_A_	CheckByte	false	{true,false}
UPC_E_	CheckByte	false	{true,false}
UPC_E1_	CheckByte	false	{true,false}
LinearUPC		true	{true,false}
NoSupp	Max	5	2...10
UPC_EAN	SecLevel	0	0...3
ConvEAN	8To13	false	{true,false}
Conv_UPC_	EToA	false	{true,false}



		Default	Range
Conv_UPC_	E1ToA	false	{true,false}
UPC_A	Preamble	1	0...2
UPC_E	Preamble	0	0...2
UPC_E1	Preamble	0	0...2
SUPPS_	Two	false	{true,false}
SUPPS_	Five	false	{true,false}
SUPPS_	AutoDisc	0	0...2
CLSI_	Editing	false	{true,false}
NOTIS_	Editing	false	{true,false}
TransCode	IdChar	false	{true,false}
Code39	FullAscii	false	{true,false}

Modes

The Modes section of the Edit Menu presents a single enumerated field.

Use the UP/DOWN arrows to select one of 5 possible settings.

[MODES]

Example: Scan_Mode

Laser Scanner

The values and defaults of the MODES enumerated set are:

Mode Value	Mode Default
Scan_Mode	Auto Select
	{Contact Wand,
	Laser Scanner,
	CCD Gun,
	Auto Select,
	None}

Miscellaneous

The Miscellaneous selection of the Edit menu also has only one field associated with it. This is an enumerated set with two possible settings. Use the UP/DOWN arrows to enable or disable the spotting beam of a laser scanner.

`[MISCELLANEOUS]`

Example: `SpottingBeam`

`enable`



Creating the Initialization File

The easiest method of creating an scanner initialization file is to select the **Save To Disk** option from the SCANPARM Main Menu. This will take the current contents of the internal data structures and write them out to a disk file titled **SCANPARM.INI**. This file may then be passed on the command line to **SCANPARM.EXE** to configure the scanner, readers, and decoders in the 3000 series portable data terminals. Since this file is an ASCII text file, it may be created or edited using any available editor, e.g., EDLIN, EDIT, Programmers WorkBench, Brief, etc.

Refer to the section on *Modifying the Scanner, Reader, and Decoders* earlier in this chapter for more information on the SCANPARM utility and how to invoke it.

Updating the Scanner, Readers, and Decoders

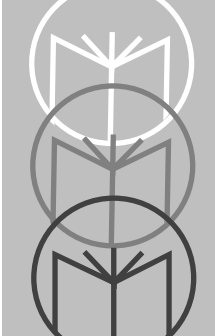
Updating the Series 3000 scanner may be accomplished using either of the following methods:

After editing and selecting **Save To Disk** from the SCANPARM utility Main Menu, exit SCANPARM and then invoke it again using the /C switch and the **SCANPARM.INI** file.

OR

Use the Update Scanner selection of the SCANPARM Main Menu to load the scanner with the current contents of the internal data structures. After exiting, these parameters are then available for use by subsequently invoked programs and utilities.





Chapter 11

Data Communications

Chapter Contents

Introduction	11-3
Using the URM API	11-12
Using the DR DOS API	11-30
Using the BIOS API	11-34
Sample Communication Programs	11-36
Managing Terminal Resources	11-41
Unattended Communications	11-43
Communicating Through a Cradle	11-44
Communication Problems	11-50
Remote PC Communication Software	11-54



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK documents:

Document Title(s)	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i> <i>BIOS Library Functions</i> <i>DR DOS</i> <i>UBASIC Record Manager</i> <i>Internal Modem Command Set</i>
<i>Series 3000 System Software Manual</i>	<i>ROM BIOS</i>

Communications is a major part of many application programs for the Series 3000 terminal. In many applications, a Series 3000 terminal spends most of the time operating as an independent, hand-held computer dedicated to data collection. At some point the terminal must use some form of data communication to transfer the data collected to the host system.

When a Series 3000 terminal is communicating with a host computer, it becomes part of the host computer system. It must use the data format required by the host, and it must match exactly the host's communication protocol.

Communications between a Series 3000 terminal and a host computer may be conducted over:

- a direct serial link
- telephone lines using a modem
- a Spectrum One radio network

Each of these methods of data communication can also involve a cradle. Each method also involves certain technical problems and varying levels of complexity that must be addressed by the application.



An application program running on a PC can simply send output to a network drive or use a communication program that performs low level control functions, such as UART and modem control. On the Series 3000, much of this lower level control has to be performed through the application. This task is simplified by using the communication drivers and programming APIs included with the ADK.

An application design must also consider efficient data formatting. The data format used by the host usually does not lend itself to compact data storage in the terminal. The application program must balance the need for efficient data formatting on the terminal with the need for quick format conversion during the communications process.

In this chapter we address issues related to direct connect and modem communication. Spectrum One Radio communication is too complex to be considered in any detail here. Refer to the *Spectrum One Development System Application Programmer's Guide* for instructions on including Spectrum One radio communications in your applications.

Batch and Interactive Communications

Communications employing Series 3000 terminals can be done in either batch or interactive mode. In *batch mode*, a communication path is established and data is sent all at one time, i.e., in one batch. In *interactive mode*, communication proceeds in a two-way exchange of data between the host and terminal.

Batch processing has the major advantage that the terminal can run independently of the host system. The terminal is used to collect data in remote locations where serial, modem, and radio communication is either unavailable or inconvenient. Later, when the terminal is returned to a location with communication facilities, its data is uploaded to the host and its data files purged. The terminal is then ready for another data gathering session.

Communication in the interactive mode has two main advantages:

1. Less data must be stored in the terminal memory, freeing resources for other uses.

2. Data can be collected in real time, since data can be retrieved as needed from the host data base, and the host data base can be immediately updated from the terminal.

Since Series 3000 terminals are designed to be portable, interactive programs are generally radio-based. The application establishes a continuous link between the terminal and the host, allowing a free exchange of data.

Radio links are not always available or reliable, however. For this reason, radio-based applications should include the ability to run in an off-line, batch-oriented data collection mode. The collected data can then be transmitted in the batch mode when radio communications are resumed.

Hybrid applications, combining batch and interactive communication modes, are often a good approach. For example, the application can collect data and write it to a data file while a background process monitors the data file and transmits the data to the host. The `BATCHRF.EXE` utility, included in the ADK, performs this type of background communication. Refer to the *BATCHRF.EXE* section in the *Utilities* chapter of the *Series 3000 Application Programmer's Reference Guide* for a detailed description of this utility.

A Model for Communications Programming

Symbol Technologies has developed a model for communications programming. In this model, each session performs the following operations:

- 1. Initializes communication parameters**

- Sets up session protocol structure
- Loads current parameters into session parameter structure
- Changes parameters in session parameter structure

- 2. Establishes the communication link**

- Gets old protocol and save it
- Sets new protocol from session protocol structure
- Gets old parameters and save them
- Sets new parameters from session parameter structure
- Opens the comm channel(s)
- Starts Protocol



3. Initiates the data transfer

Initializes the host comm process

4. Transfers the data

Transfers the main body of data

5. Terminates the transfer

Releases the host comm process

6. Releases the session

Terminates protocol

Closes comm channel(s)

Restores old protocol

Restores old parameters

The OSI Model

The International Standards Organization (ISO) has defined a reference model for the Open Systems Interconnection (OSI). Figure 11-1 shows the seven layers of the OSI model.

Programming for the Series 3000 involves two areas in the OSI model:

- The *Application* and *Presentation* layers are controlled by the application program
- All the layers below the *Presentation* layer are controlled by the protocol

The *Application* layer handles the following tasks:

- *File management*, i.e., where does the data come from when the terminal is sending to the host, and where does the data go to when the terminal is receiving from the host
- *Data translation*, i.e., how to translate data from the format used for data transmission to the format used for data storage
- *Data sequencing*, i.e., in what order is the data stored and in what order is the data transmitted

The *Presentation* layer deals with the following tasks:

- Selection of the protocol

- Initialization of communications parameters
- Selection of the data path (i.e., serial, modem, radio)
- Transfer of the data, which involves:
 - Establishing a communications session
 - Initiating the data transfer
 - Transferring the data
 - Terminating the data transfer
 - Releasing the session

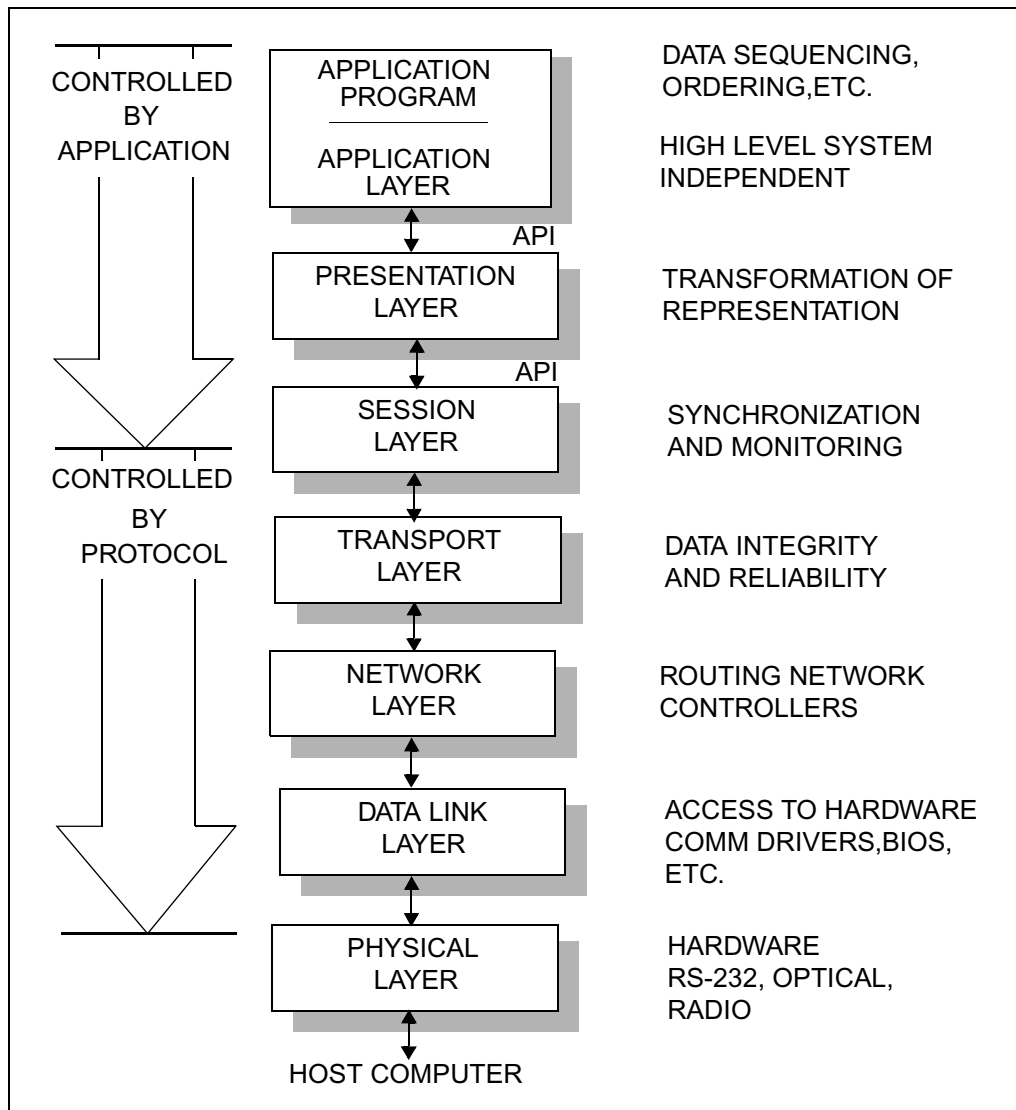


Figure 11-1. Communication Layers

Communications APIs

The Series 3000 software platform provides three libraries you can use to write a communication session. There are:

- UBASIC Record Manager (URM) Library
- DR DOS Library
- BIOS Library

Figure 11-2 illustrates the hierarchy of these APIs and how they work to control the communication session.

The UBASIC Record Manager (URM) Library

Advantages:

- Provides uniform interface to many communication devices
- Includes modem control
- Performs auto-selection, adapts to present ports
- Supports Symbol Technologies MSI communications strategies

Disadvantages:

- Adds size to the application (approximately 19K)
- Cannot be used with non-standard protocols or hardware

For more detailed information on this library, refer to the section on *Using the URM API* later in this chapter and to the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

The DR DOS Library

Advantages:

- Smaller code size
- Flexible, allowing access to nonstandard devices and protocols

Disadvantages:

- May involve many conditional expressions in the communication session code



- Lower level access is more difficult to use than the URM
- Larger source code, requiring approximately five DR DOS calls for every URM call

For more detailed information on this library, refer to the section on *Using the DR DOS API* later in this chapter and to the *DR DOS* chapter in the *Series 3000 Application Programmer's Reference Manual*.

The BIOS Library

Advantages

- Maximum low-level control of hardware
- Minimum code size
- For some non-standard protocols, this is the only method of access

Disadvantages

- Requires detailed knowledge of BIOS and hardware
- Not suitable for high-level, protocol-related communication sessions
- Less compatibility between current and future products

The BIOS library is useful for writing simple applications, such as:

- A terminal emulator (as shown by the sample program)
- Interfacing with a hardware device, such as a surveying transit or a magnetic card reader

For more detailed information on this library, refer to the section on *Using the BIOS API* later in this chapter, to the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual*, and to the *Serial I/O Services* section of the *ROM BIOS* chapter in the *Series 3000 System Software Manual*.

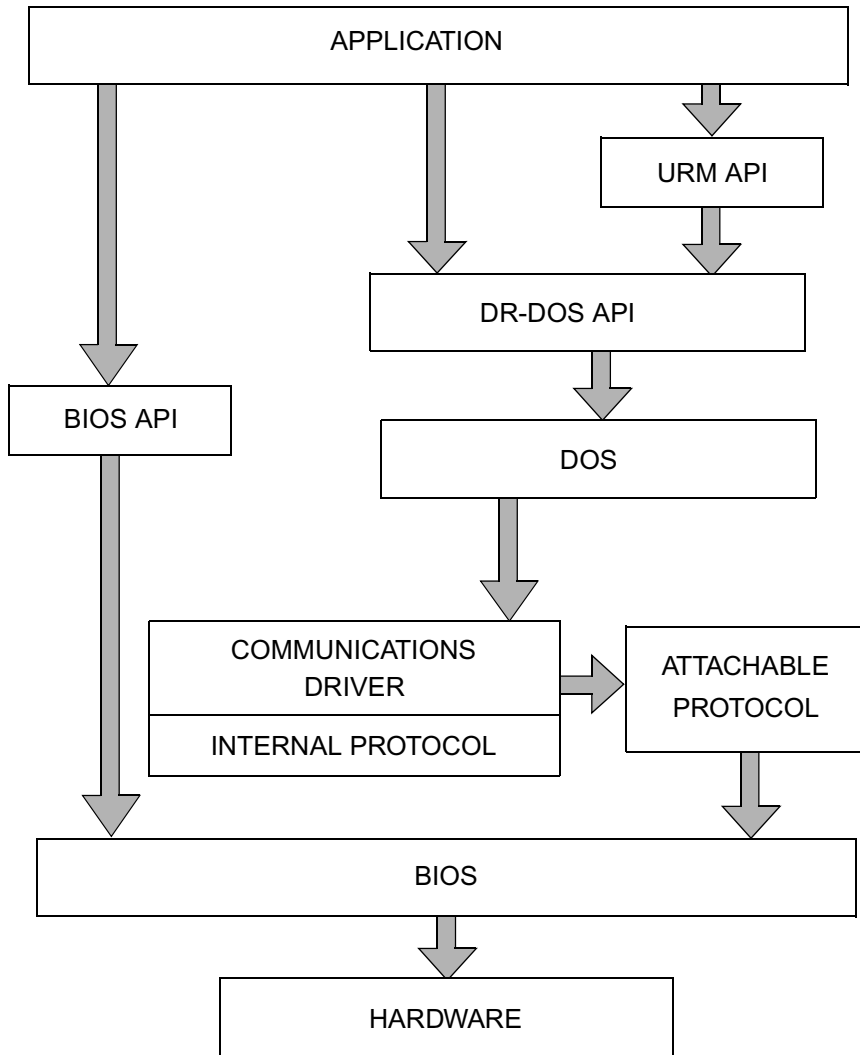


Figure 11-2. How a Comm Program Controls Hardware



Using the URM API

The URM has only five functions, but one URM function call translates into several DOS calls.

The URM simplifies communications programming by setting up extra parameters. The URM then uses these parameters to make a number of DOS calls automatically, in a “normal style” pattern.

The URM also includes a lot of conditional logic to handle many of the differences between protocols and hardware configurations. Before calling **UrmOpen**, code your application with the different parameters for each protocol. After that, you can use the same pattern of URM Read and Write calls, regardless of what protocol is used. The URM will do the right thing at the right time with the different protocols.

This allows you to use the same code for more cases. You will still have to change parameters from protocol to protocol, but the major portion of the logic for an application remains the same. You will not have to write a different loop for each kind of protocol you want to use and each different host you want to talk to. The interface between data storage and communications is only written one way.

The mechanics of how each protocol works is all part of setting up for your communications session. You will have to use many more parameters with URM, but once you have them set up properly, the body of the communication session is always the same. When you need to change a few parameters, you do not have to rewrite the send routines for each protocol.

URM can also run a nested communication session for you automatically. As part of an URM Open, if you specify the necessary parameters, it will take the session you want and embed it in another enclosing session. The enclosing session will talk to the modem, causing it to dial a phone number and get a connection. Then it will establish the communication session you asked it to establish. When this session is complete, the modem will then hang up the phone.

URM Functions

UrmOpen	Establishes a communications link to a file or device. It can also start a sub-session, for example to do the modem control.
UrmClose	Terminates a link to a file or device established by UrmOpen.
UrmReadField	Reads the next field on the communication line.
UrmWriteField	Writes a field to the open communication line
UrmPresent	Tests for presence of the URM functions

For more detailed descriptions of these functions, refer to the *Function Descriptions* section in the *UBASIC Record Manager* chapter of the *Series 3000 Application Programmer's Reference Manual*.

How to Use the URM API

A minimal URM session consists of the following:

```
Set Protocol (must use DOS for this)
Set Parameters (use DOS for this also)
UrmOpen (may have two of these)
..
..
UrmReadField
..
UrmWriteField
..
..
UrmClose (may have two of these)
```

If the application performs alternating reads and writes, a second **UrmOpen**, and matching **UrmClose**, are required. One channel is used for the reads and the other for the writes. You could do this with one Open and one Close, but two are recommended.



Alternating Protocols

Some protocols, called the *alternating* protocols, only allow transfers in one direction at a time. The application must explicitly reverse the direction of communication by doing a *Line Turn Around* when changing between writing and reading. Other protocols, the *non-alternating protocols*, allow simultaneous, bidirectional communication.

You must use two opens for the alternating protocols. The reason is that to alternate between reading and writing you must close one channel, causing the URM to do a Send End. Since the other channel is still open, the URM does not shut down the session. To resume communication, now in the reverse direction, you perform another open.

The line turn around sequence is:

```
UrmOpen (output)
UrmOpen(input)

While (not done)
  UrmWriteField
  ...
  UrmClose(output)
  UrmOpen(output) (Line Turn Around)

  UrmReadField (input)
  ...
EndWhile

UrmClose(input)
UrmClose(output)
```

Alternating protocol communication sessions operate in the Master/Slave mode. The transmitting station is the master and is allowed to transmit as long as it retains control of the line. The receiving station is the slave; all it can do is acknowledge receiving the data that has been sent. Some protocols provide a mechanism for acknowledging and requesting that the master hurry up and stop sending, but it remains up to the master to stop when it is ready.

In a Read-to-Write transition, the slave station reads until it gets an error, indicating that the master has done a Line Turn Around operation. The slave then knows it is OK for it to switch from read to write and becomes the master.

An end-of-file error at this point is OK. An end-of-file error, immediately following a trailer record is not considered an error, but rather a Read-to-Write Line Turn Around.

Including a line turn around routine in a program using a non-alternating (bidirectional) protocol will hurt anything. All the turn around does is trigger a Send End. The Send End accomplishes two things:

- It waits until all sent data is actually transmitted and acknowledged
- It triggers any protocol-related activity indicating the transfer of data in this direction is complete

If the protocol does not support a Line Turn Around, the Send End for that protocol will simply wait for all the sent data to go out.

Communicating With a Dumb Terminal

A Line Turn Around is useful when the application is communicating with a dumb terminal. The program sends a prompt which appears on the screen and then waits for an answer from the terminal keyboard.

After sending the prompt, the program does a Line Turn Around, triggering a Send End. All data is sent out before the answer is expected from the terminal operator.

Using the DOS API with the URM

There are some operations you cannot do with the URM. Applications that need to perform actions like Get Input/Output Queue Status, Flush Input/Output Queue, Get Statistics, Get Line Status, or Get Modem Status must use the DOS or BIOS APIs, usually the DOS API.

These programs will use the basic URM operations for the main communication work. They will then use DOS calls to do some activities that are incidental to the main communication session. Since these activities are not central to the communication session, the calls do not affect the activity of the URM.



Care is required, however, since there are some actions that will adversely affect the URM's ability to conduct the communication session. The URM relies on its ability to know how many lower-layer calls it has made. For example, it keeps track of the number of Opens and Closes it has performed and knows that when they are equal, the power to the device should be shut down. If you use a **DOSOpen** call after a **URMOpen** and do not match it with a **DOSClose** before the final **URMClose**, the line will stay up after the URM thinks everything is shut down.

As a general rule, do not use DOS calls for operations that the URM does automatically.

URM Parameters

Table 11-1 lists the parameters required by the URM for communications programming. The table also shows the variable names used for each of these parameters in the C1_Com session (in the file uc1.c) and the file where each variable is defined.

Table 11-1. URM Communications Parameters

Parameter	URMCOMM Variable	Defined In ...
UrmOpen and UrmClose		
File Control Blocks (Input and Output)	C1_ComOutFcb C1_ComInFcb	uc1.h
Device Name Translation Table	C1_ComTrnTbl	uc1.h
Port Name and Length	PORT Macro	urmcomm.h
Data Buffer and Length	BUFR Macro	urmcomm.h
I/O File Mode	OpenForInput OpenForPrint	urm.h fmgr.gd
Modem Control Structure	NULL in C1_com (M_ModemCtl in M_Com)	um.h
Separator Character Structures (Input and Output)	C1_ComInSep C1_ComOutSep	uc1.h
UrmReadField and UrmWriteField also require a URM Pointer Structure (C1_Urm, defined in uc1.h)		
Out Separator	C1_ComInSep	uc1.h

Table 11-1. URM Communications Parameters (Continued)

Parameter	URMCOMM Variable	Defined In ...
In Separator	C1_ComInSep	uc1.h
Record Number	C1_ComRecNum	uc1.h
Record Length	C1_ComRecLen	uc1.h
Record Type	C1_ComRecTyp	uc1.h
Record Flags	C1_ComRecFlg	uc1.h
Transmit Conversion Function	NULL	
Receive Conversion Function	NULL	

Using the Separator Character Structure

There are two basic ways in which you can to use the separator character structure referred to in Table 11-1:

1. Define one structure for each protocol.
2. Define one structure containing the parameters common to all the protocols and then modify it at run-time (as done by **ComPutData** in the URMCOMM.C and URMMODEM.C sample programs in the C:\3000\SAMPLE\COMM directory of the ADK).

For more information on the Separator Character Structure, refer to the *Structure Definitions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

Modem Communications

The URM manages a many modem control operations automatically. It goes through the following steps in determining whether to initiate modem communications in the first place:

1. When a communication session calls **UrmOpen**, it passes a pointer to a modem control structure. If this pointer is NULL, the URM does not check for a modem. Instead, it waits until either DSR is raised or the timeout limit is reached.



2. If this pointer is not NULL, the URM checks the specified port for DSR. If DSR is raised, it assumes a direct connection and does not perform any modem operations.
3. If there is no DSR on the port, the URM sends an AT command. If it gets a response, the URM knows it is talking to a modem.
 - a. It then sends an AT command to determine if it is a Symbol internal modem. If an internal modem is detected, the URM initializes it, based on the information in the Modem Control Structure and begins the dialing or answering sequence. The connection is complete when DSR is obtained. For a description of the Modem Control Structure, see the *Structure Definitions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.
 - b. If the response to the special Symbol modem AT command is incorrect, the URM assumes that there is a properly initialized external modem, and it begins the dialing or answering sequence. The connection is complete when DSR is obtained.
4. If there is no reply to the first AT command, the URM assumes a direct connection and waits up to the timeout limit for DSR.

Once a connection has been established, the application operates like any communications program. The only difference is in the setup of the parameters before you do the open.

A URM modem program differs from a URM direct-connect program in the following ways:

1. It builds a modem control structure and passes it to **UrmOpen**.
2. The duplex parameter depends on the modem connection.
3. The port may be different (COM2 may be used instead of COM1).

Modem Initialization

If you are using an internal modem, the URM performs the modem initialization for you. If you are using an external modem, you must initialize it. Once initialized, the URM uses internal and external modems in the same way.

It is your responsibility to ensure that the modem type you specify is supported by the modem hardware and that the modems on both ends of the communication line are compatible.

Whenever possible, modem configuration parameters should be stored in the modem permanently. Refer to the *Descriptions of AT Commands* section in the *Internal Modem Command Set* chapter of the *Series 3000 Application Programmer's Reference Manual*.

The following configuration parameters are recommended:

Handshake Signals

DSR	Do not force on. DSR tracks connection with remote modem.
CTS	Used to indicate modem is ready to receive commands.
CD	Do not force on. CD tracks connection with remote modem.
DTR	No answer while DTR off. Can answer once DTR is raised if so configured (see S0 Register). Hang up line when DTR is dropped.

Full-Duplex/Half Duplex

Set to half-duplex for V23

Set to full-duplex for V21, V22, 212, and 103.

Bell Mode

Set for Bell 212 or Bell 103.

Reset (CCITT Mode) for V21, V22, and V23.

S0

This modem register is set to zero when used in "smart mode" (never auto-answers).



Set to a non-zero value when used in “dumb mode” to auto-answer n rings following DTR.

Modem Sample Program

The program URMMODEM.C, included in the C:\30000\SAMPLE\COMM directory of the ADK, illustrates how to program for modem communications using the URM API. It shows how a program can include modem parameters and use them whenever desired in the application.

URMMODEM.C can connect to an external modem (via the DB25 port) or to the internal modem (which outputs via the RJ11 port). It can also connect to an acoustic coupler or to a limited RS-232 cable (via the RJ41 port). It also allows the user to select one of three modem technologies.

The following walk-through compares the URMMODEM.C communication session with the UC2.C communication session. Both of these sessions use the TWOWAY protocol. UC2.C is also on the C:\3000\SAMPLE\COMM directory of the ADK.

Port Names

The first major difference between these two programs is the port names. When you use the URM you have to deal with two different kinds of port names:

1. A port name for communication initialization, which is done directly with DOS calls. Here the application will have to use a DOS device name. UC2.C assumed that it would only be dealing with the DB25 port, so it used the constant device name COM1.
2. A port name for URM. Here the port names are subject to translation (through the device name translation table). For UC2.C, the eventual result (after the translation took place) was a device name of COM1.

UM.C (on the C:\3000\SAMPLE\COMM directory) uses two variables, M_UrmPort and M_DosPort, which it uses for URM and DOS, respectively. Based on the menu selection, M_UrmPort is either “DB25”, “RJ11” or “RJ41”. These port names are then translated by the URM using the device name translation table (in UM.H).

The translation table maps "DB25" to "COM1", "RJ11" to "COM2", and "RJ41" to "COM2". It then redirects RJ11/COM2 to the modem (TOMDM) and RJ41/COM2 to the RJ41 port (TOSPKR).

M_DosPort is either "COM1" or "COM2", standard DOS device names.

Modem Parameters

UM.C next gets the current parameters and sets them up for its communication session. Unlike UC2.C, it sets both the serial parameters and the modem parameters. Otherwise, the UM.C initialization is identical to the UC2.C initialization.

Modem Control Structure

UM.C has a modem control structure (defined in URM.GT), which UC2.C doesn't have. Some of the information in this structure is left as initialized, and some of the information is changed (the phone number and modem type). See the section on *Modem Control Structure* in the chapter on the *UBASIC Record Manager* in the *Series 3000 Application Programmer's Reference Manual* for more information on the modem control structure (**MODEMCTL_S**).

The URM and External Modems

UM.C passes **UrmOpen** the address of the modem control structure instead of NULL. If COM1 is selected, the URM will check for the presence of an external modem. UC2.C passed a NULL pointer to UrmOpen, which caused the URM to omit all modem control operations.

It is possible to use COM1 for both direct-connect communication and modem communication. If no external modem is connected and powered-on at run time, the URM will then switch to direct-connect communication.

In order for the URM to detect the external modem, it must be set up as described earlier in this chapter under the heading *Modem Initialization*.

Call Progress Monitoring

The modem program can use one other feature: Call Progress Monitoring.



If the **call_prog** pointer in the modem control structure is not NULL, the URM will call the routine pointed to by the **call_prog** pointer periodically and pass it an indication of what is happening in the progress of the call (ModemConnect, ModemLocalRing, etc.). These call progress constants are defined in URM.GD.

This is a very desirable feature. Without it, the user has no way of knowing what is going on while the modem is dialing, the phone is ringing, etc.—a process which can take quite a while. It can also give the user an indication of what went wrong when a communications failure occurs.

URM Techniques

The following paragraphs describe a number of programming techniques you will need to use when using the URM for communications.

Pack All Structures

All structures used by Symbol-supplied libraries are packed. It is recommended that you always pack your structures. If you don't, the structures you create are not interpreted correctly when passed to Symbol library routines.

You can pack in one of two ways (Microsoft C):

1. Use the /Zp switch when compiling. This packs structures for the entire module being compiled.
2. Use the pragma pack (#pragma pack(1)). This packs only from the point at which the pragma appears in the code.

Initialize the Device Name Translation Table

Your program will need to define a Device Name Translation Table. This table translates the UBASIC device names to names that can be recognized by your program. The following are examples of some of the lines you may include:

```
static char C2_ComSrc[] = "AUTO\0/DB25\0/RJ11\0/RJ42\0/";
static char C2_ComDst[] = "AUTO\0/COM1\0/COM2\0/COM2\0/";
static byte C2_ComIdx[] = {OMDEV, COMDEV, COMDEV | TOMDM
    COMDEV | TOSPKR}
static XlatPtrT C2_ComTrnTbl = {C2_ComSrc, C2_ComDst, C2_ComIdx}
```

This table translates the URM port names to DOS device names and then redirects the DOS device names to specific devices.

The following table is written for a Series 3300 terminal which has an RJ41 port and an internal modem. For a 3800 terminal, you might use a URM Port Name such as "RADIO".

Device Name Translation Table				
URM Port Name	AUTO	DB25	RJ11	RJ41
DOS Device Name	AUTO See Note 1	COM1	COM2	COM2
Redirection			TOMDM See Note 2	TOSPKR See Note 3
Notes: 1. Selects either COM1 or COM2 at runtime, depending on hardware availability and state. 2. Forces to internal modem operation on COM2. 3. Forces to acoustic external attachment (RJ41) on COM2.				

Define a Communications Buffer

You need the equivalent of the following statements in your program:

```
#define combuflen 128/* comm buffer length */  
byte combuf[combuflen];/* comm buffer */
```

This sets up a buffer equivalent to a UBASIC communication buffer. It is used by the URM functions to assemble data for sending and/or receiving. It must be supplied by the application and must be as large as the largest record you expect to send or receive, plus any prefix or postfix characters.

If your application simultaneously sends and receives data, you must define a communication buffer for each channel.



Declare URM Record Variables

Four variables must be defined for each simultaneous communication direction in your application. For example:

```
int  recnum; /*record number */
word reclen; /*record length */
byte rectyp; /* record type */
byte recflag; /* record flag */
```

These four variables are record variables and are pointed to by four pointers in the URM Pointer Structure (UrmPtrT).

There are several schemes for declaring more than one set of record variables. For example, you might declare an array of these four variables if the communication directions might be used in a loop. However, if the communication directions are independent, you might simply declare different sets using specific names. For example:

```
int  input_recnum;
word input_reclen;
byte input_rectyp;
byte input_recflg;

int  output_recnum;
word output_reclen;
byte output_rectyp;
byte output_recflg;
```

The procedure for using the record variables is to set a variable, call a URM function, and then check the variable.

Selecting a Protocol

There are two types of protocols: *stream* and *block*. The major difference between these types of protocol is in their use of record separator characters.

Block protocols do not use record separators. Rather, they have their own ways of separating blocks of data.

Stream protocols have built-in mechanism for separating records. Data is dealt with as a single stream. If records are distinguished, it is by the use of selected characters recognized by the application.

The URM provides a way to emulate block protocols by sending separators around records and then removing them as it receives records on the other end. The separator emulation is transparent to the application. Because the URM functions make stream and block protocols homogeneous, you can write applications that use different protocols in the same manner.

In order to emulate the four types of data blocks (header, trailer, data, header/trailer), two types of prefixes and two types of postfixes are supported by URM. The separators that are supported are the *header prefix*, the *non-header prefix*, the *trailer postfix*, and the *non-trailer postfix*. An application can emulate a block protocol by assigning various values to these separators.

The following three protocols are available when using the URM.

SIMPLEX

The simplex protocol is a stream protocol and is usually used for acoustic and one-way communications. It uses 7 data bits, with parity, and no flow control. It uses mark and space times when sending data to improve and stabilize acoustic communication.

The protocol name “MSISTD” is often used with the appropriate separator characters to achieve the simplex protocol.

The following separator characters are often used with the simplex protocol:

```
comm_out_separators.FieldSep = 0;  
comm_out_separators.SessionPre = '\\002
```



```
comm_out_separators.SessionPost = '\003';
comm_out_separators.HeaderPre = 0;
comm_out_separators.NonHeaderPre = 0;
comm_out_separators.TrailerPost = '\x02B'
comm_out_separators.NonTrailerPost = '\x02B'
comm_in_separators.FieldSep = 0;
comm_in_separators.SessionPre = '\x002';
comm_in_separators.SessionPost = '\x003';
comm_in_separators.HeaderPre = 0;
comm_in_separators.NonHeaderPre = 0;
comm_in_separators.TrailerPost = '\x02B
comm_in_separators.NonTrailerPost = '\x02B
```

The STX (Hex 002) and ETX (Hex 003) characters are used for session pre- and postfix characters, respectively. The “+” character (Hex 2B) is used to separate records. All others are initialized to zero.

Xon/Xoff

Xon/Xoff is a stream protocol. It uses 7 or 8 data bits (ASCII oriented) with or without parity. It uses software flow control, i.e. start and stop indicators. These control indicators are defined by *ctlstartchar* (control start character) and *ctlstopchar* (control stop character) as Hex 11 (Control Q) and Hex 13 (Control S).

The protocol name “MSISTD” is used with appropriate separator characters and flow control to achieve the Xon/Xoff protocol.

The following separator characters are often used with the Xon/Xoff protocol:

```
comm_out_separators.FieldSep = 0;
comm_out_separators.SessionPre = 0;
comm_out_separators.SessionPost = '\x01A
comm_out_separators.HeaderPre = 0;
comm_out_separators.NonHeaderPre = 0;
comm_out_separators.TrailerPost = '\r';
comm_out_separators.NonTrailerPost = '\r';
comm_in_separators.FieldSep = 0;
comm_in_separators.SessionPre = 0;
comm_in_separators.SessionPost = '
comm_in_separators.HeaderPre = 0;
```

```
comm_in_separators.NonHeaderPre = 0;  
comm_in_separators.TrailerPost = '\r';  
comm_in_separators.NonTrailerPost = '\r';
```

These characters define typical separators found in an ASCII text file: carriage returns and Control-Z (End of File). All others are initialized to zero.

Two Way

Two-way protocol is a block protocol. It uses 8 data bits, no parity, and hardware or no flow control. Because Two-way protocol is a block protocol, no separators are needed. Therefore, as shown in the lines below, they are all initialized to zeros using `memset ( )`.

```
memset(&comm_out_separators,0,sizeof(comm_out_separators));  
memset(&comm_in_separators,0,sizeof(comm_in_separators));
```

For more information concerning separators and block protocol emulation, see the section on *Selecting a Protocol* earlier in this chapter.

Using rec_flg

The **rec_flg** variable is used extensively by the URM to hold information about records. This information is used when records are read and written. For a list and explanation of each flag available for **rec_flg**, refer to the Record Flags parameter description in the *URM POINTER Structure (UrmPtrT)* section of the chapter on *UBASIC Record Manager* in the *Series 3000 Application Programmer's Manual*.

The End-of-Record Flag

The following statement sets the end of record flag (ENDOFREC) in **rec_flg**:

```
rec_flg = ENDOFREC;
```

The ENDOFREC flag, when writing fields, indicates that the current field is the last one that will be written to the record. If your application places multiple fields in a single record, enable this flag just before the last field is written to the record.



When reading fields, the ENDOFREC flag has an analogous meaning. It indicates that the current field is the last field that will be read from the record.

This flag coupled with a zero length field in the **UrmReadField** function allows an application to read and write whole records at a time. See the *Function Descriptions* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual* for a description of the **UrmReadField** function.

Headers and Trailers

The Record Flags field in the URM Pointer Structure contains a header flag (HEADERPRE) and a trailer flag (TRAILERPOST). The URM uses these flags to indicate the type of record transmitted or received.

Generally, a header block is sent at the beginning of a transmission to indicate the beginning of a data sequence or to give information about the data to follow. This is usually followed by non-header blocks in the body of the data. A trailer block (which is not also a header block) is usually sent as the last block of a data sequence.

The interpretation of what it means for a block to be a header or trailer is supplied by the application layer.

TMS/TME/DMX, for example, (which are described later in this manual, in the section *Remote PC Communication Software*) use header blocks to send commands and trailer blocks to indicate that all of the data has been successfully transferred.

Since there is a header and a trailer flag, four combinations are possible:

1. Header, no trailer (header block)
2. No header, trailer (trailer block)
3. No header, no trailer (data block)
4. Header, trailer (the only block sent).

Header Block

A header block is sent before other data blocks. Typically, this means that:

- it contains special information

or

- it contains command information instead of data information (e.g., the destination file name)

If the header flag PEADERPRE) is set, this block is transmitted as a header. Once this flag is set, every block sent will be of that type, until the flag is changed. Usually, the header flag is only set for one record.

Trailer Block

If the trailer flag (TRAILERPOST) is set, this block is transmitted as a trailer. This means it is the last block to be transmitted or the last block in a group of related blocks. Application programs usually require that a last block be received before the session is terminated. If the last block received before the end of a session is not marked as a trailer block, the application may assume that the session was prematurely terminated.

Typically, if the last block received was a trailer block, termination of the session (with an end-of-file error) is acceptable. After a trailer block is received, the session is terminated, or line-turn-around is performed, or another header is expected.

Header/Trailer Block

A combination header/trailer block is typically used in a command-oriented system where commands are placed in a header-type block—but no more data is to be sent.

If the data in a file is being requested, for example, the filename would first be sent to the host in a header/trailer command. The host would know there was no more data to follow and would reply by sending the data in that file.

If the host had received a header/no_trailer block, it would have assumed that more data was to follow.



Using the DR DOS API

Even when you are using the URM for data communications, you will need to use the DR DOS API to perform a number of operations. You can also do all of your communications using DR DOS Library calls. Doing so, however, you lose a lot of the convenience of using the URM, i.e.:

- You will have to write more code
- You will have to use the IOCTL commands
- You will have to maintain the proper sequence of operations
- You will have to program the internal modem directly
- You will have to program differently for each protocol

Nonetheless, there are times when using only DOS commands is the better way to go.

List of DR DOS Communications Functions

Table 11-2 lists functions in the DR DOS Library that can be used for communications programming. Refer to the DR DOS chapter in the *Series 3000 Application Programmer's Reference Manual* for detailed descriptions of these functions.

Table 11-2. DR DOS Data Communications Functions

Function	Description
DosClose	Closes a device or file
DosOpen	Opens a device or file
DosRead	Reads from a device or file
DosReadLine	Reads maxlen characters from a file
DosWrite	Writes to a device or file
DosWriteLine	Writes the characters in a buffer to a file
DosIoCtrlGetInfo	Gets information about a device or file
DosIoCtrlSetInfo	Sets device or driver information
DosIoCtrlRdData	Reads control strings from a device driver
DosIoCtrlWrData	Writes control strings to a device driver

Using the IOCTL Commands

Most of the power of the DR DOS Library for communications programming lies in using the Input/Output Control (IOCTL) commands.

The I/O Control Structure

The I/O Control structure (**Ioctl_S**) is used to pass control information between the application program and device drivers. It is the central aspect of IOCTL programming. The structure has three members:

```
char funcode;  
unsigned char errcode;  
union data;
```

The function code (**funcode**) describes the type of operation the Device Driver is to perform. The error code (**errcode**) contains the error codes returned from this operation. The union contains function-code dependent information. For function code 00 (**ComIoctlComParamCmd**), the union contains the structure **comparameters**. For a definition of the **union** element in the I/O Control structure, refer to *The I/O Control Structure Defined* subsection of the *Passing Structures to Functions* section of the chapter on DR DOS in the *Series 3000 Application Programmer's Reference Manual*.

A typical I/O sequence works like this:

1. Load the proper function code into **funcode**. For example, in the URMCOMM.C program included in the C:\3000\SAMPLE\COMM directory of the ADK, the function **ComGetParms** uses the following code:

```
iob->funcode = ComIoctlComParamCmd;
```

where **iob** is a pointer to an IOCTL structure.

2. Make a DOS IOCTL call using this Ioctl structure. **ComGetParms** makes this call:

```
DosIoCtrlRdData(... (char *)iob, ...);
```

This returns the current comm parameters in the structure **comparameters** and any errors in **errcode**.



3. Change some of these parameters. The function **C1_ComPrmInitialize** in the sample URM program makes these changes:

```
C1_ComParms.Data.Comparameters.parity=DATABITS8;  
C1_ComParms.Data.Comparameters.stopbits=STOPBITS1;  
...
```

4. Make an IOCTL call to set these parameters. The function **ComSetParms**, for example, uses this code:

```
iob->funcode = ComIoctlComParamCmd;  
DosIoCtrlWrData (... (char *)iob, ...);
```

This uses the data in the structure `comparameters` to set the serial port parameters.

Sequence of IOCTL Calls

The following charts show the standard sequence of DOS calls made by the URM:

ComPrmInitialize		
DOS Function(s):	IOCTL Command	Description
DosOpen	N/A	Open handle to device
DosIoCtrlGetInfo DosIoCtrlSetInfo	N/A	Set mode

ComSesEstablish		
DOS Function(s):	IOCTL Command	Description
DosIoCtrlRdData	ComIoctlSelectProtCmd	Get current protocol
DosIoCtrlWrData	ComIoctlSelectProtCmd	Set desired protocol
DosIoCtrlRdData	ComIoctlComParamCmd	Get current communication parameters
DosIoCtrlWrData	ComIoctlComParamCmd	Set desired communication parameters
DosIoCtrlWrData	ComIoctlOpenLine	Open physical link
DosIoCtrlWrData	ComIoctlStartProtocol	Establish logical connection

ComTrnInitiate ComTrnProcess ComTrnTerminate		
DOS Function(s):	IOCTL Command	Description
DosIoCtrlWrData	ComIoctlSetHdrTrlr	Issue Header/Trailer command
DosWrite DosIoCtrlWrData	ComIoctlGetWrStatus	Transmit data Check for errors
DosIoCtrlWrData	ComIoctlSendEnd	Do line turnaround Check for errors
DosRead DosIoCtrlRdData	ComIoctlGetRdStatus	Read data Check for errors and check incoming header/trailer

ComSesRelease		
DOS Function(s)	IOCTL Command	Description
DosIoCtrlWrData	ComIoctlCloseProtocol	Release logical connection
DosIoCtrlWrData	ComIoctlCloseLine	Release physical link
DosIoCtrlWrData	ComIoctlSelectProtCmd	Restore saved protocol
DosIoCtrlWrData	ComIoctlComParamCmd	Restore saved parameters
DosClose		Release handle

Programming the Internal Modem

The Series 3000 may have an internal modem. The commands for these modems are compatible with the Hayes® Smartmodem 2400™ AT commands. Refer to the chapter on the *Internal Modem Control Set* in the *Series 3000 Application Programmer's Reference Manual* for a complete description of these commands.



Commands should be sent to the modem via COM2, using **DosWrite**. Replies from the modem should be read using **DosRead**. Eight data bits with no parity is recommended.

Using the BIOS API

The BIOS API provides the lowest level of control over the communication session. This is the level of operation of the device drivers. In general, an application program does not need to use the BIOS calls.

You can use the BIOS API if you are writing an application that:

- is very simple in its use of I/O
- does not need sophisticated protocol-related operations, such as forming data packets

or an application that:

- uses a non-standard protocol
- requires difficult protocol interactions
- requires assists not handled by a standard protocol

The BIOS API provides very little help to the programmer. If you tell it to send out a string of bytes, it will send them out. If you tell it to receive some bytes, it will receive whatever bytes it is handed. It makes no attempt to validate or package the data.

The BIOSCOM.C sample program in the C:\3000\SAMPLE\COMM directory of the ADK is a dumb terminal emulator, and it provides a good example of an appropriate use of the BIOS API. Every time you press a key on the keyboard, it sends that character to the communication line. Every time a character comes in on the communication line, it is written to the display.

Another good application for the BIOS API is to provide an interface for device driver to a unique hardware device.

Note: You should not mix BIOS calls in the same session with URM or DOS calls. Mixing these calls is likely to produce unpredictable and undesirable results.

Sequence of BIOS Calls

The table below shows the standard sequence of communication BIOS calls used by the example program.

Bios Call	Ver 3.01 Bios Call	Action
Initialize Parameters		
BiosSerialInitQueues	BiosPurgeQueue	Initialize allocated queues
BiosSerialGetParams	BiosGetSerialSetup	Save current serial parameters
BiosSerialSetParams	BiosInitSerialPort or BiosSerialSetup or BiosExtSerialSetup	Set serial parameters
Open Physical Link		
BiosSerialOpen	BiosOpenPort	Open serial channel
Transmit Data		
BiosCheckKey* BiosGetChar*	Compatible	Get keystroke
BiosSerialSendBlock	BiosSendBlock	Send key to communication line
Receive Data		
BiosSerialInQStat	BiosQueueStatus	Check input queue
BiosSerialRecvrBlock	BiosRecvBlock	Get char from queue
BiosPutCharMove*	Compatible	Send char to display
Close Physical Link		
BiosSerialSendDone	BiosTransDone	Wait for all data to be sent to communication line
BiosSerialClose	BiosClosePort	Close communication line
Restore Parameters		
BiosSerialSetParams	BiosInitSerialPort or BiosSerialSetup or BiosExtSerialSetup	Restore saved serial parameters

**Not communication functions*



Sample Communication Programs

There are four sample programs included with the ADK and located in C:\3000\SAMPLE\COMM directory. These programs are:

Program	Demonstrates ...
URMCOMM.C	Direct communications using the URM
URMMODEM.C	Modem communications using the URM
DOSCOMM.C	Simple communications using the DR DOS
BIOSCOM.C	A dumb terminal emulator

URMCOMM and DOSCOMM

URMCOMM and DOSCOMM both perform a brief, two-layer communication session using a PC as the host.

To run these programs, connect the terminal to the PC COM1 port using a null modem, as shown in Figure 11-3. You can use another serial port if necessary.

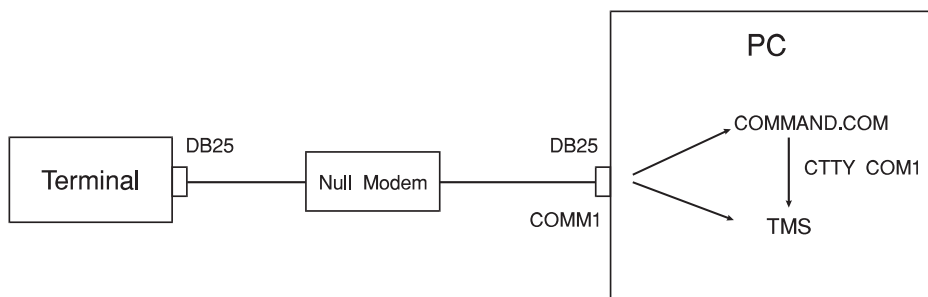


Figure 11-3. Terminal to PC Connections

Enter the DOS command CTTY COM1 on the PC to make COM1 the PC console port. With URMCOMM or DOSCOMM running on the terminal, all the PC DOS prompts will be displayed in the terminal and all the commands entered on the terminal keyboard will be executed on the PC.

Walking Through the Code

The basic program code is contained in three files:

- URMCOMM.C (or DOSCOMM.C). This contains some general-purpose communication routines and `main()`.
- UC1.C (or DC1.C), which is the first communication session.
- UC2.C (or DC2.C), which is the second communication session. It is called by the communication session in C1.

C1_Comm

`main()` calls `C1_Comm`, and then follows the standard sequence of operations prescribed by the communications model:

```
C1_ComPrmInitialize
C1_ComSesEstablish
C1_ComTrnInitiate
C1_ComTrnProcess
C1_ComTrnTerminate
C1_ComSesRelease
```

Each of these steps must include error processing. If something goes wrong after the session is established, the session must be released before exiting.

C1_ComPrmInitialize

`C1_ComPrmInitialize` performs the following operations:

1. Opens a handle to the communication device driver.
2. Sets the protocol structure to be used for this communication session. The protocol for this session is `MSISTD`.
3. Loads the current communication parameters into the parameter structure to be used for this communication session.
4. Changes some of these communication parameters.



Note: This sequence follows the general rule: read the current parameters, then change only the ones you are interested in. This prevents you from inadvertently changing values you are not concerned with.

C1_ComSesEstablish

C1_ComSesEstablish performs the following:

1. Gets the old protocol and saves it.
2. Sets the new protocol, using the protocol structure set up by C1_ComPrmInitialize.
3. Gets the old parameters and saves it.
4. Sets the new parameters, using the parameter structure set up by C1_ComPrmInitialize.
5. Starts the communication session.

C1_ComTrnInitiate

C1_ComTrnInitiate does all the pre-processing required before the main body of the data transfer can begin.

C1_ComTrnInitiate performs the following operations:

1. Sends out a blank line, followed by a carriage return.
2. Gets the PC response and throws it away.
3. These two steps clear out any garbage that may be in the PC queue when the session started.
4. Sends a carriage return (CR) to the PC in order to get back a prompt, and expects back a new line followed by the DOS prompt (with drive indicator).
5. Sends "cd \demo CR", and expects back "CR CR LF D>".

This completes the log on to the demo program on the PC.

C1_ComTrnProcess

C1_ComTrnProcess performs the following:

1. Sends "COMPGM <CR>". COMPGM.BAT on the PC changes directories and calls TMS, which is the communications program on the PC.
2. Calls C2_Com. This is a nested communication session inside the C1 session. C2_Com communicates with TMS, then exits.
3. Waits for TMS to exit and for the return of the DOS prompt.

C1_ComTrnTerminate

C1_ConTrnTerminate performs the following:

1. Sends a null string and a carriage return and gets the PC prompt.
2. Sends "CTTY CON", which returns control to the PC console.

C1_ComSesRelease

C1_ComSesRelease performs the following:

1. Closes the session.
2. Restores the protocol and parameters saved by C1_ComSesEstablish.
3. Releases the handle obtained by C1_ComPrmInitialize.

C2_COM

C2_ComPrmInitialize

C2_ComPrmInitialize performs the following:

1. Opens a port to get its own handle.
2. Sets the protocol to TWOWAY.
3. Gets the current parameters and changes the ones it is interested in.
C2_Com uses a block protocol and two-way emulation.

C2_ComSesEstablish is the same as C1_ComSesEstablish, except for different variable names.



C2_ComTrnInitiate is different because here we are talking to TMS instead of COMMAND.COM. The SEPARATE command tells TMS that we are sending a file and specifies the filename on the PC.

C2_ComTrnProcess sends a line of text 10 times. The last line is sent as a trailer to satisfy TMS. It then does a send request command, a line turn-around, gets and prints data until an error (no more data), and checks that the last record was a trailer which was followed by a proper error code.

There is no terminate activity.

C2_ComSesRelease closes the session. It then restores the protocol and parameters back to their original states (from the C1 session).

Managing Terminal Resources

Power Management

Communications sessions consume more power than any other terminal operations. This is only a problem if the terminal is operating on battery power during a communications session. If communications occur while the terminal is in a cradle or attached to a power adapter, there is no problem.

Conserving Battery Power

If a communications session is taking place while the terminal is on battery power, you should do the following:

1. Attempt to verify that the terminal battery has sufficient charge to complete a session before initiating the session. Use the **BiosCheckBattery** service to determine whether the charge is “good” or “low.” Do not initiate a session if the charge is low. This service is described in detail in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual*.

Note: This call will draw on some battery power. Be conservative in its use, i.e., use it before performing data communications.

2. Keep the communications session short. Leaving the communications line open drains the battery, so do not stay on line longer than is necessary. If possible, do not stay on line while the host processes your data. Open your communication port only when you are ready to transmit.

Determining Terminal Power Source

To determine the present power source, use the **BiosGetPowerSource** service. The value returned by this service will indicate whether the power source is battery, cradle or charger. See the *BIOS Library Functions* chapter in the *Series 3000 Application Programmer's Reference Manual* for a description of this service.



Report Battery Cell Status Service

To check the batteries, use the BIOS function **BiosCheckBattery**. This function will tell you if the main battery cells are “good” or “low”. Do not start a communication session when the batteries are “low”.

Memory Management

Your application should monitor memory consumption to assure both that there is sufficient memory to store the data it collects and to run the communications session to upload that data to the host.

The application should also be able to recover from a “memory full” condition, especially if it is to receive data from the host. Two solutions to a memory shortage are to:

- work with the incomplete data
- request the host to resend a smaller amount of data

The best solution will vary with the application.

Once the data has been successfully transferred to the host, it can be deleted from the terminal memory.

Unattended Communications

There are two types of unattended communications session:

- *Exclusive*
A single terminal has exclusive use of the communication channel
- *Multi-drop*
Several terminals are trying to use the same communication channel at the same time. This is common in installations using multi-slot cradles.

Exclusive Unattended Session

A good example of an exclusive unattended communication is a Field Service Reporting (FSR) application, where each field service technician plugs his terminal into the phone line each night. The FSR application can take two forms:

- The host calls each field service number during the night.
- Each terminal calls the host at a predetermined time.

In either case, the terminal accepts and/or transmits data and then provides a status message to the technician.

To perform an exclusive unattended communication, the application must:

1. Call **BiosSetWakeReason** with the *pwr_events* parameter set to an appropriate event. See the description of this function in the *BIOS Library Functions* chapter of the *Series 3000 Application Programmer's Reference Manual* for syntax and parameters. Use either Port 0 Ring, Port 1 Ring, or Real Time Clock (RTC) Alarm. To use Real Time Clock Alarm, the clock alarm must first be set and enabled.
2. Call **BiosPowerOff**.

When the Wake Reason event occurs, the terminal powers up and application code begins with the next statement (in this case, it begins the communication session).



Multi-Drop Unattended Session

Multi-drop unattended communication occurs when several terminals are each attempting to establish a link with the host. Typically, the terminals are installed in cradles.

Each terminal continually attempts to establish a connection with the host. When successful, it performs its communication task. When finished, it relinquishes the communication link. Another terminal will then be able to establish and complete a communication session. This process is repeated until all the terminals have succeeded.

Communicating Through a Cradle

This section describes how to write a simple communication session where the host communicates with one or more terminals through a cradle (a charging and communications module, or CCM).

The description of the session is given from the terminal side. For information on writing a communication program for the host, refer to your host system's documentation. The sample programs that are provided in the C:\3000\SAMPLE\CRADLE directory of the ADK include script and batch files used by a host PC running PROCOMM PLUS.

There are three types of cradles you may need to accommodate in your application:

- single-slot
- multi-slot
- single-slot with internal modem

Communicating with a single-slot cradle is basically the same as communicating with a single terminal. No special problems are introduced by the cradle.

In the following discussion, we will assume that you are communicating through a multi-slot cradle.

The Different Types of Sessions

There are two basic types of cradle communication sessions. These are:

- **Terminal Controlled**

In a terminal controlled communication session, the terminals in the cradle compete for the data bus. The first terminal that gets the bus performs its session, then releases the bus. This process is repeated for each terminal in the cradle.

- **Host Controlled**

In a host controlled communication session, the terminals share the bus. The host selects a terminal, performs its session, then releases the terminal. This process is repeated for each terminal in the cradle.

Note that the selection process need not be complicated. For example, the host computer can select a terminal by sending a polling message that specifies a terminal ID.

There are a few things the terminal must do in both types of sessions:

- check to make sure it is in the cradle

If the terminal is *not* in the cradle, display a message signaling to put the terminal in the cradle. This ensures that the operator will put the terminal in a cradle for the communication session instead of trying to use a modem or other device.

- set the communications parameters
- get and release control of the data bus

Cradle Communications API

Before you can begin programming, you must select an application program interface (API). Symbol terminals support the three communication APIs that are discussed earlier in this chapter. They are: URM, DOS I/O, and BIOS. See also the following chapters in the Series 3000 Application Programmer's Reference Manual descriptions of these APIs: *BIOS Library Functions*, *DR DOS*, and *UBASIC Record Manager*.



Of these APIs, DOS I/O commands are the easiest to use. If you choose to use BIOS calls, but then you may need to write several assembly language service routines. Even when you use DOS I/O commands, you must still use BIOS services to do things like get and release control of the data bus.

Checking that the Terminal is in the Cradle

Use either the **BiosGetPowerSource** or the **BiosGetCradleType** service to determine whether or not the terminal is in the cradle.

If the **BiosGetPowerSource** service returns 1, the terminal is in the cradle (CCM). If the **BiosGetCradleType** service returns a value between 3 and 7, the terminal is in a cradle. These services are described in detail in the *BIOS Library Functions* chapter of the *Series 3000 System Software Manual*.

Set Communications Parameters

The main difference between communicating with a cradle and other types of communications sessions is the duplex mode parameter setting. Symbol provides a special *multi-access* mode for cradle communications.

Multi-access mode supports single-point to multiple-point protocols. This mode does not have the same type of control parameters as full- and half-duplex modes. Instead it works with the cradle itself and the BIOS **Request/Release Data Bus** service, which is described in the *Serial I/O Services* section of the ROM BIOS chapter in the *Series 3000 System Software Manual*.

Note: The XON/XOFF flow control does not work with multi-access mode. Use CTS, DSR, or CD conditioning instead.

Getting and Releasing the Data Bus

Use the BIOS **Request/Release Data Bus (INT A5h, Function 90h)** service to gain and release control of the cradle's data bus. This command works only with multi-access mode.

To gain control of the data bus, set the AL register to 0. To release control, set AL to 1. If the service returns 0, the terminal does *not* have control of the cradle's data bus. If the service returns 1, the terminal does have control of the cradle's data bus.

Sample Cradle Programs

There are two sample programs included with the ADK in directory C:\3000\SAMPLE\CRADLE:

- TTCC.C (for terminal controlled cradle communication)
- HHCC.C (for host controlled cradle communication)

The setup is as follows:

Host System: PC running PROCOMM

protocol: ASCII

baud rate: 9600

data bits: 8

stop bits: 1

parity: none

Terminal: Symbol Series 3000

protocol: MSISTD

baud rate: 9600

data bits: 8

stop bits: 1

parity: none

duplex: multi-access

Cradle: Four-slot

Connection: Null modem between the cradle and host.

The Terminal Controlled Communication Example

The terminal controlled communication program is called TTCC.EXE. The host program that goes with this program is called TTCC.BAT. TTCC.BAT starts PROCOMM PLUS on the PC, then executes the TTCC.ASP script file. These batch and script files are in the C:\3000\SAMPLE\CRADLE directory in the ADK.



In this example program, the terminals in the cradle compete for the data bus. The first terminal to get the bus uploads data to the host. This data consists of four fixed-length text strings:

1. "Terminal ID number *xxx* \n\r"
2. "This is text 2: Hello world\n\r"
3. "This is text 3: THE QUICK BROWN FOX JUMPED OVER THE LAZY DOG\n\r"
4. "This is the last text: End of text. \n\r x03"

where *xxx* is the terminal's ID number and x03 is the end of transmission (EOT) character.

Next, it downloads data from the host. This data consists of a single line of text:

```
"Message received from terminal xxx\x03",
```

where *xxx* is the terminal's ID number and x03 is the end of transmission (EOT) character.

Once the text has been received, the terminal releases the cradle's data bus, closes its communication line, and exits to DOS.

The Host Controlled Communication Example

The host controlled communication program is called HTCC.EXE. The host program that goes with this program is called HTCC.BAT. HTCC.BAT starts PROCOMM PLUS on the PC, then executes the HTCC.ASP script file. These batch and script files are in the C:\3000\SAMPLE\CRADLE directory in the ADK.

The HTCC.ASP script file sends a polling message from the host to the cradle. This polling message contains a terminal ID number. The terminal whose ID matches the one in the polling message gets control of the bus.

When the terminal gets control of the bus, it uploads four text strings to the host. The four text strings:

1. "Terminal ID number *xxx* \n\r"
2. "This is text 2: Hello world\n\r"

3. "This is text 3: THE QUICK BROWN FOX JUMPED OVER THE LAZY DOG\n\r"
4. "This is the last text: End of text. \n\r x03"
where *xxx* is the terminal's ID number and x03 is the end of transmission (EOT) character.

When the host receives the EOT, it transmits a single string acknowledgment:

"Message received from terminal *xxx*\x03",

where *xxx* is the terminal's ID number and x03 is the end of transmission (EOT) character.

Once it has received the EOT, the terminal releases the cradle's data bus, closes its communication line, and exits to DOS. The host then sends another polling message to the cradle, and the terminal indicated in the message gets control of the cradle's data bus. When the host sends the acknowledgment message to the last terminal, it closes the communication line to the cradle.



Communication Problems

The communication channel is a critical part of a communication session, some parts of which you have no control over. Communications problems can disrupt a session in many ways. If any of these events are not handled properly, the program may crash and you will be left with data in the terminal that can't be transmitted.

Direct connect, RS-232 communications are relatively secure, so few problems need to be anticipated.

Telephone communications, however, can encounter a wide range of problems, some of them involving external equipment (e.g., modems) as well as the telephone system itself. An application must cope with a variety of events that can terminate a telephone communication session and/or invalidate the data. Typical events of this sort are : noise on the line, the telephone system itself dropping the line, busy telephone numbers, or a wrong number.

Communications involving a portable terminal can suffer from an additional problems, notably power loss if the battery dies during a session.

Common Causes of Communication Failure

The following are some of the problems you should be prepared to address in your data communications applications:

Program Can't Communicate

Make sure the `\zp` switch or the `# pragma pack(1)` is used to pack IOCTL structures.

No DSR/DTR

No DSR/DTR (Data Set Ready/Data Terminal Ready) means there is no physical connection between the terminal and the host (Serial Communications).

No Response

The terminal sends out a signal, but the host does not reply. This could be caused by many things: an external modem not turned on, the host

computer not listening, or the host trying to transmit at the same time as the terminal.

Busy

The modem calls, but the telephone line is busy.

No Carrier Tone

The modem calls and gets a continuous stream of rings, but no answer (no carrier is present). Or the modem calls, and a person picks up the other end (a voice line). Here again, no carrier is present.

Features such as Call Progress Monitoring (in the URM) must be used to assure the user that the call has gone through and that data is being sent. See the *Modem Control Structure* section of the *UBASIC Record Manager* chapter in the *Series 3000 Application Programmer's Reference Manual*.

False Trigger

Suppose the cables are hooked between two computers and the power is turned on to both. The power-up generates noise on the line. If your application is properly designed, it will look only for certain kinds of characters and will not mistake this power-up noise for data.

Synchronization Problems

Once a false trigger has been received (for example), the true incoming character sequence will be offset by the amount of the noise, and synchronization between the two computers cannot take place.

Or the noise may make the host think it is the host's time to talk while the terminal thinks it is the terminal's time to talk. This means that neither the host nor the terminal is listening—they are both trying to talk. Meanwhile, the user gets tired of waiting and hangs up.

Or the user may start a communication session and transmit a lot of data—but because the host is expecting a certain sequence, it may not



recognize the data. The user thinks the communication session is proceeding properly, but no data is being exchanged.

Timeout

Timeouts can be related to No Response (described earlier in this listing). Data is sent to the host and the host responds, but the response has been garbled and the terminal can't recognize it.

Or the application dials a number and then waits for a dial tone. After a while, the terminal has to give up and will generate a TIMEOUT error.

Battery Failure

While you are performing a communication task, the battery may go dead. This is described in more detail under *Power Management* in this chapter.

Carrier Loss

This can be caused by many things. For example, a connection has been established between two modems, and the telephone company drops the line.

Or the terminal is hooked up, and someone trips over the communications cable.

Or you may have a situation where a line has been cut, but the host cannot detect this. It still thinks it is communicating and won't terminate. In this case, the application must provide some way to kill the communication process and regain control.

Data Corruption

Some protocols, such as TWOWAY, use checksums to insure that data is transferred properly. Other protocols, such as MSISTD, do not. When using a non-checking protocol, the application must provide a method of verifying the data.

DSR/DTR Loss

This is similar to Carrier Loss (described earlier in this listing) in telephone communication. If you directly connect two computers, and someone accidentally knocks the communication cable loose (or pulls the power cord out), DSR/DTR is lost.

Excessive Retries

Data has been sent, but the checksum didn't match. The host assumes that line noise corrupted the transmission, and the packet is sent again. Same result: it gets garbled again. Eventually, the terminal has to give up and generate an EXCESSIVE RETRIES error.

Memory Exhausted

More data is sent to the terminal than it has room for.

Data Overrun

Data is sent faster than the terminal can receive it. The buffers overflow, and data is lost.

This may also cause a synchronization problem (described earlier in this listing).

Recovering from a Lost Session

If a communication session is not completed, so that the host receives only part of the data, you have two available solutions:

1. Instruct the host to discard the data it has received; then retransmit all of the data.
2. Instruct the host to add the data in a second transmission to the data from the first transmission.



Remote PC Communication Software

Symbol Technologies' Terminal Management System (TMS™), Extended Terminal Management System (TME™), and Data Management System (DMX™) are PC utilities designed for communications with Symbol Technologies terminals. These utilities batch data received from terminals into a "call" file. TMS™ supports one channel, TME™ supports four channels, and DMX™ supports four channels and an optional host line.

To request a file from TMS™/TME™/DMX™ using the MSI Two-way protocol, the terminal must send an EBCDIC header consisting of a command and filename to the host. The format of this header for DMX™ is illustrated in Figure 11-4.

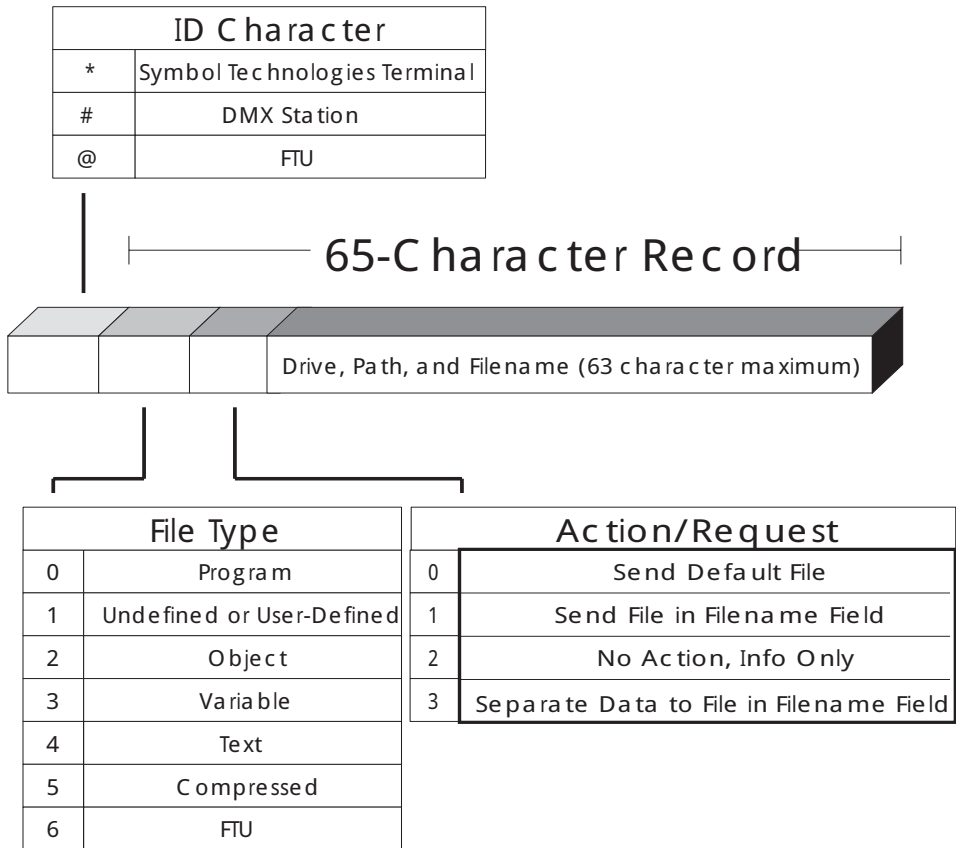


Figure 11-4. Header Block Format (DMX)

The header format for TMS™ and TME™ is a subset of the DMX™ format. For these protocols, observe the following:

- Only EBCDIC is supported (DMX™ also supports ASCII). ID Character must be 5C (asterisk character in EBCDIC).
- 16-character header records in header block
- FTU file type is not supported



Header Block Formatting

In the MSI Two-way communication mode, a Symbol Technologies terminal operator can request a particular program or data file from a PC running TMS/TME/DMX. The operator can do so by transmitting header block data from the terminal to the PC.

The header block consists of a special identification character (5C for asterisk character in EBCDIC or 2A for asterisk character in ASCII), followed by one to four 65-character header records. A header record contains file type information, the type of action requested (e.g., send a file or separate received data into a file), and an optional drive letter, path, and mandatory filename.

Information in a header block can be in the EBCDIC or ASCII character code depending on the ID character (TMS, however, only accepts EBCDIC). The header block must also be transmitted in the non-transparent mode (no DLE character). The header block must start with the SOH character (header record) (hexadecimal 01). If the Symbol Technologies terminal is not sending data to the IBM PC, the header block must end with the ETX character (trailer record) (hexadecimal 03). If data is to be included in the transmission, the header block must end with the ETB character (non-trailer record) (hexadecimal 26).

A second header block may be sent only after the Symbol Technologies terminal has received the requested file from the IBM PC. To send more than one file during the same transmission, include the header block in the last file.

The header block's filename field is long enough for a drive letter, path, and a full-length DOS filename (e.g., A:\MYFILES\TXTFILES\DATAFILE.TXT).

Figure 11-5 illustrates the effects of sending an ASCII and an EBCDIC asterisk character as an ID character to DMX. The ASCII asterisk allows you to specify ASCII filenames in the header block.



D = 2A

If 2A asterisk character, DMX expects ASCII



D = 5C

If 5C asterisk character, DMX expects EBCDIC

Figure 11-5. Sending ASCII and EBCDIC Asterisk

The Separate Command

In the sample program C:\3000\SAMPLE\COMM\URMCOM.C (in **C2_ComTSrnInitiate()**), the **separate** command sends the following string of characters to the PC:

```
0x5C,0xF4,0xF3,0xE7,0xE7,0xE7,0xE7,0xE7,0xE7,
0xE7,0xE7,0x4B,0xE7,0xE7,0xE7,0x00
```

This cryptic string of characters, when the EBCDIC hex is translated to ASCII, produces the following ASCII characters:

```
*43XXXXXXXXX.XXX
```

Referencing the header format on previous pages, this code tells the remote PC that the header is originating from a Symbol Technologies terminal (*), that the filetype is a text file (4), and that the terminal wants to send (separate) data to a file with the filename in the following field. The filename is **XXXXXXXXX.XXX**.

This record is tagged as a header. This is because more records will follow, i.e. the file itself will be sent in records (tagged as non-header/non-trailer) and finally the last record in the file will be sent as a trailer.



The Send Request Command

The send request command is almost identical to the previous function. It differs in two ways: (1) A different EBCDIC string is sent and (2) A Line Turn Around must be performed following the command.

The string in this function translates to:

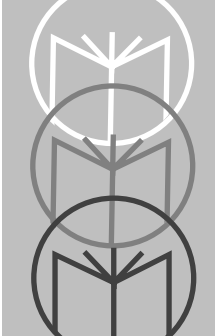
***41YYYYYYYY.YYY**

According to the header format, this code tells the remote PC that the header is originating from a Symbol Technologies terminal (*), that the filetype is a text file (4), and that the terminal is requesting a file be sent to the terminal from the remote PC (1). The filename on the PC is YYYYYYYY.YYY.

Following the sending of the Request Command (which must be sent as a trailer), a Line Turn Around must be performed. This signals to the remote communication software (TMS/TME/DMX) that the application is not sending anymore.

Two-way protocol dictates that in order to turn the line around, you must indicate to the other terminal that you are not sending anymore. This is done via an EOT (End of Transmission). It should also be indicated by the last record sent which should be tagged as a trailer record.

If the send request command is the only block that will be sent to the remote PC, i.e. no data blocks are needed by the remote PC, this block must be tagged as a header *and* a trailer.



Chapter 12

Writing Resident Programs

Chapter Contents

Introduction	12-3
Types of Resident Programs.....	12-5
Separating the Data from the Code.....	12-8
Calling a Resident Program	12-9
Memory Models for Resident Programs	12-10
Writing Resident Programs	12-11
The Resident Library Builder.....	12-14



Introduction

As you read this chapter, we recommend that you have available for ready reference the following Series 3000 ADK document:

Document Title	Chapter Title (s)
<i>Series 3000 Application Programmer's Reference Manual</i>	<i>Utilities</i>

An NVM resident program starts out its life as a resident program. DOS is not even aware of its existence.

Early in the system boot-up process, the BIOS provides NVM resident programs an opportunity to initialize themselves and to establish the mechanism by which they will be invoked. After DOS is loaded, a calling program can use this mechanism to invoke the program.

NVM resident programs have the following advantages:

- The code runs directly from NVM rather than being copied to RAM for execution. This leaves more memory for data.
- The code is available to be called by other programs.

The disadvantage is that an NVM resident program cannot have a writable data segment.

Split-resident programs have the advantages of NVM resident programs, but have a writable data segment in RAM. They have the following advantages:

- The code runs directly from NVM leaving more memory available for data collection.
- The program has a writable data segment in RAM.

This chapter explains how to build split-resident programs.



NVM resident and split-resident programs can be used to develop library files. They can then be loaded to run from NVM, saving valuable RAM but remaining available to many executable programs running in RAM. Building libraries is a more advanced topic and is usually done by a systems programmer. We include instructions here for completeness.

Sample Programs

The sample programs for resident programming are in the following ADK directories:

- **C:\3000\SAMPLE\RESIDENT\READ** contains programs with a read-only data segment
- **C:\3000\SAMPLE\RESIDENT\WRITE** contains programs with a writable data segment

These examples show how to create an application that uses various types of libraries, how to create the libraries, and how to use the Resident Library Builder.

Types of Resident Programs

There are three types of resident programs of particular importance for programming the Series 3000:

- NVM resident programs
- split-resident programs
- resident libraries

NVM Resident Programs

NVM resident programs require special considerations, mainly because they do *not* have writable data segments. In normal operation, NVM is read only memory. Since an NVM resident program runs in place in the NVM, any attempt to write to its data segment will fail and cause a program failure.

To allow a program to operate, its stack must be separated from its data segment. C compilers normally assume that the stack is part of the data segment, that $SS=DS$. This cannot be true for resident programs.

To override this assumption and make a program loadable as NVM resident, include the **-Aw** switch on the compiler command line:

```
cl -Aw options program.c
```

Then specify the resulting program as a resident file when you run USRCFG (refer to the chapter on *NVM Configuration* in this guide).

Split-Resident Code Files

The User Configuration Tool (USRCFG) allows you to separate the code and data segments of an application program when building an NVM image. The code segment is loaded as an NVM resident program. The data segment resides in the NVM disk as a small executable file providing a data segment.



The only preparation you need to make for USRCFG to split a program is to build it using the special Symbol libraries provided with the ADK in C:\3000\LIB\LIBRS or C:\3000\VLIB\LIBRS. These libraries provide the special code required by USRCFG to split the program. For a description of how these libraries are created, refer to the *Modifying C Libraries* section of the chapter on *Installing the ADK* in this guide.

See the WRITE.MAK file in the C:\3000\SAMPLE\RESIDENT\WRITE directory of the ADK for an example of how to build the program.

To load the program as split resident, load it using the */rs* switch in the USRCFG response file. Refer to the chapter on *NVM Configuration* in this guide and to the *USRCFG* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual* for further information on the User Configuration Tool.

Resident Libraries

Creating NVM resident libraries is a more involved process than for the two previous types of resident programs. Resident libraries exist in NVM as code segments executing in the background, providing a set of functions for other foreground programs to access.

A foreground program can make three types of library calls:

1. A call to a library function which is linked into the code of the foreground program.
2. A call to a resident library function using a software interrupt.
3. A call to a linked Interface Library function which is linked into the code of the foreground program. The interface library uses a software interrupt to call a resident library function.

The last two options require that a library be present as a resident program.

A typical application using resident libraries uses two libraries: the resident library itself and a linkable interface library. (See the description of the File Manager libraries in the *Managing Files* chapter of this guide for an example of how these are used.)

The resident library consists of a set of routines loaded as a resident program in NVM and run as a TSR. (They can also be loaded and run as a RAM TSR, but in this chapter we focus on NVM resident libraries.) The foreground program uses a software interrupt to call the various resident library routines. The interrupt can be made either directly or indirectly through an interface library.

When a library is made resident, an interface library is usually also provided. The interface library contains very short routines that make the software interrupt call to the full routine in the resident library.

The interface library is linked with any application that needs to access the resident library routines, just as with a normal linked library. The short routines in the interface library should have the same names and calling parameters as the functions in the resident library.

Any functions in a resident library used as an entry point for a calling program must include the **`_loadds`** keyword as a preface to a function definition. **`_loadds`** saves the caller's DS, loads the resident DS, and then restores the caller's DS before returning.



Separating the Data from the Code

C programs use three kinds of memory:

1. Static data, uses Data Segment register (DS)
2. The stack, uses Stack Segment register (SS)
3. The heap

Typically, the stack is contained in the static data and they jointly form **DGROUP**. The default for the Microsoft C compiler sets DS=SS, and both point to the start of **DGROUP**. The offsets used to reference these two areas are different offsets from the same base address.

The C compiler switch **-Aw** overrides the compiler default, thus separating the stack from the data segment and making the stack unwritable. Unless a program specifically defines a data segment, it does not have one, just as required by an NVM resident program. Do not do this for split-resident programs. Instead, compile the program using the libraries in C:\3000\VLIB\LIBRS on the ADK. The User Configuration Tool (USRCFG) then separates code and data.

Calling a Resident Program

A program calls a resident program by passing a stack to the resident program. The stack must contain any parameters required by the resident program and the return address of the calling program. The stack is usually a different stack than the resident program's stack.

Resident library function used as an entry point for a calling program include the **_loadds** keyword as a preface to a function definition. **_loadds** saves the caller's DS, loads the resident DS, and then restores the caller's DS before returning.

Some C run-time library routines assume that DS=SS, and so cannot be called by a resident program. Any routine that writes to the static data in **DGROUP** cannot be called by resident programs. An example of such a routine is **PRINTF** which defines a private static data buffer but is part of the static data area for the program. It builds the line to be printed in that buffer, as it interprets the format codes. Because static data is not writable, **PRINTF** will generate an error.



Memory Models for Resident Programs

The Microsoft Visual C memory models use the following segment structure:

- The tiny model has one segment for both code and data. Programs using the tiny model cannot be loaded NVM resident or split-resident.
- The small model has one code segment and one data segment.
- The medium model has multiple code segments and one data segment.
- The compact model has one code segment and multiple data segments.
- The large model has multiple code and data segments.

Resident programs can use the small, medium, or compact models.

Split-resident programs must be compiled using the small or medium memory model. Larger models rely on code knowing the location of the data at link time. Since split-resident program code and data are separated when the NVM image is built, the code cannot know in advance where in memory the data will be.

The Microsoft Visual C implementation of multiple data segments requires that the code segments know where the data segments are. The compiler needs this information so it can generate code to set up segment registers at various times to point to the right data.

In the small and medium models, the DS register is set up by the startup code and assumed to have the right value forever after.

Writing Resident Programs

The ADK supplies modified code and library routines to assist you in writing resident and split resident programs. These are located in the following directories:

C:\3000\VLIB\SMALL

C:\3000\VLIB\MEDIUM

Included in these directories are two startup code modules:

CRT0.OBJ

CRT0DAT.OBJ

and three library routines:

HARDERR.OBJ

SIGNAL.OBJ

DOSPAWN.OBJ

Split-Resident Programming Considerations

In general, the following types of files *cannot* be loaded as split-resident programs:

1. Files that contain a data segment or data group outside **DGROUP**. These include:
 - Files that are compiled with the large, huge, or compact models.
 - Files that link the Microsoft floating point package. This package creates and accesses a data group **FMGROUP** outside **DGROUP**.
 - Files that place the stack in a segment outside of **DGROUP**.

Check the MAP file generated by the linker to see if any data segment is outside **DGROUP**. If all data segments are in **DGROUP**, the “**ENDCODE**” class is followed immediately by the “**BEGDATA**” class in the “**Class**” column.



2. Files that reference the value of **DGROUP** during assembly or compiling time.

The following 8086 instruction will cause a user configuration error when attempting to load the program as split-resident:

```
mov ax, DGROUP
```

The C compiler generates the above line of code in the following conditions:

- the **_loadds** attribute is used with a function
 - the **_interrupt** attribute is used with a function
 - the compiler switch **/Au** is used
3. Files that are composed of overlay modules. These will generate a run time error because the overlay manager will fail to load a new overlay module into the user NVM.
 4. Files that write to the code segment during run time.

A Split-Resident Program as a TSR

A split-resident program that runs as a foreground process can be written pretty much like any normal program as long as the situations described above are avoided. A split-resident program that is to run as a TSR, however, is more difficult.

Most TSR's want to have writable data to keep track of historical or state information. If a split-resident program is a TSR, it has to find the data segment during the entry process.

The Resident Library Builder utility provided with the ADK can create a split-resident TSR for you automatically. This utility is described later in the *Resident Library Builder* section of this chapter.

Interrupt Handlers

If a resident program is used as a software interrupt handler, it must meet the following requirements:

- Save and restore all registers using **_saveregs**.
- Return using **IRET**, not **RET**, using **_interrupt**.

A resident program can also be a hardware interrupt handler. Such a program must meet all the requirements of a hardware interrupt handler. These requirements include:

- Save and restore all registers using **_saveregs**.
- Return using **IRET**, not **RET**, using **_interrupt**.
- Reset any hardware ports required.

You may find it easier to write the entry point into a resident program in assembly language and then call the C code from the assembly language.



The Resident Library Builder

An application program calls resident library functions using software interrupts. The application can invoke the interrupt directly, but more commonly it does so indirectly using an interface library.

The Resident Library Builder utility simplifies the process of generating both the resident library and the interface library. It also produces the necessary header files to provide the correct files and declarations. The Resident Library Builder consists of two parts, GENRES.EXE and GENINTF.EXE.

The header (.H) files allow an application programmer to include the interface library correctly. Resident library header files cause the linker to link the interface library routines instead of using the standard Microsoft C routines. These header files declare all resident library functions and all pointers as far. This modification ensures that the application program sets data segment registers correctly.

Procedure

The following is a summary of how to build a resident library and associated files using the Resident Library Builder:

1. Create Routines
2. Compile Routines
3. Run **GENRES.EXE**
4. Make the Library
5. Create two assembly programs files: the installation routine and interrupt handler
6. Run **GENINTF.EXE**
7. Create Interface Library
8. Write and Compile the Install Program and the Interrupt Handler Program

Creating Resident Library Routines

The first step is to write the functions to include in your resident library.

Each routine should have a name unique to the 8th character (the DOS file name prefix limit). This is because each routine is compiled as a separate file that inherits the name of the routine. When compiled and processed by GENINTF, the resulting file will inherit the name of the function.

For routines that already exist and do not have unique eight-character names, GENINTF changes the last character to a single digit from 0 to 9 and uses that as the name. Therefore, your list can contain, at most, ten non-unique routine names. If that limit is exceeded, GENINTF issues an error.

Programming Notes

The following guidelines must be followed for routines with various residency requirements:

NVM resident with no data segment:

- The file is composed of code only.
- The whole file can be included in the system NVM area.
- Use the stack space as a Read/Write working area (local variables only).
- The INSTALL routine should set the interrupt handler address in the appropriate interrupt vector.
- The interrupt handler must save and restore the application DS and load its own DS to point to CONST_TEXT. It then should dispatch to the appropriate resident routine.
- Compile with the following switches:
 - /AL switch to select the large model
 - /GS switch to disable stack overflow checking

NVM resident with read-only data segment:

- The file is composed of code only.
- The whole file fits in the system NVM area.



- Use the stack space as a Read/Write working area (local variables only).
- Include constant data in the CODE group. The constant data can be placed in the CODE group using the following sample technique:

```
CONST_TEXT      Segment Public 'Code'
Public          Line
Line            db          'This is a line'
CONST_TEXT      ends
```

- The interrupt handler must save and restore the application DS and load its own DS to point to CONST_TEXT. It then should dispatch to the appropriate resident routine.
- Compile with the following switches:
 - /Aw to separate SS and DS
 - /AL to select the large model
 - /GS to disable stack overflow checking

Split-resident with read/write data segment

- The file is composed of CODE and DATA group.
- Only the CODE part can be loaded to NVM.
- The INSTALL routine should set the interrupt handler address in the appropriate interrupt vector and make the whole resident library, including the data segment, a TSR. (After this program is processed by the User Configuration Tool, the code part of the resident library is loaded to NVM and the data part is loaded to RAM after the INSTALL program is run once.)
- The INSTALL program should be compiled with either the Small or Medium model. These are the only two models that can be split.
- The Interrupt handler should save and restore the application DS and load the DS for the resident library.
- If written in C, compile with the /Aw switch to set SS != DS; DS not reloaded on function entry; Compile with the /AM switch to select the medium model.
- With the User Configuration Tool (USRCFG), use the /rs switch for "Load

a Resident code file with split code and data.”

- From the map file generated by the linker, ensure that no data is loaded between the ENDCODE and the BEGDATA class. This causes problems in addressing the data segment. Hence, no double variables or floating point variables should be used in a C program.

Run GENRES.EXE

GENRES.EXE is the first of two programs that help you to generate a resident library. You supply one input file containing a list of all the routines in your library. GENRES outputs a MAKE description file and LIB response file set (.MAK and .RSP) for a resident library.

Syntax

```
genres routinelist.TXT
```

where:

```
routinelist.TXT
```

lists all (writable and readable) .OBJ routines to be placed in your resident library. For example: MYLIB.TXT

Sample

```
GENRES MYLIB.TXT
```

Output

```
routinelist.MAK
```

MAKE description file used by MAKE to generate a resident library.

```
routinelist.RSP
```

Response file used by the librarian utility (LIB) to generate a resident library.



Make the Library

To generate a resident library, execute MAKE using the MAKE description file (.MAK) output from the GENRES utility. Copy the MAKE description files (.MAK) and the LIB response files (.RSP) to the directory that contains all the object file library routines. The following commands create your resident library. MYLIB.RSP, the response file used in connection with LIB, is called from within the Make description file. LIB assembles the routines into a library file.

Syntax

```
make library.MAK
```

where:

```
library.MAK
```

MAKE description file used by MAKE to generate a resident library.
Output of GENRES.EXE. For example: MYLIB.MAK.

Sample

```
MAKE MYLIB.MAK
```


Create the Install Program and the Interrupt Handler

The installation and interrupt handler programs must be created in assembly language. See the following sample programs in the ADK:

C:\3000\SAMPLE\RESIDENT\READ\INSTALLR.ASM (*read-only*)

C:\3000\SAMPLE\RESIDENT\WRITE\INSTALLW.ASM (*read/write*)

The Install program should do the following:

- Set the interrupt handler's address to an unused interrupt vector. In general, select an interrupt vector in the range from E8 to ED, but you are not restricted to these. Vectors 60 to 67 are usually available, but may not be software compatible with other programs. When the application program generates this interrupt, it calls the interrupt handler.
- For resident library routines with a writable data segment, save the data segment register in the next interrupt vector. Generate a DOS TSR call to cause the data segment to stay in RAM.

The Interrupt Handler program should do the following:

- Declare all the resident routines as external.
- Set up a dispatch table in a data segment that contains the addresses of the resident routines.
- Check the input command code and dispatch to the appropriate resident routines.
- Adjust the stack pointer to skip the interface routine return address and the flags so that it can return directly to the application when the resident routine exits.
- For the resident library with a writable data segment, load the value of the system variable STKHQQ in BX to the reserved location. STKHQQ is used to check stack overflow. Because all resident routines use the application's stack space, they should also use the application's STKHQQ value to check for overflow. The application's STKHQQ is moved to BX by the interface routines. Because the 'No Data Segment' and 'Read-Only Data Segment' resident library types have no space to save the application's STKHQQ value, they are compiled with the /GS switch to



disable stack checking.

Assemble the interrupt handler using the macroassembler **/MX** switch to preserve the case of the resident routine names.

Generate the Interface Library and Header Files

GENINTF.EXE generates the interface library and header files. It requires two parameters: a name for the resulting interface library and an interrupt vector number.

Three optional input parameters can be given for modifying Microsoft C header files. GENINTF changes the model dependent declaration of '_CDECL' to 'far' and all pointer declarations to far pointers.

Syntax

```
genintfinterfacelib_nameinterrupt_#[header_list][path_original_h]  
[path_new_h]
```

where:

interfacelib_name

This file contains the interface library routine names preceded by an underscore (e.g., `_routine1`, `_routine2`). The default filename extension is `".TXT"`. The filename prefix is used for the interface library, the MAKE file, and interrupt handler source output files. The underscore is not required for assembly language routines.

interrupt_#

The interrupt number to invoke the resident routines. Use any interrupt number not being used by the system. See `\3000\3000\INTNEW.ASM` to identify an unused interrupt vector number.

header_list

Text file containing a list of header files. The header files must be in the directory specified by *path_original_h*.

path_original_h

The directory path containing the header files specified in *header_list*.

path_new_h

Directory path that specifies the path where the terminal's Interface Library .H files will reside. For example: **C:\3000\3000**.

Sample

```
GENINTF interfacelib_name 0E0H IFACELIB C:\3000 C:\3000\3000\
```

where *interfacelib_name* is a name supplied by the user.

Output

interfacelib_name.MAK

MAKE description file.

interfacelib_name.RSP

Response file for the librarian utility (LIB).

interfacelib_name.EXT

MASM source file declaring all interface routine names as external.
Include the text from this file in the interrupt handler source file.

interfacelib_name.DD

MASM source file reserving double word space in the dispatch table for each interface routine. Include the text from this file in the interrupt handler source file.

*.ASM

Source files for each interface routine. A .ASM file is generated for each routine listed in the *interfacelib_name*.TXT input file.



*.H

Modified Microsoft C header files. GENINTF creates a duplicate of each .H file listed in *header_list* and modifies it. It then copies the modified .H files to *path_new_h*. The application must include the modified version of the header files.

Make the Interface Library

Run the MAKE utility using the MAKE description file (.MAK) generated by GENINTF.EXE:

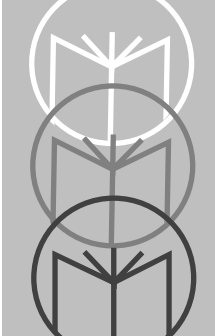
```
make interfacelib_name.MAK
```

MAKE calls MASM to assemble .ASM files creating .OBJ files, and calls LIB to create the interface library. (See *interfacelib_name.MAK* for the sample description file.)

Using the Results

At this point you have the files you need to write an application using the resident and interface libraries:

- When you invoke the User Configuration Tool (USRCFG) to build the NVM image, specify the resident library as a resident or split-resident file, as appropriate. For more detailed information on the User Configuration Tool, refer to the chapter on *NVM Configuration* in this guide and to the *USRCFG* section of the *Utilities* chapter in the *Series 3000 Application Programmer's Reference Manual*.
- Include (`#include`) the header files in your application program.
- Link in the interface library with your application as with a normal interface library.
- If the resident routines are replacements of existing Microsoft C library routines, link using the `/NOE` switch.



Appendix A

Monarch Printer Drivers

Appendix Contents

Introduction	A-3
Installing the Monarch 9490 Printer Driver	A-3
Installing the Monarch 9450 Printer Driver	A-4



Introduction

This appendix describes the installation of the Monarch 9490 and 9450 Printer Drivers. The software for these drivers is located on the ADK 3.4 development diskettes.

Note: The Monarch Printer Drivers are supported by Monarch Marking Systems directly. Monarch Marking Systems provides telephone technical support for its printer drivers at a nominal charge. Please contact the Monarch Expert Solutions Center at (800) 543-6650 for technical support, pricing and services.

Installing the Monarch 9490 Printer Driver

To load the Monarch 9490 Printer Driver:

1. Ensure the Monarch 9490 software is version 3.0 or higher.
2. Make sure the Symbol Series 3800 terminal BIOS is version 3.07-1 or higher.
3. Use bidirectional PIM (Printer Interface Module), part number 116587-32.
4. Configure the 9490 printer for 9600, no parity, 8 bits, XON/XOFF. The Monarch 9490 Test Label COM settings should be: 9600,N,8,1,X.
5. Download the file MMS9490.EXE into the LRT/LDT 38xx terminal using the TDREM facility. See chapter 2 for more information on the use of this facility.
6. Once the download is successfully completed, load the driver into the terminal's memory using the following command:

```
MMS9490/C:9600,n,8,1,x
```

The driver is now installed and ready for use.

If necessary, the test program RENEGADE.EXE can be loaded into the terminal during debugging. This program tests the driver and displays the results. It can be used to verify setup, cables, and connections.



Installing the Monarch 9450 Printer Driver

To load the Monarch 9450 Printer Driver:

1. Ensure the Monarch 9450 software is version 2.0 or higher.
2. Make sure the Symbol Series 3800 terminal BIOS is version 3.07-1 or higher.
3. Use bidirectional PIM (Printer Interface Module), part number 116587-32.
4. Configure the 9450 printer for 9600, no parity, 8 bits, XON/XOFF.
The Monarch 9450 Test Label COM settings should be: 9600,N,8,1,X.
5. Download the file MMS9450.EXE into the LRT/LDT 38xx terminal using the TDREM facility. See chapter 2 for more information on the use of this facility.
6. Once the download is successfully completed, load the driver into the terminal's memory using the following command:

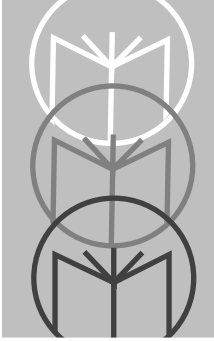
```
MMS9450/C:9600,n,8,1,x
```

The driver is now installed and ready for use.

If necessary, the test program RASCAL.EXE can be loaded into the terminal during debugging. This program tests the driver and displays the results. It can be used to verify setup, cables, and connections.

Note: The above instructions are generic to the Series 3800 terminals. When using other supported terminals, use the following cable:

part number 116587-16 (using 10-pin modular jack)



Index

Symbols

.CFL files 4-11
_MAP 4-15
_cursor_tbl. 5-8
_dos_getdiskfree 7-10
_kybd_tbl. 5-8
_rep_cell_msg. 5-8

A

All window commands. 8-26
ALT state. 9-4
Alternating Protocols. 11-14
ANSI compatibility driver 8-19
ANSI3000 compatibility driver 8-19
APIs, communication 11-9
Application layer. OSI model. 11-6
Application Programming Interfaces (APIs)
 2-14
Attributes, Character 8-17
Attributes, character 8-17
AUTOEXEC.BAT 1-7, 4-9, 4-11, 4-19

B

backlight 9-9
Backlighting 8-18
Bar code lengths
 Any Length 10-33
 Length within Range 10-33
 One Discrete Length 10-33
 Two Discrete Lengths 10-33
Bar code menu labels. 10-41
 Aiming Dot 10-43
 Code Lengths 10-48
 Code types. 10-44

 Decode Options 10-57
 Set Default Parameter. 10-42
 Slab Raster 10-43
 Special decode options 10-71
 UPC-A Preamble. 10-68
 UPC-E0 Preamble 10-69
 UPC-E1 Preamble 10-70

Bar code menu parameters

 Code types 10-39
 Decode options 10-40
 Special decode options 10-41

Bar code menus .. 10-30, 10-31, 10-32, 10-39

Bar code scanning 10-3

basic character editing 8-23

Batch mode communications 11-4

Battery Cell Status. 11-42

BEGDATA 4-15

BIOS 11-34, 11-46

BIOS API. 11-34

BIOS calls 11-35

BIOS Interface Libraries 2-15

BiosCheckBattery 11-42

BiosGetPowerSource 11-41

BLDINIT.EXE (Terminal Initialization Con-
figuration Tool) 5-3, 5-4, 5-5, 5-6, 5-8,
5-11, 5-12

BLDSCAN.EXE 10-6, 10-14

Bounded Searching. 6-7

Buffer. 11-23

Buffer, Comm 11-23

C

Call Progress Monitoring. 11-21



- Calling a resident program. 12-9
- CCM (Charging and Communications Module). See Cradle.
- Character 8-17
- Character attributes 8-17
- character window 8-24
- characters. 9-4
- Charging and Communications Module (CCM). See Cradle.
- Check digits 10-75
- Clearing the Display 8-18
- Clock 3-9
- Clustering 6-6
- Code files. 12-5
- Resident. 4-14
- Code lengths. 10-33
- Cold Boot. 2-13
- Comm . . . 11-15, 11-23, 11-34, 11-43, 11-54
- Comm API. 11-34
- Command Line mode 4-14
- Command Line mode switches 4-14
- Output, main 4-14
- Output, print. 4-14
- Command shell 4-9
- COMMAND.COM. 4-11
- Command.com. 2-9
- Communicating through a cradle . . . 11-44
- Communication APIs 11-9
- BIOS. 11-34
- DOS 11-15
- DR DOS 11-30
- URM (UBASIC Record Manager) 11-12, 11-15
- Communication problems 11-50
- Communication programs, sample. . . 11-36
- Communication Status Codes 2-12
- Communications buffer 11-23
- Communications failure
- Battery Failure 11-52
- Busy 11-51
- Carrier Loss. 11-52
- Data Corruption 11-52
- Data Overrun. 11-53
- DSR/DTR Loss 11-53
- Excessive Retries 11-53
- False Trigger 11-51
- Memory Exhausted. 11-53
- No Carrier Tone 11-51
- No DSR/DTR. 11-50
- No Response 11-50
- Program Can't Communicate . . . 11-50
- Recovering from a Lost Session . 11-53
- Synchronization Problems. . . . 11-51
- Timeout. 11-52
- Communications Model. 11-5
- Communications, batch and interactive modes 11-4
- Communications, data 11-3
- Compressed programs 7-7
- CONFIG.SYS 1-7, 4-8, 4-11, 4-19
- Configuration
- NVM. 4-10
- Configuration, PC 1-9
- Configurations, NVM. 4-7
- Connecting terminal to development PC 11-10
- Connecting Terminal to PC 2-10
- contrast 9-9
- control state 9-4
- Control Structure. 11-21
- Cradle communication sessions, types of 11-45
- Cradle communications 11-44
- Host controlled 11-45, 11-48
- Terminal controlled 11-45, 11-47
- Cradle Communications API 11-45
- Cradle management API
- Checking that terminal is in cradle 11-46
- Getting/Releasing the data bus . 11-46
- Setting communications parameters 11-46
- Cradle programming, single slot . . . 11-44
- Cradle programs, sample. 11-47

Cursor control. 8-16
Cursor modes 8-16
Cursor shapes 8-16
Cursor Translation 5-9
Cursor Translation Table 5-8, 5-9
Custom Environment, Creating 4-19

D

Data communications. See Communications, data.

Data Management System (DMX) 11-54

Default Initialization Parameters. . . . 5-12

Device Name Translation Table. 11-22

Device Names 11-20

Disable Power Key Service 9-26

Disk Drives 3-6

Display 3-10, 8-4, 8-5, 8-8, 9-9

 Backlighting 8-18

 Clearing 8-18

 Contrast 8-18

 Fonts 8-21

 Logical screen 8-8

 Multiple pages 8-10

 Pages 8-7, 8-15

 Positioning 8-6

 Screen size 8-3

 Virtual screen 8-4

 Virtual screen segments 8-6

Display adapter settings 9-29

Display Paging 8-15

Display performance, improving 8-12

Display Physical screen 8-4

Display routines

 BIOS 8-13

 C routines 8-12

 DOS routines 8-12

display segments 8-6

DMX 11-54

DOS I/O Control (IOCTL) commands . 9-21

DOS I/O control (IOCTL) commands

 Opening/closing a handle 9-22

 Parameter structure 9-21

 Selecting a function 9-22

Download Error Codes 2-12

Downloading 2-5, 2-11

DR DOS 4-3, 4-13

DR DOS data communications functions 11-30

DR DOS file routines 8-13

Dumb Terminal 11-15

Dumb Terminal, Communication with 11-15

E

EMS 7-13

EMS requirements 7-5

End-of-Record Flag 11-27

Error Codes

 Download 2-12

Error codes, scanning 10-10

ETA3000.SYS 5-3

Extended Terminal Management System (TME) 11-54

External modem 11-21

External, Initialization 11-18

F

File Functions 2-14

File Management APIs, DOS 6-13

File management functions

 DR DOS 7-10

 DR DOS Library File Functions . . 6-15

 UBASIC File Manager 7-12

File Manager 6-13, 7-13

File Manager, Residency of 6-10

File request header 11-56

File routines, DR DOS 8-13

.FNT file 8-21

Font Builder

 Command reference 8-26

Font builder 8-21, 8-23, 8-24, 8-26

 Character editing 8-24



Font loaded as TSR	8-22
Font window	8-24
Text comparison	8-25
Text window commands	8-30
Font builder (FONTBLD.EXE)	8-22
Font Builder editing screen	
Character window	8-24, 8-28
Font window	8-23, 8-27
General commands	8-26
Help window	8-24
Text window	8-24, 8-25, 8-30
Font Builder main screen. See Font Builder editing screen.	
Font builder, Running	8-22
Font formats	8-21
Font TSR, Making a	8-31
FONBLD (Font builder)	8-22
FONTBLD.EXE	5-8, 5-9
Fonts	8-21
fonts	8-4
FREEDISK	7-4
FREEMEM	7-4
Func key	9-10

G

GENINTF.EXE	12-14
GENRES.EXE	12-14, 12-17

H

Hardware Environment	3-3
Hayes Modem Initialization	11-18
Header Block	11-29
Header Block Format	11-56
Header Trailer Block	11-29
Headers and Trailers	11-28
Help Window	8-24
HEX File to NVM	2-11
Hex file, creating	2-9
Hypertext Sample Programs	xiv

I

I/O Control Structure (Ioctl_S)	11-31
I/O Ports	3-6
IEC825 Class 1	10-30
IEC825 Class 1 restrictions	10-30
INIT.EXE	5-3, 5-7
Initialization parameters, default	5-12
Initialization parameters, terminal	5-12
Initialization, Default Parameters	5-12
Initialization, scanner	10-88
Input	4-14
Input/Output Control	11-31
Installing	xiv
Installing ADK Software	1-5
Interactive mode communications	11-4
Internal	3-8
Internal modem	11-33
Internal modems	3-8
International Standards Organization (ISO)	11-6
Interrupt handlers	12-13
INTNEW.ASM	12-20
IOCTL	11-32
IOCTL (Input/Output Control)	11-31
IOCTL parameter structure	9-21

K

KBD3000.EXE. See Keyboard redefinition.	
KBDMAKE.EXE. See Keyboard Map Definition utility.	
KBDMAKE.EXE. See Keyboard Mapping utility.	
Key processing	
Using BIOS routines	9-18
Using DOS routines	9-18
Keyboard	3-10, 5-8, 9-4, 9-10, 9-23, 10-8
Accessing the device driver	9-16
Alt key	9-10
Character translation	9-4
Ctrl key	9-10
Func key	9-10

Func state 9-4
 Meta codes 9-9
 Processing input 9-14
 Redefining 9-35
 Sample programs 9-37
 Scan code translation 9-5
 Shifted state 9-4
 States 9-4
 Translation restrictions 9-11
 Translation table 9-8, 9-36
 Using DOS I/O control functions 9-21
 Keyboard data, accessing
 Using BIOS routines 9-18
 Using C runtime library functions 9-16
 Using DOS file I/O commands 9-16
 Using DOS routines 9-18
 Keyboard files, generating 9-34
 Keyboard input routines 9-23
 Keyboard input, processing 9-14
 Keyboard layout 9-30
 Keyboard Map Definition utility 9-29
 Keyboard mapping 9-28
 Keyboard Mapping utility
 Creating and editing a keyboard layout
 9-30
 Generating keyboard files 9-34
 Key definition screen 9-32
 Key selection screen 9-33
 Starting the keyboard mapper 9-30
 Keyboard Mapping Utility (KBD-
 MAKE.EXE) 5-8
 Keyboard Mapping utility (KBDMAKE.EXE)
 9-28, 9-29
 Keyboard operation 9-4
 Keyboard redefinition 9-11, 9-28, 9-36
 Keyboard Redefinition Table 5-8
 Keyboard states 9-9
 Keyboard Timeout Services 9-23
 Keyboard translation 9-34
 Keyboards 9-4

L

Laser emission restrictions 10-30
 Line Turn Around 11-14, 11-15
 Logical Protocol, Selecting 11-25
 Logical screen 8-8
 Low memory 7-8

M

maximum virtual screen size 8-5
 Memory
 Video 8-5
 Memory Functions 2-15
 Memory Functions, DR-DOS 7-9
 Memory Management 11-42
 Memory Management by C Programs 12-8
 Memory management functions
 DR DOS Library 7-9
 EMS 7-13
 Microsoft Visual C Library 7-9
 UBASIC Memory Manager 7-11
 Memory models for resident programs 12-10
 Memory requirements, program 7-4
 Memory, maximizing 7-6
 Compressed programs 7-7
 Managing available memory 7-9
 NVM Disk 7-7
 NVM Split Resident 7-7
 RAM disk resident programs 7-8
 Using program residency strategies 7-6
 mfree 7-13
 Microsoft Visual C runtime functions 6-14
 MODALL 1-7
 MODALLV.BAT 1-8
 Modem3-8, 3-9, 11-17, 11-18, 11-20, 11-21, 11-33
 Modes, cursor 8-16
 MSICFG.EXE 1-9
 MSICONFIG 1-9
 MSISIMPLEX 11-25
 MSITWOWAY 11-27
 MSIXONXOFF 11-26



N

Notational conventions	xiii
NULLSYS.HEX	4-11, 4-12, 4-14
NVM	4-11
Configuration and download process	4-10
NVM configurations	4-7
NVM requirements	7-5
NVM resident libraries	12-6
NVM resident programs	12-3, 12-5

O

OSI Model	11-6
Overwrite Protection	6-7

P

Packing Structures	11-22
Parameter menu scanning	10-39
Parameters, Scanning	10-32
Path	1-7
PC (Remote)	11-54
PC BIOS compatibility	3-11
PC communication software	11-54
PDF 417	10-18
Data Mode	10-20
Scanning mode options	10-43
Submenu	10-18
Support	10-7
Templates	10-21
Physical screen	8-4
Port Names	11-20
Power	9-26
Power Management	3-10
Power management	11-41
Battery Cell Status	11-42
Conserving Battery Power	11-41
Determining terminal power source	11-41
Power management, keyboard controls	9-23

Power On/Off	9-9
Power Source Service	11-41
Power, conserving terminal	9-23
Disable Power Key service	9-26
Keyboard timeout services	9-23
Power Off Terminal service	9-23
Waking up the terminal	9-24
Presentation layer, OSI model	11-6
Processing keys and labels	9-14
Program Loader	2-11
Program Residency	12-4
Program to RAM Disk	2-5
Programming notes	12-15
Programs, compressed	7-7
Protocols	11-25, 11-26, 11-27

R

Radio	3-9
radio communications	
Spectrum One	3-9
Spectrum24	3-9
RAM disk configuration	5-8, 5-11
RAM Disk Configuration Routine	5-11
RAM Disk Resident Programs	7-8
ram_cfg	5-8
Real Time Clock (RTC) wakeup	9-25
rec_flg, using	11-27
Record Storage Efficiency	6-5
Record Type Information	6-6
Record Update Support	6-7
Record Variables	11-24
Redefinition Table	5-8
Remote communication software	11-54
Replace battery message	5-10
Replace Cell Message	5-8
Replace Cells Message Table	5-10
Report Battery Cell Status Service	11-42
Request/Release Data Bus	11-46
Resident libraries, NVM	12-6
Resident library	12-14
Resident Library Builder	12-14, 12-15

Create the Install Program 12-19
 Creating resident library routines 12-15
 Generate the interface library and header files. 12-20
 Make the interface library. 12-22
 Making the library 12-18
 Read/write Data segment 12-19
 Running GENRES.EXE 12-17
 Write and compile the interrupt handler program. 12-19
 Resident program, calling a 12-9
 Resident program, writing a 12-11
 Resident Programs. 4-10
 Resident programs, memory models . 12-10
 Resident programs, NVM. 12-3, 12-5
 Response File 4-20
 Response File mode. 4-19
 Response file, sample 4-19

S

Sample Programs xiv, 6-13, 10-8, 12-4
 Sample programs 11-20, 11-36, 11-47
 ANSI3000. 8-20
 DOS File Management APIs. 6-16
 Keyboard. 9-37
 Memory management APIs 7-16
 Sample response file 4-19
 Scan codes 9-4
 scan codes 9-4
 SCAN3000.EXE. 10-6, 10-9, 10-14
 SCAN3500.EXE. 10-6, 10-11
 Scanner 3-8
 Scanner initialization file 10-88
 Scanner modification. 10-77, 10-89
 Scanning 10-12, 10-13
 Scanning error codes 10-10
 Scanning parameter descriptions 10-32
 Scanning Sequence Examples 10-31
 Scanning, methods of incorporating into applications 10-12
 Scanning, parameter menu. 10-39
 SCANPARM Utility. See SCANPARM.EXE.
 SCANPARM.EXE 10-77, 10-88
 Screen 8-6
 Backlighting. 8-18
 Contrast 8-18
 Multiple display pages. 8-7
 Physical. 8-4
 Viewing angle 8-18
 Virtual. 8-4
 Screen, logical 8-8
 scrolling 8-8
 Send End. 11-15
 Send Request Command 11-58
 Separate Command 11-57
 Separator Character Structure. 11-17
 Sequence of BIOS calls 11-35
 Sequence of IOCTL calls 11-32
 Sequential Access by Record Type. 6-6
 Shapes, cursor 8-16
 Simplex. 11-25
 Simplex Protocol 11-25
 single slot cradle 11-44
 Sleep Mode, Ways to Reactivate. 9-24
 Speaker 3-9
 Split-resident code files 12-5
 Split-resident programs 12-3, 12-5, 12-10, 12-11, 12-16
 System requirements 1-4
 Development PC. 1-4
 Hardware 1-4
 Software 1-4
 Terminal 1-4

T

TDREM. 2-5, 2-9
 Terminal Initialization Configuration Tool.
 See BLDINIT.EXE
 Terminal initialization parameters. 5-12
 Terminal Management System (TMS) 11-54
 Terminal resources. See Power management.
 text window 8-24



Timeout	9-23
TME (Extended Terminal Management System)	11-54
TMS (Terminal Management System)	11-54
Trailer Block	11-29
Translation table, 56 key scan code	9-6
Translation Table, Device Name	11-20
Translation table, keyboard	9-36
Two Way Protocol	11-27

U

UBASIC File Manage	6-4
UBASIC File Manager	
File management functions	6-8
File Manager data structures	6-10
Loading the File Manager	6-10
Record types	6-4
Storage methods	6-4
UBASIC KEYIN Approach	10-13
UBASIC WANDIN% Approach	10-12
Unattended	11-43
Unattended communications	11-43
Exclusive	11-43
Multi-Drop	11-44
Updating development PC files	
AUTOEXEC.BAT	1-7
CONFIG.SYS	1-7
URM API	11-12
URM Comm API	11-12, 11-22, 11-24
URM Communications API	11-16
URM communications functions	11-13
URM Programming	11-17
User Configuration Tool	4-3
Command line mode	4-14
File precedence	4-8
Prompt mode	4-12
Response File mode	4-17
User Configuration Tool (USRCFG.EXE)	5-7
User files	4-9
USRCFG	2-10, 4-12
USRCFG.EXE	4-3

USRCFG.EXE. See User Configuration Tool.

V

Variant Records	6-5
Viewing Angle	8-18
Virtual screen	8-4
Attributes	8-5
Displaying segments	8-6
Maximum size	8-5

W

Waking up the terminal	9-24
Get Last Wake Up Cause	9-25
Real time Clock Alarm wakeup ..	9-25
Wake up causes	9-26
Wakeup events	9-25
Wedge Approach	10-12
Windowing	8-14
Write Protection, Global	6-10

X

Xon/Xoff Protocol	11-26
-------------------------	-------

Tell Us What You Think...

We'd like to know what you think about this Manual. Please take a moment to fill out this questionnaire and fax this form to: (516) 738-3318, or mail to:

Symbol Technologies, Inc.
One Symbol Plaza M/S B-4
Holtsville, NY 11742-1300
Attn: Technical Publications Manager

IMPORTANT: If you need product support, please call the appropriate customer support number provided. Unfortunately, we cannot provide customer support at the fax number above.

User's Manual Title: _____
(please include revision level)

How familiar were you with this product before using this manual?

☐ Very familiar ☐ Slightly familiar ☐ Not at all familiar

Did this manual meet your needs? If not, please explain. _____

What topics need to be added to the index?, if applicable _____

What topics do you feel need to be better discussed? Please be specific. _____

What can we do to further improve our manuals? _____

Thank you for your input—We value your comments.

